

TechTribers

Java Interview Series · Part 16 · Q740–Q799

Generics · Reflection · Annotations

Bounds, inference, erasure and heap pollution shown with real javac diagnostics, reflection and method handles with measured costs, JEP 500 on JDK 26, dynamic proxies and hidden classes, and annotation processors plus four mini frameworks built from scratch.

60	60	21	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `GR-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**

A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**

What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**

The next question the interviewer is likely to ask, with the answer.

60	6	60	21
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

🧬	01	Generics I: Type Parameters, Bounds and Inference	Q740–Q751	5
🧪	02	Generics II: Erasure at Run Time and Advanced Patterns	Q752–Q763	23
🔬	03	Reflection I: Class Objects, Members and Access	Q764–Q772	40
🪄	04	Reflection II: Proxies, Method Handles and Dynamic Code	Q773–Q781	54
🏷️	05	Annotations I: Declaring, Meta-Annotations and Reading Them	Q782–Q791	68
🏭	06	Annotations II: Processing and Building Annotation-Driven Frameworks	Q792–Q799	84

© TechTribers. For personal use only. Do not redistribute.

All questions

🧬 Generics I: Type Parameters, Bounds and Inference

Q740 What is a raw type, why is it dangerous, and what does an "unchecked" warning really mean? 5

Q741 What is the difference between `List<?>`, `List<Object>` and the raw `List`? 6

Q742 How do bounded type parameters and multiple bounds work (`T extends Number & Comparable<T>`), and what does `T` erase to? 8

Q743 Why is `Collections.max` declared as `<T extends Object & Comparable<? super T>>`? 9

Q744 What are self-bounded generics (`Enum<E extends Enum<E>>`), and how do you build an inheritable fluent builder? 11

Q745 How does generic type inference work: diamond, target typing, type witnesses and `var` with diamond? 12

Q746 Why does `Comparator.comparing(p -> p.name()).reversed()` not compile, and how do you fix it? 13

Q747 What is wildcard capture, and why do you need a private helper to swap elements in a `List<?>`? 15

Q748 How do you apply PECS when designing real APIs, and why should return types never use wildcards? 16

Q749 Why can't a static method use the class's type parameter, and how do generic constructors and static factories work? 18

Q750 What can't you do with generics? (`new T()`, `new T[]`, `instanceof`, primitives, static `T`, `catch T`, class literals) 19

Q751 Why can't you create a generic array, and how do `ArrayList` and `toArray` work around it? 21

🧪 Generics II: Erasure at Run Time and Advanced Patterns

Q752 What are bridge methods, and when does the compiler generate them? 23

Q753 Why do overloads like `f(List<String>)` and `f(List<Integer>)` clash, and how does erasure affect overriding? 25

Q754 What is heap pollution, and what are the rules for `@SafeVarargs`? 26

Q755 How do you handle unchecked casts and `@SuppressWarnings("unchecked")` safely? 28

Q756 What are reifiable types, and when is `instanceof List<String>` allowed with pattern matching? 29

Q757 What generic type information survives erasure, and how do you read it at run time? 30

Q758 What is a super type token (Jackson `TypeReference`, Gson `TypeToken`), and what are its limits? 32

Q759 What is a type-safe heterogeneous container (`Class<T>` as a key)? 33

Q760 How does the generic singleton factory work (`Collections.emptyList()`, `Function.identity()`)? 35

Q761 Why can't a generic class extend `Throwable`, while `<X extends Throwable> void f() throws X` is legal? 36

Q762 What does boxing cost in generic collections, and what is the `contains(5L)` trap? 37

Q763 Coding: write a type-safe event bus with `Class<T>` keys and `Consumer<? super T>` handlers. 38

© TechTribers. For personal use only. Do not redistribute.

🔬 Reflection I: Class Objects, Members and Access

Q764 What are the ways to get a `Class` object, and what is special about primitives, arrays and `getClass()`'s return type? 40

Q765 What is the difference between `getMethods()` and `getDeclaredMethods()` (and fields, constructors), and how do you collect all fields up the hierarchy? 42

Q766 How do you create objects reflectively, why is `Class.newInstance()` deprecated, and what is `InvocationTargetException`? 43

Q767 How do you find and invoke overloaded, varargs, static and generic methods reflectively? 45

Q768 Can reflection read private fields and change `final` fields? (`setAccessible`, `trySetAccessible`, JEP 500) 46

Q769 How do you reflect on records, enums and sealed types? 48

Q770 What is the `java.lang.reflect.Type` hierarchy (`ParameterizedType`, `TypeVariable`, `WildcardType`, `GenericArrayType`)? 49

Q771 How do you create and manipulate arrays reflectively with `java.lang.reflect.Array`? 51

Q772 How slow is reflection really, and how should you cache reflective lookups? (measured, plus `ClassValue`) 52

🪄 Reflection II: Proxies, Method Handles and Dynamic Code

Q773 How do JDK dynamic proxies work (`Proxy`, `InvocationHandler`, `invokeDefault`)? 54

Q774 JDK proxies vs subclass proxies (CGLIB, ByteBuddy): why does self-invocation bypass `@Transactional`? 56

Q775 What are `MethodHandles`, and how do they differ from core reflection (`Lookup`, `invokeExact`, `invoke`, combinators)? 58

Q776 What is `LambdaMetafactory`, and how do frameworks turn reflective calls into fast lambdas? 59

Q777 What are hidden classes (JEP 371), and how do you define one at run time? 61

Q778 Why can't the same class loaded by two class loaders be cast to itself, and how do you compile and load code at run time? 62

Q779 What is `ServiceLoader` (SPI), and how do provider files and module `provides` / `uses` work? 64

Q780 When should you avoid reflection? (AOT and native images, refactoring safety, performance, encapsulation) 65

Q781 Coding: write a reflective object differ for audit logging ("what changed between two versions of an entity?"). 66

🏷️ Annotations I: Declaring, Meta-Annotations and Reading Them

Q782 How do you declare a custom annotation: element types, defaults, the `value` shorthand and the constant rules? 68

Q783 What do the built-in annotations `@Override`, `@FunctionalInterface`, `@Deprecated` and `@SuppressWarnings` actually enforce? 70

Q784 What is `@Retention` (`SOURCE`, `CLASS`, `RUNTIME`), and why can't reflection see most annotations? 71

Q785 What is `@Target`, and which `ElementType` values exist (including `TYPE_USE`, `TYPE_PARAMETER`, `RECORD_COMPONENT`, `MODULE`)? 73

Q786 How does `@Inherited` work, and what does it not inherit? 74

Q787 How do repeatable annotations work, and why does `getAnnotation` return `null` for repeated ones? 76

Q788 What is the difference between `getAnnotations`, `getDeclaredAnnotations` and `getAnnotationsByType` (present, directly present, associated)? 77

Q789 How are annotations represented at run time, and what are their `equals`, `hashCode` and `toString` rules? 79

Q790 What are type annotations, where do they attach (`@Nullable String[]` vs `String @Nullable []`), and what is JSpecify? 80

Q791 How do annotations on record components propagate to fields, accessors and constructor parameters? 82

🏭 Annotations II: Processing and Building Annotation-Driven Frameworks

Q792 What is annotation processing (JSR 269), and how do you write a processor that validates code and generates a class? 84

Q793 How does Lombok differ from a standard annotation processor, and what changed for processors in JDK 23? 87

Q794 How do composed annotations and meta-annotations work (Spring style), and how do you search for them correctly? 88

Q795 Coding: build a mini Bean Validation framework with runtime annotations. 90

Q796 Coding: build a mini dependency injection container with `@Inject` and `@Singleton`. 92

Q797 Coding: build a mini test runner like JUnit with `@Test`, `@BeforeEach` and expected exceptions. 94

Q798 Coding: build an annotation-driven request router with parameter annotations (like `@GetMapping` and `@PathVariable`). 96

Q799 Why does `Parameter.getName()` return `arg0`, and what does the `-parameters` compiler flag change? 98

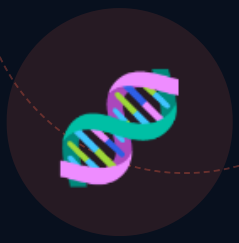
© TechTribers. For personal use only. Do not redistribute.

Section 01

Generics I: Type Parameters, Bounds and Inference

Raw types, wildcards versus Object, bounds and what they erase to, recursive and self-bounded types, how inference really works, wildcard capture, and the hard limits of Java generics, each shown with real javac diagnostics.

Questions 740–751 · 12 questions



Q740 GR-001

• Intermediate

What is a raw type, why is it dangerous, and what does an "unchecked" warning really mean?

⚡ Quick answer

A **raw type** is a generic type used without type arguments (`List` instead of `List<String>`). It exists only for compatibility with pre-Java-5 code. Using it switches off generic type checking for that expression: you can put an `Integer` into a `List<String>` through a raw alias, and the failure surfaces later as a `ClassCastException` at a **read** site far away. An **unchecked warning** is javac telling you it cannot prove type safety at this point; ignoring it is how heap pollution gets into a codebase.

Without `-Xlint` , javac only prints a one-line *note* ("uses unchecked or unsafe operations") for the whole file, which most builds ignore. That is why the bad write above compiles cleanly and the program only fails at `names.get(1)` , where the compiler inserted an invisible `checkcast String` .

What a raw type loses compared with the alternatives:

- `List` (raw): no checks at all. Any element in, `Object` out, unchecked warnings everywhere.
- `List<Object>` : type-safe, holds anything, but a `List<String>` is **not** assignable to it.
- `List<?>` : type-safe, accepts any `List<X>` , read as `Object` , cannot add (except `null`).

Legitimate uses of raw types today are almost nil: class literals (`List.class`), `instanceof List` , and a few JDK internals such as `Collections.swap` (which casts to a raw `List` to avoid a helper method).

Java

```java
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real
diagnostics (helper shown in full in Q750)
static int sumRaw(List raw) {                    // raw parameter: accepts any List
    int s = 0;
    for (Object o : raw) s += (Integer) o;
    return s;
}
```


© TechTribers. For personal use only. Do not redistribute.

```
List<String> names = new ArrayList<>(List.of("a"));
List raw = names;                                   // legal: raw List is a supertype of every
List<X>
raw.add(42);                                        // unchecked call: the List<String> now holds an
Integer
System.out.println("size=" + names.size() + ", element 1 = " + raw.get(1));
try { String s = names.get(1); }                    // the compiler inserted a cast to String here
catch (ClassCastException e) { System.out.println("CCE on READ: " + e.getMessage().split(" \\(")
[0]); }
String bad = "void f() { List raw = new ArrayList<String>(); raw.add(1); }";
System.out.println("default javac:      " + javac(bad));
System.out.println("-Xlint:unchecked:   " + javac(bad, "-Xlint:unchecked"));
System.out.println("-Xlint:all -Werror: " + javac(bad, "-Xlint:all", "-Werror"));
```

▶ Output

```
size=2, element 1 = 42
CCE on READ: class java.lang.Integer cannot be cast to class java.lang.String
default javac:      compiles
-Xlint:unchecked:   warning: unchecked call to add(E) as a member of the raw type java.util.List
-Xlint:all -Werror: warning: found raw type: java.util.List | warning: unchecked call to add(E) as a
member of the raw type java.util.List | error: warnings found and -Werror specified
```

⚠️ **Interview trap**

"Raw `List` and `List<Object>` are the same." They are not: `List<Object>` is still checked by the compiler; a raw `List` disables checking, and it also erases **every** generic signature on that object's methods, even ones unrelated to the missing type argument.

💡 **Pro tip**

Turn on `-Xlint:all -Werror` (Maven `compilerArgs`, Gradle `options.compilerArgs`) in new projects. It converts every unchecked warning into a build failure, and then each remaining `@SuppressWarnings("unchecked")` is a documented, reviewed decision.

? Q741 GR-002 • Intermediate

What is the difference between `List<?>`, `List<Object>` and the raw `List`?

⚡ **Quick answer**

`List<Object>` is a list that can hold anything, but only a `List<Object>` can be passed to it (generics are invariant). `List<?>` is a list of *some unknown* type: every `List<X>` is assignable to it, you read elements as `Object`, and you cannot add anything except `null`. Raw `List` accepts everything both ways and gives up type checking. Use `List<?>` for read-only "any list" parameters, `List<Object>` when you really mean a heterogeneous list.

The diagnostics show the core rule: **wildcards widen what you can accept, and narrow what you can do**. With `List<?>` the compiler only knows the element type is "capture of ?", so it refuses to put a `String` in; the list might be a `List<Integer>`. Operations that don't need the element type (`get` as `Object`, `remove(int)`, `clear`, `size`, adding `null`) all compile.

© TechTribers. For personal use only. Do not redistribute.

The three spellings also differ for `instanceof`: `x instanceof List<?>` is legal and checked; `x instanceof List<Object>` is a compile error, because the runtime can't check the type argument.

Java

```java
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real diagnostics (helper shown in full in Q750)
static int countNonNull(List<?> xs) {            // accepts List<String>, List<Integer>, ...
    int n = 0;
    for (Object x : xs) if (x != null) n++;
    return n;
}

System.out.println(countNonNull(List.of("a", "b")) + countNonNull(Arrays.asList(1, null, 3)));
String[] cases = {
    "void f(List<Object> l) { } void g() { f(new ArrayList<String>()); }",
    "void f(List<?> l) { } void g() { f(new ArrayList<String>()); }",
    "void f(List<?> l) { l.add(\"x\"); }",
    "void f(List<?> l) { l.add(null); Object o = l.get(0); l.remove(0); l.clear(); }",
    "void f(List<Object> l) { l.add(\"x\"); l.add(1); }",
    "void f(List l) { l.add(\"x\"); }"
};
for (String c : cases) System.out.println(c + "\n   -> " + javac(c, "-Xlint:unchecked"));
```

▶ Output

```
4
void f(List<Object> l) { } void g() { f(new ArrayList<String>()); }
   -> error: incompatible types: java.util.ArrayList<java.lang.String> cannot be converted to
java.util.List<java.lang.Object>
void f(List<?> l) { } void g() { f(new ArrayList<String>()); }
   -> compiles
void f(List<?> l) { l.add("x"); }
   -> error: incompatible types: java.lang.String cannot be converted to capture#1 of ?
void f(List<?> l) { l.add(null); Object o = l.get(0); l.remove(0); l.clear(); }
   -> compiles
void f(List<Object> l) { l.add("x"); l.add(1); }
   -> compiles
void f(List l) { l.add("x"); }
   -> warning: unchecked call to add(E) as a member of the raw type java.util.List
```

Which lists can you pass in, and what can you do with them?

Parameter type	accepts List<String>?	read element as	add("x")?	type-checked?
`List<Object>`	no (invariant)	Object	yes	yes
`List<?>`	yes	Object	only null	yes
`List<? extends T>`	if String <: T	T	only null	yes
`raw List`	yes	Object	yes (unchecked)	NO

Rule: wildcards widen what a method ACCEPTS and narrow what it can DO. Raw types switch the checks off.

© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> In API signatures, prefer `Collection<?>` over raw `Collection` for "accept any collection and only read or remove". It documents intent and keeps the caller's code warning-free.

> 🎯 **Likely follow-up**
> Can you add to a `List<? extends Number>`? Only `null`. The compiler doesn't know whether it's a `List<Integer>` or a `List<Double>`, so nothing else is provably safe (PECS: an `extends` list is a producer).

📏 **Q742** GR-003 • Intermediate

How do bounded type parameters and multiple bounds work (`T extends Number & Comparable<T>`), and what does `T` erase to?

> ⚡ **Quick answer**
> `<T extends B>` restricts `T` to subtypes of `B` and lets the body call `B`'s methods on a `T`. Multiple bounds use `&`: at most **one class**, and it must come **first**, followed by any number of interfaces. After compilation `T` is replaced by its **leftmost bound** (or `Object` if unbounded), and the compiler inserts casts to the other bounds where their methods are used, so the order of bounds changes the binary signature.

Two consequences interviewers like:

- **Erasure follows the leftmost bound.** `largest` erases to `Number largest(List)`, `firstOf` to `Comparable firstOf(List)`. Reordering bounds in a library is therefore a **binary-incompatible** change even though source still compiles.
- **Bounds unlock methods, not just restrictions.** Inside `largest` you can call both `doubleValue()` and `compareTo()` on a `T` because the compiler knows `T` is both.

A bounded type parameter is not the same as a wildcard bound: `<T extends Number> void f(List<T>)` gives the method a *name* for the element type (useful when the type appears twice, for example in the return type), while `void f(List<? extends Number>)` doesn't need one.

Java

```java
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real diagnostics (helper shown in full in Q750)
static <T extends Number & Comparable<T>> T largest(List<T> xs) {
    T best = xs.get(0);
    for (T x : xs) if (x.compareTo(best) > 0) best = x;      // Comparable method
    System.out.println("sum as double = " + xs.stream().mapToDouble(Number::doubleValue).sum());  // Number method
    return best;
}

static <T extends Comparable<T> & java.io.Serializable> T firstOf(List<T> xs) { return xs.get(0); }

System.out.println("largest = " + largest(List.of(3, 9, 4)));
```

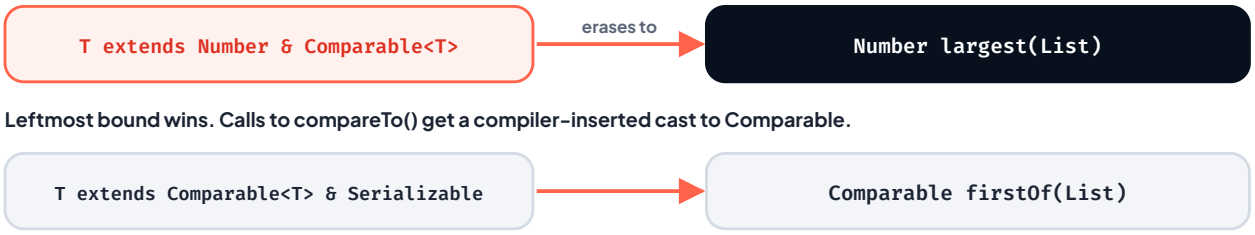

© TechTribers. For personal use only. Do not redistribute.

```
for (String name : List.of("largest", "firstOf")) {
    Method m = Main.class.getDeclaredMethod(name, List.class);
    System.out.println(name + ": erased return type = " + m.getReturnType().getSimpleName() + ",
declared bounds = " + Arrays.toString(m.getTypeParameters()[0].getBounds()));
}
System.out.println(javac("<T extends Comparable<T> & Number> void f() { }"));
System.out.println(javac("<T extends Number & Integer> void f() { }"));
```

▶ Output

```
sum as double = 16.0
largest = 9
largest: erased return type = Number, declared bounds = [class java.lang.Number,
java.lang.Comparable<T>]
firstOf: erased return type = Comparable, declared bounds = [java.lang.Comparable<T>, interface
java.io.Serializable]
error: interface expected here
error: interface expected here
```

<T extends Number & Comparable<T>> — one class first, then interfaces



Leftmost bound wins. Calls to compareTo() get a compiler-inserted cast to Comparable.

Reordering bounds changes the erased signature: source-compatible, NOT binary-compatible.

Collections.max uses <T extends Object & Comparable<? super T>> so it still erases to Object max(Collection).

⚠ **Interview trap**

`<T extends Number & Integer>` fails: after the first bound, only interfaces are allowed (and `Integer` is final anyway). The class bound must be first, which is why `Collections.max` is declared `T extends Object & Comparable<...>`, not the other way round.

Q743 GR-004 • Advanced

Why is `Collections.max` declared as `<T extends Object & Comparable<? super T>>`?

⚡ **Quick answer**

Two separate reasons. `Comparable<? super T>` lets `T` be a subclass whose `compareTo` is inherited from a superclass: `Apple extends Fruit implements Comparable<Fruit>` is not a `Comparable<Apple>`, so the naive bound `T extends Comparable<T>` rejects a `List<Apple>`. `Object &` makes the erasure of `T` `Object` instead of `Comparable`, so the erased signature stays `Object max(Collection)`, which is binary-compatible with the pre-Java-5 method.

© TechTribers. For personal use only. Do not redistribute.

The general rule this illustrates: when a type parameter appears as the argument of a **consumer** interface such as `Comparable` or `Comparator`, use `? super T`. `Comparable<T>` consumes a `T` in `compareTo(T)`, so an implementation written for any supertype of `T` works.

The `Object &` part is pure binary compatibility. Without it the erasure would be `Comparable max(Collection)`, and every class compiled against Java 1.4's `Object max(Collection)` would fail with `NoSuchMethodError`. You will rarely need this trick in your own code, but it explains a signature everybody has seen and few can decode.

Java

```
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real diagnostics (helper shown in full in Q750)
static class Fruit implements Comparable<Fruit> {
    final int weight; Fruit(int w) { weight = w; }
    public int compareTo(Fruit o) { return Integer.compare(weight, o.weight); }
    public String toString() { return getClass().getSimpleName() + "(" + weight + ")"; }
}
static class Apple extends Fruit { Apple(int w) { super(w); } }              // Apple is Comparable<Fruit>, not Comparable<Apple>

static <T extends Comparable<? super T>> T maxOf(Collection<? extends T> xs) {
    Iterator<? extends T> it = xs.iterator(); T best = it.next();
    while (it.hasNext()) { T x = it.next(); if (x.compareTo(best) > 0) best = x; }
    return best;
}

List<Apple> apples = List.of(new Apple(3), new Apple(7), new Apple(5));
System.out.println(maxOf(apples) + " " + Collections.max(apples));
String fruit = "static class Fruit implements Comparable<Fruit> { public int compareTo(Fruit o) { return 0; } } static class Apple extends Fruit { } ";
System.out.println("naive bound: " + javac(fruit + "<T extends Comparable<T>> T max(List<T> xs) { return null; } void f() { max(new ArrayList<Apple>()); }"));
Method m = Collections.class.getMethod("max", Collection.class);
System.out.println(m.toGenericString());
System.out.println("erased return type: " + m.getReturnType().getName());
```

▶ Output

```
Apple(7) Apple(7)
naive bound: error: method max in class T cannot be applied to given types;
public static <T extends java.lang.Object & java.lang.Comparable<? super T>> T
java.util.Collections.max(java.util.Collection<? extends T>)
erased return type: java.lang.Object
```

💡 **Pro tip**

Practical version for your own APIs: `static <T extends Comparable<? super T>> void sort(List<T> list)` is exactly `List.sort`'s natural-order contract. If you write a generic `max`, `min`, sorted collection or priority queue, copy this bound.

© TechTribers. For personal use only. Do not redistribute.

Q744 GR-005 • Advanced

What are self-bounded generics (`Enum<E extends Enum<E>>`), and how do you build an inheritable fluent builder?

⚡ **Quick answer**

A **recursive (self-bounded) type parameter** `class Base<T extends Base<T>>` lets the base class refer to "the concrete subclass" in its own signatures. `Enum<E extends Enum<E>>` uses it so `compareTo(E)` only accepts constants of the **same** enum. The same trick, plus an abstract `self()` method, gives a builder hierarchy where methods inherited from the parent builder still return the **child** builder type, so chaining never loses the subclass methods.

Without the self type, `add()` would return `Pizza.Builder`, and `new Calzone.Builder().add("ham").sauceInside()` would fail to compile because `sauceInside()` doesn't exist on the parent builder. Overriding every parent method with a covariant return type also works, but doesn't scale.

Why `self()` and not `(B) this`? The unchecked cast also works and Lombok's `@SuperBuilder` generates something similar, but `self()` keeps the code warning-free and lets each subclass prove its own type.

Limits worth stating: the pattern is a convention, not enforced. `class Rogue extends Pizza.Builder<Calzone.Builder>` compiles and would return the wrong object from `self()`. Java has no real `This` type; Kotlin and Scala don't either, they use the same trick.

Java

```
static abstract class Pizza {
    final List<String> toppings;
    Pizza(Builder<?> b) { toppings = List.copyOf(b.toppings); }
    abstract static class Builder<B extends Builder<B>> {             // B = the concrete builder
type
        final List<String> toppings = new ArrayList<>();
        B add(String t) { toppings.add(t); return self(); }            // returns the SUBCLASS type
        abstract Pizza build();
        protected abstract B self();
    }
}
static final class Calzone extends Pizza {
    final boolean sauceInside;
    Calzone(Builder b) { super(b); sauceInside = b.sauceInside; }
    static final class Builder extends Pizza.Builder<Builder> {
        boolean sauceInside;
        Builder sauceInside() { sauceInside = true; return this; }
        @Override Calzone build() { return new Calzone(this); }
        @Override protected Builder self() { return this; }
    }
}
enum Day { MON, TUE } enum Color { RED }

Calzone c = new Calzone.Builder().add("ham").sauceInside().add("cheese").build();   // add() returns
Calzone.Builder
System.out.println(c.toppings + " sauceInside=" + c.sauceInside);
TypeVariable<?> e = Enum.class.getTypeParameters()[0];
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println("Enum<" + e + " extends " + e.getBounds()[0].getTypeName() + ">");
System.out.println(Day.MON.compareTo(Day.TUE) + " (compareTo(Color.RED) would not compile)");
```

▶ Output

```
[ham, cheese] sauceInside=true
Enum<E extends java.lang.Enum<E>>
-1 (compareTo(Color.RED) would not compile)
```

🎯 Likely follow-up

Where else do you see it? `Comparable<T>` in sorted domain types, AssertJ's `AbstractAssert<SELF extends AbstractAssert<SELF, ACTUAL>, ACTUAL>` so that chained assertions keep the specific assert type, and Testcontainers' `GenericContainer<SELF extends GenericContainer<SELF>>`.

🔮 **Q745** GR-006 • Intermediate

How does generic type inference work: diamond, target typing, type witnesses and `var` with diamond?

⚡ Quick answer

javac infers type arguments from the **arguments** and, since Java 8, also from the **target type** (assignment, return, method parameter). The **diamond** `new HashMap<>()` infers constructor type arguments the same way, and since Java 9 works with anonymous classes. When inference can't see a target (in the middle of a call chain, for example), you supply an explicit **type witness**: `Stream.<Number>of(1, 2.0)`. Combining `var` **with a diamond** has no target, so it infers `Object`: `var list = new ArrayList<>()` is an `ArrayList<Object>`.

Inference happens **per call**, not across a fluent chain. In `Stream.of(1, 2.0).toList()`, `Stream.of` is inferred from its arguments alone (an intersection type `Number & Comparable<...>`), and `toList()` then has no say over it, so the result is not a `List<Number>`. The witness `Stream.<Number>of(...)` fixes the first call's type argument.

When to write a witness in real code:

- A generic call whose result is used as a receiver: `Collections.<String>emptyList().stream()`.
- Overloaded methods where inference picks a surprising one.
- Readability when the target type is far away.

The `var` rule is simple and catches people in code review: `var` takes the type of the initializer, and the initializer `new ArrayList<>()` has nothing to infer from, so you get `ArrayList<Object>`. Write `var names = new ArrayList<String>();` instead.

Java

```
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real
diagnostics (helper shown in full in Q750)
static void takes(List<String> l) { System.out.println("takes size " + l.size()); }

static void show(Object o) { System.out.println(o); }
```


© TechTribers. For personal use only. Do not redistribute.

```
Map<String, List<Integer>> m = new HashMap<>();                // diamond: inferred from the
assignment target
takes(Collections.emptyList());                                // Java 8: inferred from the
parameter type
Comparator<String> byLen = new Comparator<>() {                // diamond on an anonymous class
(Java 9+)
    public int compare(String a, String b) { return a.length() - b.length(); }
};
var list = new ArrayList<>();                                  // no target type: ArrayList<Object>
list.add(1); list.add("two");
show(list);
String[] cases = {
    "void f() { List<Number> n = java.util.stream.Stream.of(1, 2.0).toList(); }",
    "void f() { List<Number> n = java.util.stream.Stream.<Number>of(1, 2.0).toList(); }",
    "void f() { var l = new ArrayList<>(); l.add(1); String s = l.get(0); }"
};
for (String c : cases) System.out.println(c + "\n   -> " + javac(c));
```

▶ Output

```
takes size 0
[1, two]
void f() { List<Number> n = java.util.stream.Stream.of(1, 2.0).toList(); }
   -> error: incompatible types: java.util.List<java.lang.Number&java.lang.Comparable<? extends
java.lang.Number&java.lang.Comparable<?
>&java.lang.constant.Constable&java.lang.constant.ConstantDesc>&java.lang.constant.Constable&java.la
ng.constant.ConstantDesc> cannot be converted to java.util.List<java.lang.Number>
void f() { List<Number> n = java.util.stream.Stream.<Number>of(1, 2.0).toList(); }
   -> compiles
void f() { var l = new ArrayList<>(); l.add(1); String s = l.get(0); }
   -> error: incompatible types: java.lang.Object cannot be converted to java.lang.String
```

⚠️ **Interview trap**

Assuming the diamond is "just syntax sugar" that can't change behaviour. With an anonymous class, Java 9 only allows the diamond if the inferred type is **denotable**; an inferred intersection type is rejected with an error about a non-denotable type.

🔄 **Q746** `GR-007` • Intermediate

Why does `Comparator.comparing(p -> p.name()).reversed()` not compile, and how do you fix it?

⚡ **Quick answer**

Because inference doesn't flow backwards through a method chain. `.reversed()` is called **on** the result of `comparing(...)`, so `comparing` has no target type; its lambda parameter `p` is inferred as `Object`, and `p.name()` doesn't exist on `Object`. Fixes: a method reference (`comparing(Person::name)`), an explicitly typed lambda (`(Person p) -> p.name()`), or a type witness (`Comparator.<Person, String>comparing(...)`). Without the trailing `.reversed()` the plain lambda compiles, because then the target type is visible.

© TechTribers. For personal use only. Do not redistribute.

Note the last chain case: `thenComparing(p -> p.name())` **does** compile, because by then the receiver is already a `Comparator<Person>`, and `thenComparing`'s parameter is a `Function<? super Person, ...>`, which gives the lambda its type.

Why the error message mentions `Object`: the type variable `T` of `comparing` gets no constraint from anywhere (no target, a lambda without parameter types), so it is resolved to its bound, `Object`, and the lambda body is then checked against `Object`.

Java

```
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real
diagnostics (helper shown in full in Q750)
record Person(String name, int age) { }

String rec = "record Person(String name, int age) { } void f(List<Person> ps) { ";
String[] sorts = {
    "ps.sort(Comparator.comparing(p -> p.name()));",
    "ps.sort(Comparator.comparing(p -> p.name()).reversed());",
    "ps.sort(Comparator.comparing(Person::name).reversed());",
    "ps.sort(Comparator.comparing((Person p) -> p.name()).reversed());",
    "ps.sort(Comparator.<Person, String>comparing(p -> p.name()).reversed());",
    "ps.sort(Comparator.comparing(Person::age).thenComparing(p -> p.name()));"
};
for (String s : sorts) System.out.println(s + "\n   -> " + javac(rec + s + " }"));
List<Person> people = new ArrayList<>(List.of(new Person("Ravi", 31), new Person("Asha", 28), new
Person("Meera", 31)));
people.sort(Comparator.comparingInt(Person::age).reversed().thenComparing(Person::name));
System.out.println(people);
```

▶ Output

```
ps.sort(Comparator.comparing(p -> p.name()));
   -> compiles
ps.sort(Comparator.comparing(p -> p.name()).reversed());
   -> error: cannot find symbol
ps.sort(Comparator.comparing(Person::name).reversed());
   -> compiles
ps.sort(Comparator.comparing((Person p) -> p.name()).reversed());
   -> compiles
ps.sort(Comparator.<Person, String>comparing(p -> p.name()).reversed());
   -> compiles
ps.sort(Comparator.comparing(Person::age).thenComparing(p -> p.name()));
   -> compiles
[Person[name=Meera, age=31], Person[name=Ravi, age=31], Person[name=Asha, age=28]]
```

💡 **Pro tip**

Default to method references in comparator chains (`comparing(Person::age).thenComparing(Person::name)`). They carry their own type, so they survive `.reversed()` and long chains, and they read better.

© TechTribers. For personal use only. Do not redistribute.

🎯 **Likely follow-up**

Why does `comparingInt(Person::age).reversed()` reverse only the age order, while `.thenComparing(...).reversed()` reverses everything? `reversed()` wraps the whole comparator built so far. To reverse just one key use `comparing(Person::name, Comparator.reverseOrder())`.

Q747 GR-008 • Advanced

What is wildcard capture, and why do you need a private helper to swap elements in a `List<?>`?

⚡ **Quick answer**

Every time an expression of type `List<?>` is used, javac **captures** the wildcard as a fresh, unnamed type (`capture#1 of ?`, `capture#2 of ?` ...). A value read from the list is only known to be `Object`, and `set` demands the captured type, so `l.set(i, l.get(j))` is rejected even though both sides are the same list: the compiler doesn't track that. A **capture helper**, a private generic method `<T> void swapHelper(List<T> l, ...)`, gives the captured type a name, so reads and writes agree. The public API keeps the simpler `List<?>` signature.

The JDK takes a shortcut. `Collections.swap(List<?> list, int i, int j)` assigns the list to a **raw** `List` and suppresses the warning; the source has a comment saying the alternative is exactly a capture helper. Both are type-safe here because the method only moves elements that came from the same list.

The second compile shows each use being captured separately: two `set` calls, two errors, against `capture#1` and `capture#2`. Wildcard capture also explains why error messages say `capture#1 of ?` and why `List<?>` parameters can call `l.add(l.get(0))` in neither form. When you see a capture error, the fix is almost always: introduce a type parameter, either on the method itself or on a private helper.

Java

```
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real
diagnostics (helper shown in full in Q750)
public static void swap(List<?> list, int i, int j) { swapHelper(list, i, j); }     // public API:
any list

private static <T> void swapHelper(List<T> l, int i, int j) {                    // T names the
captured type
    T tmp = l.get(i);
    l.set(i, l.get(j));
    l.set(j, tmp);
}

List<String> xs = new ArrayList<>(List.of("a", "b", "c"));
swap(xs, 0, 2);
System.out.println(xs);
System.out.println(javac("void swap(List<?> l, int i, int j) { l.set(i, l.get(j)); }"));
System.out.println(javac("void swap(List<?> l, int i, int j) { Object t = l.get(i); l.set(i,
l.get(j)); l.set(j, t); }"));
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
[c, b, a]
error: incompatible types: java.lang.Object cannot be converted to capture#1 of ?
error: incompatible types: java.lang.Object cannot be converted to capture#1 of ? | error:
incompatible types: java.lang.Object cannot be converted to capture#2 of ?
```

⚠️ Interview trap

Changing the public signature to `<T> void swap(List<T> l, ...)` works too, and many teams do it. The interview point is knowing *why* the wildcard version fails and that the helper keeps the simpler public signature; Effective Java (Item 31) recommends the wildcard form for public APIs when a type parameter appears only once.

🔌 **Q748** GR-009 • Advanced

How do you apply PECS when designing real APIs, and why should return types never use wildcards?

⚡ Quick answer

Put `? extends T` on parameters you only **read from** (producers) and `? super T` on parameters you only **write to** or that **consume** a `T` (consumers, including `Comparator<? super T>`, `Consumer<? super T>`, `Predicate<? super T>`). Then your method accepts the widest range of arguments. Keep **return types** concrete (`List<T>`, not `List<? extends T>`); a wildcard return forces every caller to deal with an unknown type and they can't add to it.

Look at the JDK for the canonical patterns: `Collections.copy(List<? super T> dest, List<? extends T> src)`, `Stream.map(Function<? super T, ? extends R>)`, `Collection.addAll(Collection<? extends E>)`, `List.sort(Comparator<? super E>)`, `Map.computeIfAbsent(K, Function<? super K, ? extends V>)`.

Rules of thumb that hold up in review:

- A type that appears **once** in a parameter list can usually be a wildcard; a type that links two parameters, or a parameter and the return type, needs a named type parameter.
- **Functional-interface parameters** almost always need wildcards: `Function<? super T, ? extends R>`. Without them a caller holding a `Function<Object, String>` can't pass it.
- **Don't** add wildcards to return types or fields. They leak complexity to every caller and make the result read-only.
- If a parameter is both producer and consumer (you read and write the same list), use exact `List<T>`.

Java

```
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real
diagnostics (helper shown in full in Q750)
static <T> void copyInto(Collection<? super T> dst, Iterable<? extends T> src) { for (T t : src)
dst.add(t); }

static <T> Optional<T> maxBy(Collection<? extends T> xs, Comparator<? super T> cmp) {
    T best = null;
```


© TechTribers. For personal use only. Do not redistribute.

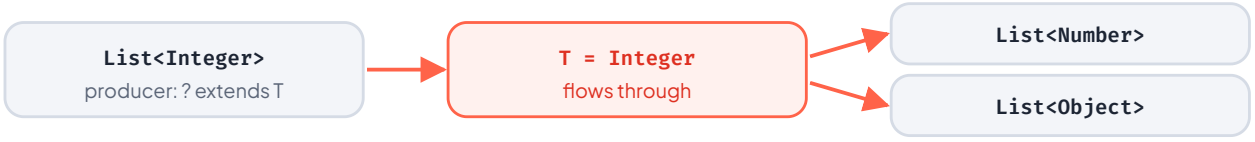
```
    for (T x : xs) if (best == null || cmp.compare(x, best) > 0) best = x;
    return Optional.ofNullable(best);
}

List<Integer> ints = List.of(4, 9, 2);
List<Number> numbers = new ArrayList<>(); List<Object> objects = new ArrayList<>();
copyInto(numbers, ints); copyInto(objects, ints);                    // T = Integer, two
different consumers
Comparator<Object> byText = Comparator.comparing(Object::toString);       // a comparator for a
SUPERTYPE
System.out.println(numbers + " " + objects + " " + maxBy(ints, byText).orElseThrow());
System.out.println(Stream.class.getMethod("map",
Function.class).toGenericString().replace("java.util.function.", "").replace("java.util.stream.",
""));
System.out.println(javac("List<? extends Number> nums() { return new ArrayList<Integer>(); } void
caller() { nums().add(1); }"));
```

▶ Output

```
[4, 9, 2] [4, 9, 2] 9
public abstract <R> Stream<R> Stream.map(Function<? super T, ? extends R>)
error: incompatible types: int cannot be converted to capture#1 of ? extends java.lang.Number
```

copyInto(Collection<? super T> dst, Iterable<? extends T> src) with T = Integer



read from it	`? extends T`	`Iterable<? extends T>, Collection.addAll(Collection<? extends E>)`
write to it / it consumes T	`? super T`	`Comparator<? super T>, Consumer<? super T>, Predicate<? super T>`
both read and write	`exact T`	`List<T> (e.g. an in-place sort or swap helper)`
return type, field	`no wildcard`	`List<T>: wildcards here leak complexity to every caller`

⚠️ **Interview trap**

Writing `Comparator<T>` in a sort helper. It compiles and works in your tests, then fails for the first caller who passes `Comparator<Object>` or a comparator for a superclass. The JDK uses `Comparator<? super T>` everywhere for this reason.

© TechTribers. For personal use only. Do not redistribute.

Q749 GR-010 • Intermediate

Why can't a static method use the class's type parameter, and how do generic constructors and static factories work?

⚡ Quick answer

A class type parameter belongs to each **instance's** parameterization (`Box<String>` , `Box<Integer>`), but there is only **one** `Box.class` and one set of statics shared by all of them, so `static T` makes no sense and is a compile error. Static code that needs a type declares its **own** type parameter: `static <T> Box<T> of(T v)` . Constructors can also declare their own type parameters, independent of the class's, which is useful for converting inputs.

The static factory is usually the better API: it can have a meaningful name (`of` , `copyOf` , `empty`), cache instances, return a subtype, and before Java 7 it was the only way to get type inference. Most of the JDK's modern collection API (`List.of` , `Map.entry` , `Optional.of`) is static generic factories.

Generic constructors are rare but real: `new HashSet<>(Collection<? extends E>)` uses a wildcard, while a constructor with its own `<S>` is useful when the input type must be named twice, as in `Box(S raw, Function<? super S, ...> parse)` . The explicit-witness syntax `new <StringBuilder>Box<Integer>(...)` is legal but almost never needed.

Java

```
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real diagnostics (helper shown in full in Q750)
static final class Box<T> {
    private final T value;
    private static int created;                                  // ONE counter shared by all Box<X>
    private Box(T v) { value = v; created++; }
    static <T> Box<T> of(T v) { return new Box<>(v); }           // the method's own T, not the class's
    <S extends CharSequence> Box(S raw, Function<? super S, ? extends T> parse) { this(parse.apply(raw)); }   // generic constructor
    T get() { return value; }
}

Box<String> s = Box.of("java");
Box<Integer> n = new Box<>("2026", Integer::parseInt);           // constructor's S inferred as String
Box<Integer> w = new <StringBuilder>Box<Integer>(new StringBuilder("7"), sb -> sb.length() * 6);
// explicit witnesses
System.out.println(s.get() + " " + (n.get() + 1) + " " + w.get() + " created=" + Box.created);
System.out.println(javac("static class B<T> { static T make() { return null; } }"));
System.out.println(javac("static class B<T> { static List<T> cache; }"));
```

▶ Output

```
java 2027 6 created=3
error: non-static type variable T cannot be referenced from a static context
error: non-static type variable T cannot be referenced from a static context
```


© TechTribers. For personal use only. Do not redistribute.

🎯 **Likely follow-up**

Is a static generic method's `T` related to the class's `T` when they share the name? No. In `Box.of`, `T` shadows the class's `T`; they are independent variables. Many style guides use a different letter (`<U>`) to avoid confusion.

Q750 GR-011 • Intermediate

What can't you do with generics? (`new T()`, `new T[]`, `instanceof`, primitives, static `T`, `catch T`, class literals)

⚡ **Quick answer**

Because of **erasure**, a type parameter doesn't exist at run time, so anything that needs the actual type at run time is illegal: `new T()`, `new T[n]`, `x instanceof T` or `instanceof List<String>`, `T.class` and `List<String>.class`, `catch (T e)`, generic subclasses of `Throwable`, and static fields or methods using the class's `T`. Primitives can't be type arguments (`List<int>`). Two overloads with the same erasure clash. The standard workaround is to pass a `Class<T>`, a `Supplier<T>` or an `IntFunction<T[]>`.

This snippet shows the helper used throughout this part: it compiles a string with the JDK's own compiler (`javax.tools.ToolProvider.getSystemJavaCompiler()`) and prints the real diagnostics, so every message here is what `javac` 21 prints.

The workaround for each restriction:

- `new T()`: take a `Supplier<T>` (`ArrayList::new`) or a `Class<T>` and call `getDeclaredConstructor().newInstance()`.
- `new T[n]`: take an `IntFunction<T[]>` (`String[]::new`), the pattern of `Collection.toArray(IntFunction)`, or `Array.newInstance(Class<T>, n)`.
- `instanceof T`: use `Class<T>.isInstance(o)` and `Class<T>.cast(o)`.
- `List<int>`: box (`List<Integer>`), or use primitive-specialised types (`IntStream`, `int[]`, Eclipse Collections' `IntList`). Project Valhalla aims to lift this one.
- Overload clash: give the methods different names (`sumInts`, `sumLongs`).

Java

```
static String javac(String body, String... opts) {                // compile a class body in
memory; report javac errors and warnings
    javax.tools.JavaCompiler jc = javax.tools.ToolProvider.getSystemJavaCompiler();
    javax.tools.DiagnosticCollector<javax.tools.JavaFileObject> dc = new
javax.tools.DiagnosticCollector<>();
    javax.tools.JavaFileObject src = new
javax.tools.SimpleJavaFileObject(java.net.URI.create("string:///T.java"),
javax.tools.JavaFileObject.Kind.SOURCE) {
        @Override public CharSequence getCharContent(boolean ignore) {
            return "import java.util.*; import java.util.function.*; import java.io.*; import
java.lang.annotation.*; public class T { " + body + " }";
        }
    };
    List<String> o = new ArrayList<>(List.of("-proc:none", "-d",
```


© TechTribers. For personal use only. Do not redistribute.

```
System.getProperty("java.io.tmpdir"))); o.addAll(List.of(opts));
    boolean ok = jc.getTask(null, jc.getStandardFileManager(dc, null, null), dc, o, null,
List.of(src)).call();
    StringJoiner sj = new StringJoiner(" | ");
    for (javax.tools.Diagnostic<? extends javax.tools.JavaFileObject> d : dc.getDiagnostics())
        if (d.getKind() != javax.tools.Diagnostic.Kind.NOTE)
            sj.add((d.getKind() == javax.tools.Diagnostic.Kind.ERROR ? "error: " : "warning: ") +
d.getMessage(Locale.ENGLISH).split("\\n")[0]);
    return ok && sj.length() == 0 ? "compiles" : sj.toString();
}
String[] cases = {
    "<T> T make() { return new T(); }",
    "<T> T[] arr(int n) { return new T[n]; }",
    "<T> boolean is(Object o) { return o instanceof T; }",
    "boolean is(Object o) { return o instanceof List<String>; }",
    "List<int> nums;",
    "static class B<T> { static T shared; }",
    "<T extends Exception> void f() { try { } catch (T e) { } }",
    "static class Oops<T> extends Exception { }",
    "Object c = List<String>.class;",
    "void f(List<String> a) { } void f(List<Integer> a) { }"
};
for (String c : cases) System.out.printf("%-60s -> %s%n", c, javac(c));
```

▶ Output

```
<T> T make() { return new T(); }                            -> error: unexpected type
<T> T[] arr(int n) { return new T[n]; }                     -> error: generic array creation
<T> boolean is(Object o) { return o instanceof T; }         -> error: java.lang.Object cannot be
safely cast to T
boolean is(Object o) { return o instanceof List<String>; }  -> error: java.lang.Object cannot be
safely cast to java.util.List<java.lang.String>
List<int> nums;                                             -> error: unexpected type
static class B<T> { static T shared; }                      -> error: non-static type variable T
cannot be referenced from a static context
<T extends Exception> void f() { try { } catch (T e) { } }  -> error: unexpected type
static class Oops<T> extends Exception { }                  -> error: a generic class may not
extend java.lang.Throwable
Object c = List<String>.class;                              -> error: <identifier> expected |
error: <identifier> expected
void f(List<String> a) { } void f(List<Integer> a) { }      -> error: name clash:
f(java.util.List<java.lang.Integer>) and f(java.util.List<java.lang.String>) have the same erasure
```


© TechTribers. For personal use only. Do not redistribute.

Erasure means T does not exist at run time — so these are illegal, and here is the fix

✗ `new T()`	→ `Supplier<T>` / `Class<T>.getDeclaredConstructor().newInstance()`
✗ `new T[n]`	→ `IntFunction<T[]> (String[]::new)` / `Array.newInstance(type, n)`
✗ `o instanceof T`	→ `Class<T>.isInstance(o)`, `Class<T>.cast(o)`
✗ `List<int>`	→ `List<Integer>`, `IntStream`, `int[]`, primitive collections
✗ `static T field`	→ a static generic method `<U>` with its own type parameter
✗ `catch (T e)` / `Ex<T> extends Exception`	→ catch concrete types; `<X extends Throwable> ... throws X` is fine
✗ `List<String>.class`	→ `List.class`, or a super type token (`TypeReference<List<String>>`)
✗ `f(List<String>) + f(List<Integer>)`	→ different method names (same erasure clashes)

⚠️ **Interview trap**

`instanceof List<String>` is **not** always illegal since Java 16: it compiles when the static type already guarantees the type argument, for example on a `Collection<String>` variable. It is illegal only when the check would be unchecked (see Q756).

Q751 GR-012 • Advanced

Why can't you create a generic array, and how do `ArrayList` and `toArray` work around it?

⚡ **Quick answer**

Arrays are **reified and covariant**: an array knows its element type at run time and checks every store (`ArrayStoreException`). Generics are **erased and invariant**. A `new T[n]` or `new List<String>[n]` would create an array whose run-time check is weaker than its compile-time type, breaking the guarantee, so javac forbids it. Collections store elements in an `Object[]` and cast on read (`ArrayList` does exactly this). To hand out a correctly typed array, the caller supplies the type: `toArray(String[]::new)` or `Array.newInstance(String.class, n)`.

The `Stack` shows the two rules for the `(E[]) new Object[n]` idiom: it is safe **only while the array stays private**. `pop()` returns an element, and the compiler-inserted cast on that element is correct. `leak()` returns the array itself; the caller's `String[] a = ...` gets a `checkcast String[]` on an `Object[]` and fails.

Why the language bans `new List<String>[10]` (classic example from the JLS and Effective Java): if it were allowed, `Object[] objs = lists; objs[0] = List.of(42);` would pass the array store check (the runtime only sees `List`), and the `Integer` would come out of `lists[0].get(0)` as a `String`. Arrays of **unbounded wildcard** types (`List<?>[]`) are allowed because they are reifiable; nothing unchecked can happen.

Java

```
static final class Stack<E> {
    private E[] items; private int size;
    @SuppressWarnings("unchecked") Stack(int cap) { items = (E[]) new Object[cap]; }   // really an
Object[]
```


© TechTribers. For personal use only. Do not redistribute.

```
    void push(E e) { items[size++] = e; }
    E pop() { return items[--size]; }                    // fine: the unchecked cast never
escapes as E[]
    E[] leak() { return items; }                         // BUG: exposes the Object[] typed as
E[]
}

@SuppressWarnings("unchecked")
static <T> T[] newArray(Class<T> type, int n) { return (T[])
java.lang.reflect.Array.newInstance(type, n); }

Stack<String> s = new Stack<>(4); s.push("a"); s.push("b");
System.out.println("pop: " + s.pop());
try { String[] a = s.leak(); } catch (ClassCastException e) { System.out.println("leak: " +
e.getMessage().split(" \\(")[0]); }
String[] typed = newArray(String.class, 2);
System.out.println("reflective array: " + typed.getClass().getSimpleName());
System.out.println("toArray: " + List.of("x", "y").toArray(String[]::new).getClass().getSimpleName()
+ " vs " + List.of("x").toArray().getClass().getSimpleName());
List<?>[] wild = new List<?>[2]; wild[0] = List.of(1); wild[1] = List.of("s");      // allowed:
List<?> is reifiable
System.out.println("List<?>[] ok: " + Arrays.toString(wild));
```

▶ Output

```
pop: b
leak: class [Ljava.lang.Object; cannot be cast to class [Ljava.lang.String;
reflective array: String[]
toArray: String[] vs Object[]
List<?>[] ok: [[1], [s]]
```

⚠️ **Interview trap**

`Arrays.asList(1, 2).toArray()` returned `Integer[]` before Java 9 and `Object[]` since JDK-6260652 was fixed in 9. Code that cast the result to `Integer[]` broke on upgrade.

💡 **Pro tip**

When you need a typed array from a collection, always use `toArray(String[]::new)` (Java 11). `toArray()` returns `Object[]`, and `toArray(new String[0])` was the older idiom; it is still as fast as pre-sizing on modern JVMs.

© TechTribers. For personal use only. Do not redistribute.

Section 02

Generics II: Erasure at Run Time and Advanced Patterns

Bridge methods, erasure clashes, heap pollution and @SafeVarargs, safe unchecked casts, what generic information survives in the class file, super type tokens, heterogeneous containers, generic exceptions, the real cost of boxing, and a type-safe event bus.

Questions 752–763 · 12 questions



Q752 GR-013 • Advanced

What are bridge methods, and when does the compiler generate them?

⚡ Quick answer

A **bridge method** is a synthetic method javac adds so that erasure doesn't break overriding. `class Fruit implements Comparable<Fruit>` declares `compareTo(Fruit)`, but after erasure the interface method is `compareTo(Object)`; javac adds a bridge `compareTo(Object)` that casts its argument to `Fruit` and calls the real method. Bridges are also generated for **covariant return types**. They show up in reflection (`isBridge()`, `isSynthetic()`), in stack traces, and explain the `ClassCastException` you get when calling through a raw type.

What the bridge for `Fruit` looks like in bytecode (`javap -c`): `public int compareTo(java.lang.Object)` with `ACC_BRIDGE, ACC_SYNTHETIC`, whose body is `checkcast Fruit; invokevirtual compareTo(Fruit)`. That `checkcast` is the `ClassCastException` above.

Where bridges matter in practice:

- **Reflection-based frameworks** must skip bridges, or they see each method twice (Spring's `BridgeMethodResolver`, Jackson's introspection).
- **Annotations are copied onto bridges** (javac does this since JDK 8, as the `@Deprecated` column shows), so scanning both the bridge and the real method can process an annotation twice.
- **Stack traces** of generic code sometimes show a frame for the bridge with the same method name and a line number of the class declaration.

Java

```
static class Fruit implements Comparable<Fruit> {
    @Deprecated public int compareTo(Fruit o) { return 0; }
}
static class Node { Object value() { return "node"; } }
static class TextNode extends Node { @Override String value() { return "text"; } }        //
covariant return
```


© TechTribers. For personal use only. Do not redistribute.

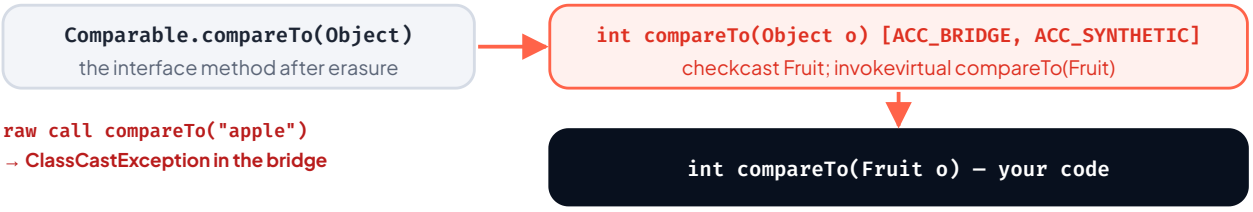
```
static void dump(Class<?> c) {
    Arrays.stream(c.getDeclaredMethods()).sorted(Comparator.comparing(Method::toString))
          .forEach(m -> System.out.println("  " + m.getReturnType().getSimpleName() + " " +
m.getName() + "(" + (m.getParameterCount() == 0 ? "" : m.getParameterTypes()[0].getSimpleName())
                  + ") bridge=" + m.isBridge() + " synthetic=" + m.isSynthetic() + " @Deprecated
copied=" + m.isAnnotationPresent(Deprecated.class)));
}

System.out.println("Fruit:"); dump(Fruit.class);
System.out.println("TextNode:"); dump(TextNode.class);
Comparable raw = new Fruit();
try { raw.compareTo("apple"); }                              // goes through the bridge, which
casts to Fruit
catch (ClassCastException e) { System.out.println("raw call: " + e.getMessage().split(" \\(")[0]); }
```

▶ Output

```
Fruit:
  int compareTo(Fruit) bridge=false synthetic=false @Deprecated copied=true
  int compareTo(Object) bridge=true synthetic=true @Deprecated copied=true
TextNode:
  Object value() bridge=true synthetic=true @Deprecated copied=false
  String value() bridge=false synthetic=false @Deprecated copied=false
raw call: class java.lang.String cannot be cast to class Main$Fruit
```

class Fruit implements Comparable<Fruit> — what javac actually emits



Also generated for covariant returns: String value() overriding Object value() gets a bridge Object value().

Bridges carry copies of the method's annotations (javac, JDK 8+): frameworks must skip isBridge() methods.

⚠️ Interview trap

"Bridges are a JVM feature." They are not; the JVM knows nothing about generics. Bridges are ordinary methods generated by javac, flagged `ACC_BRIDGE` so tools can recognise them.

🎯 Likely follow-up

Why does `getDeclaredMethods()` need sorting in the snippet? Its order is unspecified, and it does change between JDK builds. Never depend on reflection's member order.

© TechTribers. For personal use only. Do not redistribute.

Q753 GR-014 • Intermediate

Why do overloads like `f(List<String>)` and `f(List<Integer>)` clash, and how does erasure affect overriding?

⚡ **Quick answer**

Two methods whose parameter types have the **same erasure** would get the same JVM descriptor (`f(Ljava/util/List;)V`), so javac rejects them as a **name clash**, even though their generic signatures differ. The same rule applies across a hierarchy: a subclass method `f(List<Integer>)` neither overrides nor overloads an inherited `f(List<String>)` , so it's an error. Overriding works on erasure in one direction only: a **raw** `f(List)` *is* allowed to override `f(List<String>)` (for migration), with an unchecked warning.

Read the cases in pairs:

- `<T> void g(T)` and `g(Object)` clash because `T` erases to `Object` . With `<T extends Number>` the erasure becomes `Number` , so it clashes with `g(Number)` but legally overloads `g(Object)` .
- In a subclass, `f(List<Integer>)` is an error, not an overload: same erasure, different generic signature, and "neither overrides the other".
- A raw `f(List)` overrides `f(List<String>)` . This exists so that a library could be generified while old subclasses kept compiling.
- `f(Collection<String>)` doesn't override `f(List<String>)` at all; `@Override` turns that silent overload into an error. This is exactly what `@Override` is for.

The usual fix for the overload clash is different method names, or a single method taking `List<? extends Number>` .

Java

```
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real
diagnostics (helper shown in full in Q750)
String base = "static class Base { void f(List<String> l) { } } ";
String[] cases = {
    "void f(List<String> a) { } void f(List<Integer> a) { }",
    "<T> void g(T t) { } void g(Object o) { }",
    "<T extends Number> void g(T t) { } void g(Number n) { }",
    "<T extends Number> void g(T t) { } void g(Object o) { }",
    base + "static class Sub extends Base { void f(List<Integer> l) { } }",
    base + "static class Sub extends Base { @Override void f(List l) { } }",
    base + "static class Sub extends Base { @Override void f(Collection<String> l) { } }"
};
for (String c : cases) System.out.println(c.replace(base, "Base{f(List<String>)} + ") + "\n   -> " +
javac(c, "-Xlint:unchecked"));
```

▶ **Output**

```
void f(List<String> a) { } void f(List<Integer> a) { }
   -> error: name clash: f(java.util.List<java.lang.Integer>) and
f(java.util.List<java.lang.String>) have the same erasure
<T> void g(T t) { } void g(Object o) { }
   -> error: name clash: g(java.lang.Object) and <T>g(T) have the same erasure
<T extends Number> void g(T t) { } void g(Number n) { }
```


© TechTribers. For personal use only. Do not redistribute.

```
   -> error: name clash: g(java.lang.Number) and <T>g(T) have the same erasure
<T extends Number> void g(T t) { } void g(Object o) { }
   -> compiles
Base{f(List<String>)} + static class Sub extends Base { void f(List<Integer> l) { } }
   -> error: name clash: f(java.util.List<java.lang.Integer>) in T.Sub and
f(java.util.List<java.lang.String>) in T.Base have the same erasure, yet neither overrides the other
Base{f(List<String>)} + static class Sub extends Base { @Override void f(List l) { } }
   -> compiles
Base{f(List<String>)} + static class Sub extends Base { @Override void f(Collection<String> l) { } }
   -> error: method does not override or implement a method from a supertype
```

> 💡 **Pro tip**
> Overloads that differ only by generic or functional-interface parameter types are a design smell even when they compile: `submit(Callable<T>)` vs `submit(Runnable)` in `ExecutorService` is the classic source of "ambiguous method call" errors with lambdas.

☣ **Q754** GR-015 • Advanced

What is heap pollution, and what are the rules for `@SafeVarargs`?

> ⚡ **Quick answer**
> **Heap pollution** is when a variable of a parameterized type refers to an object that isn't of that type, for example a `List<String>` that contains an `Integer`. It's only possible through unchecked operations, and the failure appears later as a `ClassCastException` in code that has no cast. **Generic varargs** are a built-in source: a `T...` or `List<String>...` parameter is really an array of the erased type. `@SafeVarargs` is a promise that the method doesn't store into or leak that array; it's allowed only on `static`, `final` and (since Java 9) `private` methods and constructors, which can't be overridden.

Why `pickTwo` fails although `toArray("a", "b")` called directly would work: inside `pickTwo`, `T` is erased, so the compiler creates the varargs array as `new Object[]{a, b}`. Returning it typed as `T[]` and assigning it to `String[]` fails. The bug is in `toArray`, which returned its varargs array; the crash happens two calls away.

A method is safe to annotate when it:

- never **writes** into the varargs array, and
- never lets a reference to the array **escape** (return it, store it in a field, pass it to code that might do either).

Passing the array to another `@SafeVarargs` method or reading its elements is fine. `List.of(E...)`, `Arrays.asList(T...)`, `Stream.of(T...)` and `EnumSet.of(E, E...)` are all `@SafeVarargs` in the JDK.

Java

```java
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real
diagnostics (helper shown in full in Q750)
static <T> T[] toArray(T... xs) { return xs; }                // leaks the varargs array

static <T> T[] pickTwo(T a, T b) { return toArray(a, b); }       // here T[] is created as Object[]

static void pollute(List<String>... lists) {                     // stores into the varargs array
```


© TechTribers. For personal use only. Do not redistribute.

```
    Object[] arr = lists;
    arr[0] = List.of(42);                                    // no ArrayStoreException: runtime
type is List[]
}

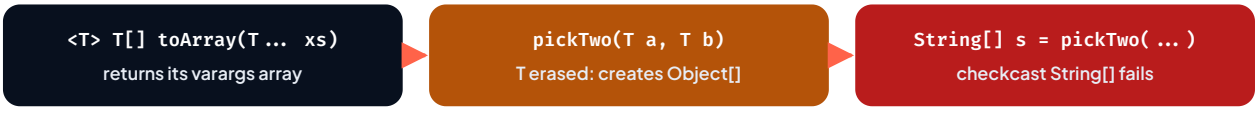
@SafeVarargs static <T> List<T> concat(List<? extends T>... lists) {   // safe: only reads the
elements
    List<T> out = new ArrayList<>();
    for (List<? extends T> l : lists) out.addAll(l);
    return out;
}

try { String[] s = pickTwo("a", "b"); } catch (ClassCastException e) { System.out.println("pickTwo:
" + e.getMessage().split(" \\(")[0]); }
List<String> strs = new ArrayList<>(List.of("x"));
List<String>[] holder = new List[]{strs}; pollute(holder);
try { String s = holder[0].get(0); } catch (ClassCastException e) { System.out.println("pollute: " +
e.getMessage().split(" \\(")[0]); }
System.out.println("concat: " + concat(List.of(1, 2), List.of(3.5)));
System.out.println(javac("<T> void f(T... xs) { }", "-Xlint:unchecked"));
System.out.println(javac("@SafeVarargs <T> void f(T... xs) { }"));
System.out.println(javac("@SafeVarargs private <T> void f(T... xs) { }"));
```

▶ Output

```
pickTwo: class [Ljava.lang.Object; cannot be cast to class [Ljava.lang.String;
pollute: class java.lang.Integer cannot be cast to class java.lang.String
concat: [1, 2, 3.5]
warning: Possible heap pollution from parameterized vararg type T
error: Invalid java.lang.SafeVarargs annotation. Instance method <T>f(T...) is neither final nor
private.
compiles
```

Heap pollution: the bad write and the crash are in different places



@SafeVarargs is a promise:

✓ only READ the varargs array ✗ never write into it ✗ never return or store it

Allowed on: static, final and private methods (Java 9+) and constructors — they cannot be overridden.

Silences "unchecked generic array creation" at EVERY call site; @SuppressWarnings only silences your method.

⚠️ **Interview trap**

Adding `@SuppressWarnings("unchecked")` instead of `@SafeVarargs` on a generic varargs method. It silences the warning in **your** method only; every caller still gets "unchecked generic array creation". `@SafeVarargs` suppresses it at all call sites, which is why the JDK uses it.

© TechTribers. For personal use only. Do not redistribute.

🎯 **Likely follow-up**

Why can't `@SafeVarargs` go on a normal instance method? An override could break the promise, and the annotation is not inherited. That's why it's restricted to methods that can't be overridden.

🚦 **Q755** GR-016 • Intermediate

How do you handle unchecked casts and `@SuppressWarnings("unchecked")` safely?

⚡ **Quick answer**

An unchecked cast `(T) obj` performs **no check** at the cast; the real check happens later, wherever the value is used as a concrete type. Prefer a **checked** cast through a type token, `type.cast(obj)`, which fails immediately and names the problem. When an unchecked cast is genuinely safe, put `@SuppressWarnings("unchecked")` on the **narrowest** element (a single local variable declaration, not the method or class) and add a comment proving why it's safe. To find pollution early in legacy code, wrap collections with `Collections.checkedList/Set/Map`.

The first call shows how quietly an unchecked cast misbehaves: assigned to an `Object`, the wrong value flows on with no error at all. Only when a caller binds `T` to `Integer` does the compiler insert a `checkcast Integer`, and the exception points at the caller, not at the bad data.

A checklist for code review:

- Can the method take a `Class<T>` (or a type token) instead? Then use `type.cast`, or `type.isInstance` with a fallback.
- If not, is the cast provably safe? Typical safe cases: you just created the object (`(E[]) new Object[n]` kept private), the value came from a structure you control with the same key type, or a singleton that holds no `T` (see Q760).
- Suppress on a variable declaration only, so later edits to the method don't hide new warnings.

Java

```
static <T> T uncheckedGet(Map<String, Object> m, String key) {
    @SuppressWarnings("unchecked") T t = (T) m.get(key);     // no check here at all
    return t;
}

static <T> T checkedGet(Map<String, Object> m, String key, Class<T> type) { return
type.cast(m.get(key)); }

Map<String, Object> config = Map.of("port", "8080");
Object viaUnchecked = uncheckedGet(config, "port");                    // T = Object: nothing
fails yet
System.out.println("unchecked returned: " + viaUnchecked);
try { Integer port = uncheckedGet(config, "port"); }                   // the cast happens HERE,
in the caller
catch (ClassCastException e) { System.out.println("caller fails: " + e.getMessage().split(" \\(")
[0]); }
```


© TechTribers. For personal use only. Do not redistribute.

```
try { checkedGet(config, "port", Integer.class); }
catch (ClassCastException e) { System.out.println("checked fails at source: " + e.getMessage()); }
List<String> guarded = Collections.checkedList(new ArrayList<>(), String.class);
List raw = guarded;
try { raw.add(42); } catch (ClassCastException e) { System.out.println("checkedList: " +
e.getMessage().split(" \\(")[0]); }
```

▶ Output

```
unchecked returned: 8080
caller fails: class java.lang.String cannot be cast to class java.lang.Integer
checked fails at source: Cannot cast java.lang.String to java.lang.Integer
checkedList: Attempt to insert class java.lang.Integer element into collection with element type
class java.lang.String
```

💡 **Pro tip**

`Collections.checkedList` is a debugging tool, not a production pattern: wrap a collection you suspect is being polluted by raw-type legacy code, and the stack trace will point at the **write**, not at a distant read.

🔍 **Q756** GR-017 • Advanced

What are reifiable types, and when is `instanceof List<String>` allowed with pattern matching?

⚡ **Quick answer**

A **reifiable** type is one whose full type information is available at run time: primitives, non-generic classes, raw types, arrays of reifiable types, and parameterized types with only **unbounded wildcards** (`List<?>`). Only reifiable types can be checked with `instanceof` in general. Since **Java 16** (pattern matching for `instanceof`), a non-reifiable check is allowed if it is **provably safe** from the static type: on a `Collection<String>` , `c instanceof List<String> l` compiles, because any `List` that is a `Collection<String>` must be a `List<String>` .

The rule javac applies: an `instanceof` against a parameterized type is allowed when the cast from the expression's static type to that type would be **checked** (not "unchecked"). From `Object` to `List<String>` it would be unchecked; from `Collection<String>` to `List<String>` it's checked, because `List` 's type argument is fully determined by `Collection` 's.

The same rule drives `switch` patterns over generic sealed hierarchies: `case Ok<String>(var v)` on a `Result<String>` is accepted for the same reason. The last line is the root of it all: there is one `ArrayList.class` for every parameterization.

Java

```
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real
diagnostics (helper shown in full in Q750)
static String describe(Collection<String> c) {
    if (c instanceof List<String> list) return "list, first=" + list.get(0);        // safe: Java
16+
    if (c instanceof Set<String> set) return "set of " + set.size();
```


© TechTribers. For personal use only. Do not redistribute.

```
    return "other";
}

System.out.println(describe(List.of("a", "b")) + " | " + describe(Set.of("x")));
String[] cases = {
    "boolean f(Object o) { return o instanceof List<String>; }",
    "boolean f(Object o) { return o instanceof List<?>; }",
    "void f(Collection<String> c) { if (c instanceof List<String> l) { } }",
    "void f(Collection<String> c) { if (c instanceof ArrayList<Integer> l) { } }",
    "void f(List<? extends Number> c) { if (c instanceof ArrayList<? extends Number> l) { } }",
    "void f(Object o) { if (o instanceof Map.Entry<?, ?> e) { } }"
};
for (String c : cases) System.out.println(c + "\n   -> " + javac(c));
System.out.println("runtime class of List<String> and List<Integer> equal: " + (new
ArrayList<String>().getClass() == new ArrayList<Integer>().getClass()));
```

▶ Output

```
list, first=a | set of 1
boolean f(Object o) { return o instanceof List<String>; }
   -> error: java.lang.Object cannot be safely cast to java.util.List<java.lang.String>
boolean f(Object o) { return o instanceof List<?>; }
   -> compiles
void f(Collection<String> c) { if (c instanceof List<String> l) { } }
   -> compiles
void f(Collection<String> c) { if (c instanceof ArrayList<Integer> l) { } }
   -> error: incompatible types: java.util.Collection<java.lang.String> cannot be converted to
java.util.ArrayList<java.lang.Integer>
void f(List<? extends Number> c) { if (c instanceof ArrayList<? extends Number> l) { } }
   -> compiles
void f(Object o) { if (o instanceof Map.Entry<?, ?> e) { } }
   -> compiles
runtime class of List<String> and List<Integer> equal: true
```

⚠️ **Interview trap**

"`instanceof` can never be used with generics." Outdated since Java 16. The precise answer is: only when the check can't be verified at run time, which javac reports as "cannot be safely cast".

Q757 GR-018 • Advanced

What generic type information survives erasure, and how do you read it at run time?

⚡ **Quick answer**

Erasure removes type arguments from **objects and bytecode instructions**, not from **declarations**. The class file keeps a `Signature` attribute for every generic class, superclass, implemented interface, field, method parameter and return type, and reflection exposes it through `getGenericSuperclass()`, `getGenericInterfaces()`, `Field.getGenericType()`, `Method.getGenericReturnType()` and `getGenericParameterTypes()`. What is truly gone: the type arguments of an **object** (`new ArrayList<String>()`), of **local variables**, and of type parameters bound at a **call site**.

© TechTribers. For personal use only. Do not redistribute.

This is the mechanism behind every "generic-aware" framework: Jackson and Gson read field and method signatures to deserialize `List<Address>` fields, Spring resolves `Repository<User, Long>` from the implemented interface, JPA reads `List<Order> orders` to find the target entity, and JUnit resolves parameter types.

What you **can't** recover:

- `new ArrayList<String>()` as an object only knows it's an `ArrayList<E>`.
- A generic method's `T` as chosen at one call site.
- A local variable's type arguments (not stored in a `Signature` attribute; only in debug info, if at all).

That last gap is exactly what super type tokens (next question) work around: they turn a call-site type into a **declaration** (a superclass) so it lands in the class file.

Java

```
static class UserRepo implements Function<String, List<Integer>> {
    Map<String, Set<Long>> cache = new HashMap<>();
    public List<Integer> apply(String s) { return List.of(); }
    <T extends Comparable<T>> T best(List<? extends T> xs) { return null; }
}

static String clean(Object t) { return t.toString().replace("java.util.function.",
"").replace("java.util.", "").replace("java.lang.", ""); }

System.out.println("interface:   " + clean(UserRepo.class.getGenericInterfaces()[0]));
Field f = UserRepo.class.getDeclaredField("cache");
System.out.println("field type:  " + clean(f.getType()) + "  generic: " +
clean(f.getGenericType()));
Method best = UserRepo.class.getDeclaredMethod("best", List.class);
System.out.println("method:      " + clean(best.toGenericString().replace("Main$", "")));
List<String> local = new ArrayList<>();
System.out.println("object:      " + local.getClass().getSimpleName() + " declares <" +
local.getClass().getTypeParameters()[0] + ">, the String is gone");
```

▶ Output

```
interface:   Function<String, List<Integer>>
field type:  interface Map  generic: Map<String, Set<Long>>
method:      <T extends Comparable<T>> T UserRepo.best(List<? extends T>)
object:      ArrayList declares <E>, the String is gone
```

What erasure removes — and what the class file keeps (Signature attribute)

KEPT: declarations → reflection can read them

```
class UserRepo implements Function<String, List<Integer>>
getGenericInterfaces()
Map<String, Set<Long>> cache;
Field.getGenericType()
<T extends Comparable<T>> T best(List<? extends T>)
Method.getGenericReturnType() / params
abstract subclass: new TypeRef<List<User>>() {}
```

GONE: uses and objects

```
new ArrayList<String>()
→ just an ArrayList<E>
List<String> local = ... ;
→ local variable type args (no Signature)
T chosen at a call site of <T> f()
→ unrecoverable
list.getClass() == ArrayList.class (one class)
```

Jackson, Spring's ResolvableType, JPA and JUnit resolve generics from DECLARATIONS, never from objects.

© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> `javap -v YourClass.class | grep Signature` shows the raw signature strings, such as `Ljava/util/Map<Ljava/lang/String;Ljava/util/Set<Ljava/lang/Long;>;>;`. Useful when a framework claims it "can't resolve the generic type".

Q758 GR-019 • Advanced

What is a super type token (Jackson `TypeReference`, Gson `TypeToken`), and what are its limits?

> ⚡ **Quick answer**
> A **super type token** is an abstract generic class you instantiate as an **anonymous subclass**: `new TypeRef<Map<String, List<Integer>>>() {}`. Because the anonymous class's **superclass declaration** includes the full type argument, it is stored in the class file, and the constructor reads it back with `getClass().getGenericSuperclass()`. That lets you pass types like `List<User>` that have no class literal. Limits: it must be subclassed with a concrete type (inside a generic method it captures `T`, not the caller's type), raw use fails, and each use site creates a new class.

Real-world equivalents: Jackson `new TypeReference<List<User>>() {}`, Gson `new TypeToken<List<User>>() {}.getType()`, Spring `new ParameterizedTypeReference<List<User>>() {}` (for `RestTemplate` / `RestClient` / `WebClient`), Guava `TypeToken`. All read `getGenericSuperclass()` exactly as above.

The generic-method trap is the important one: `insideGenericMethod()` returns `List<T>` with an unresolved type variable, whatever the caller wanted. A helper like `<T> List<T> fromJson(String json)` can't work with a type token created inside it; the token must be created where the concrete type is written, or the caller passes the `Type` in.

Java

```
static abstract class TypeRef<T> {
    final Type type;
    protected TypeRef() {
        Type sup = getClass().getGenericSuperclass();
        if (!(sup instanceof ParameterizedType p)) throw new IllegalStateException("TypeRef created
without a type argument");
        type = p.getActualTypeArguments()[0];
    }
}

static <T> Type insideGenericMethod() { return new TypeRef<List<T>>() { }.type; }

Type t = new TypeRef<Map<String, List<Integer>>>() { }.type;
System.out.println("captured: " + t.getTypeName().replace("java.util.", "").replace("java.lang.",
""));
System.out.println("raw class: " + ((ParameterizedType) t).getRawType().getTypeName());
System.out.println("in generic method: " + insideGenericMethod().getTypeName());
try { new TypeRef() { }; } catch (IllegalStateException e) { System.out.println("raw: " +
e.getMessage()); }
```

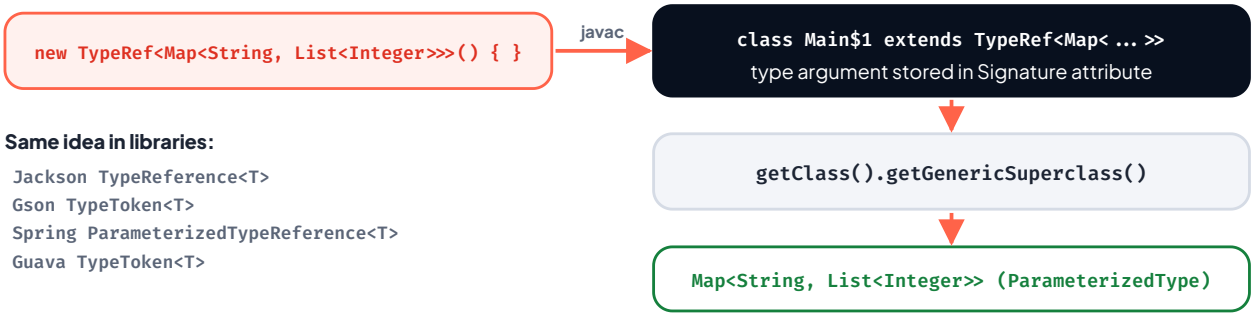

© TechTribers. For personal use only. Do not redistribute.

```
Object a = new TypeRef<String>() { }, b = new TypeRef<String>() { };
System.out.println("two use sites, two classes: " + (a.getClass() != b.getClass()));
```

▶ Output

```
captured: Map<String, List<Integer>>
raw class: java.util.Map
in generic method: java.util.List<T>
raw: TypeRef created without a type argument
two use sites, two classes: true
```

Super type token: turn a type argument into a superclass declaration



Same idea in libraries:

- `Jackson TypeReference<T>`
- `Gson TypeToken<T>`
- `Spring ParameterizedTypeReference<T>`
- `Guava TypeToken<T>`

Limit: created inside <T> List<T> f() it captures the variable T, not the caller's type.

⚠️ **Interview trap**

Writing `objectMapper.readValue(json, List.class)` and expecting `List<User>`. Jackson then builds a `List<LinkedHashMap>`, and the `ClassCastException` appears only when you read an element as `User`. That's heap pollution produced by a library, and a super type token is the fix.

🎯 **Likely follow-up**

Can you build a `TypeRef` without subclassing? Not from Java syntax, but you can construct a `ParameterizedType` instance yourself (Guava's `Types.newParameterizedType`, Jackson's `TypeFactory.constructCollectionType(List.class, User.class)`).

Q759 GR-020 • Advanced

What is a type-safe heterogeneous container (`Class<T>` as a key)?

⚡ **Quick answer**

A container whose **keys** are type tokens (`Class<T>`) instead of the container itself being parameterized, so each key fixes the type of its own value: `put(Class<T> type, T value)` and `T get(Class<T> type)`. Internally values are stored as `Object` and retrieved with `type.cast`, which is a checked cast, so the container stays type-safe even if a raw-type caller tries to cheat. It's the pattern behind `ClassToInstanceMap`, attribute and context maps, and annotation lookup (`getAnnotation(Class<A>)`).

Why it works: `Class` is generic in its own type (`String.class` is a `Class<String>`), so the key carries the value's type to the compiler, and the `Class` object carries it to the run time for the checked cast.

© TechTribers. For personal use only. Do not redistribute.

Limitations to mention:

- **No parameterized types as keys**: there is no `List<String>.class`; `List.class` would mix `List<String>` and `List<Integer>` under one key. Use a super type token as the key (Guava `TypeToInstanceMap`).
- **Exact-class keys**: `get(Number.class)` doesn't find a value stored under `Integer.class`. Decide whether lookup should walk supertypes (like the event bus in Q763).

In Spring, `ApplicationContext.getBean(Class<T>)` and `Environment.getProperty(String, Class<T>)` are type-token APIs; JUnit 5's `ExtensionContext.Store.get(key, Class<V>)` is literally this container.

Java

```
static final class Favorites {
    private final Map<Class<?>, Object> values = new HashMap<>();
    <T> void put(Class<T> type, T value) { values.put(Objects.requireNonNull(type),
type.cast(value)); }   // cast guards raw callers
    <T> T get(Class<T> type) { return type.cast(values.get(type)); }
}

Favorites f = new Favorites();
f.put(String.class, "Java");
f.put(Integer.class, 0xCAFE);
f.put(Class.class, Favorites.class);
String s = f.get(String.class); int i = f.get(Integer.class); Class<?> c = f.get(Class.class);
System.out.println(s + " " + Integer.toHexString(i) + " " + c.getSimpleName());
try { f.put((Class) Integer.class, "not an int"); }                        // a raw-type caller
trying to cheat
catch (ClassCastException e) { System.out.println("rejected: " + e.getMessage()); }
System.out.println("annotation lookup is the same pattern: " + Deprecated.class.getSimpleName() + "
-> " + Thread.class.getMethod("stop").getAnnotation(Deprecated.class).forRemoval());
```

▶ Output

```
Java cafe Favorites
rejected: Cannot cast java.lang.String to java.lang.Integer
annotation lookup is the same pattern: Deprecated -> true
```

💡 Pro tip

`Thread.stop()` is a real example of `@Deprecated(forRemoval = true)`; in JDK 20+ it always throws `UnsupportedOperationException`.

© TechTribers. For personal use only. Do not redistribute.

Q760 GR-021 • Advanced

How does the generic singleton factory work (`Collections.emptyList()`, `Function.identity()`)?

⚡ Quick answer

When an object holds **no values of type** `T` (an empty list, an identity function, a natural-order comparator), one instance can safely serve every parameterization. A generic static method returns that single instance through an unchecked cast: `(List<T>) EMPTY_LIST`. The cast is safe because erasure makes all parameterizations the same object, and the instance never produces a `T` that wasn't given to it. It saves allocation and is how `Collections.emptyList()`, `List.of()`, `Comparator.naturalOrder()` and `Collections.emptyMap()` work.

Each line is a real JDK singleton: `Collections.EMPTY_LIST`, `ImmutableCollections.EMPTY_LIST` for `List.of()`, and `Comparators.NaturalOrderComparator.INSTANCE` (an enum singleton). `Function.identity()` is implemented as `return t -> t;`. A **non-capturing** lambda evaluates to the same instance on every evaluation in current HotSpot, so it's a singleton in practice, but the specification doesn't promise that. Don't compare lambdas with `==` in real code.

The unchecked cast is acceptable only because of the reasoning in the comment: the object stores nothing typed `T`. A singleton `ArrayList` shared this way would be a disaster: one caller adds a `String`, another reads it as an `Integer`.

Java

```
private static final UnaryOperator<Object> IDENTITY = x -> x;

// safe: IDENTITY returns its argument unchanged and holds no T
@SuppressWarnings("unchecked")
static <T> UnaryOperator<T> identity() { return (UnaryOperator<T>) IDENTITY; }

UnaryOperator<String> s = identity(); UnaryOperator<Integer> n = identity();
System.out.println(s.apply("same") + " " + n.apply(42) + " one instance: " + (s == (Object) n));
System.out.println("emptyList shared: " + (Collections.<String>emptyList() == (Object) Collections.
<Integer>emptyList()));
System.out.println("List.of() shared: " + (List.<String>of() == (Object) List.<Integer>of()));
System.out.println("naturalOrder shared: " + (Comparator.<String>naturalOrder() == (Object)
Comparator.<Integer>naturalOrder()) + " class " +
Comparator.naturalOrder().getClass().getSimpleName());
System.out.println("Function.identity() twice same object: " + (Function.identity() ==
Function.identity()));
```

▶ Output

```
same 42 one instance: true
emptyList shared: true
List.of() shared: true
naturalOrder shared: true class NaturalOrderComparator
Function.identity() twice same object: true
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
Returning `Collections.emptyList()` from a method whose callers then call `add()`. It is immutable and shared; you get `UnsupportedOperationException`. Same for `List.of()`.

Q761 GR-022 • Advanced

Why can't a generic class extend `Throwable`, while `<X extends Throwable> void f() throws X` is legal?

⚡ **Quick answer**
A `catch` clause is resolved at run time by the exception's **class**, and erasure would make `MyEx<String>` and `MyEx<Integer>` the same class, so `catch (MyEx<String> e)` couldn't be honoured. That's why generic subclasses of `Throwable` and `catch (T e)` are illegal. **Throwing** a type variable is fine, though, because `throws` is only checked at compile time. `<X extends Throwable> T orElseThrow(Supplier<? extends X>) throws X` gives callers the precise checked exception, and a type variable in a `throws` clause with no constraint is inferred as `RuntimeException`, so callers that throw nothing need no `try`.

Two inference rules make this pattern ergonomic (JLS 18.1.3, 18.4): a type variable that appears **only** in `throws` clauses is marked as a "throws" variable, and if nothing constrains it, it resolves to `RuntimeException` instead of `Exception`. That's why `retry(3, () -> "...")` needs no `try`, while the `IOException` lambda forces one.

The `catch (Exception e) { ... throw e; }` line compiles under the method's `throws X` because of **precise rethrow**: since `e` is effectively final, javac knows it can only be what `s.get()` throws (`X`) or an unchecked exception. This is also the mechanism behind the "sneaky throw" trick (EX-016), which abuses an unchecked cast to `X`.

Java

```
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real diagnostics (helper shown in full in Q750)
@FunctionalInterface interface ThrowingSupplier<T, X extends Exception> { T get() throws X; }

static <T, X extends Exception> T retry(int times, ThrowingSupplier<T, X> s) throws X {   // throws the CALLER's checked type
    for (int i = 1; ; i++) {
        try { return s.get(); } catch (Exception e) { if (i >= times) throw e; }        // precise rethrow: e is X or unchecked
    }
}

String v = retry(3, () -> "no checked exception, no try needed");                       // X inferred as RuntimeException
System.out.println(v);
try { retry(2, () -> { throw new IOException("disk"); }); }                              // X inferred as IOException
catch (IOException e) { System.out.println("caught precise type: " + e); }
System.out.println(Optional.class.getMethod("orElseThrow",
Supplier.class).toGenericString().replace("java.util.function.", "").replace("java.lang.",
"").replace("java.util.", ""));
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(javac("static class Oops<T> extends RuntimeException { }"));
System.out.println(javac("<X extends Exception> void f(Runnable r) { try { r.run(); } catch (X e) {
} }"));
```

▶ Output

```
no checked exception, no try needed
caught precise type: java.io.IOException: disk
public <X extends Throwable> T Optional.orElseThrow(Supplier<? extends X>) throws X
error: a generic class may not extend java.lang.Throwable
error: unexpected type
```

💡 **Pro tip**

Libraries use this to keep checked exceptions in functional code: Vavr's `CheckedFunction`, jOOλ's `Unchecked`, and Spring's `ThrowingFunction` (Spring 6). A `ThrowingSupplier<T, X>` like the one above is often all you need.

📦 **Q762** GR-023 • Advanced

What does boxing cost in generic collections, and what is the `contains(5L)` trap?

⚡ **Quick answer**

Type arguments must be reference types, so `List<Integer>` stores **pointers to `Integer` objects**: on a 64-bit JVM with compressed references that is about **20 bytes per element** (4-byte reference + 16-byte `Integer`) versus **4** for `int[]`, measured below. `HashMap<Integer, Integer>` costs about **72 bytes per entry**. Boxing also costs allocation, GC and cache misses. A correctness trap comes with it: collection lookups take `Object`, so `List<Integer>.contains(5L)` and `Map<Long, ?>.get(5)` compile and quietly return `false` / `null`, because a `Long` never equals an `Integer`.

Measured (Temurin 25.0.4.1, x64, compressed oops, 1,000,000 elements, heap used after `System.gc()`): `int[]` about 4 bytes per element, `ArrayList<Integer>` 20, `HashMap<Integer,Integer>` 72 per entry. With `-XX:+UseCompactObjectHeaders` (product feature in JDK 25, off by default) the `HashMap` drops to 64 bytes per entry, while `Integer` stays at 16 bytes because of 8-byte object alignment. The allocation counter in the snippet confirms the same 20-versus-4 ratio.

Options when it matters: primitive arrays; `IntStream` / `LongStream` instead of `Stream<Integer>`; primitive collections (Eclipse Collections, fastutil, HPPC); `BitSet` for dense integer sets. **Project Valhalla**: JEP 401 (value objects) has been integrated into the JDK 28 mainline as a **preview** (GA planned for March 2027), so no released JDK has it yet. With the preview on, `Integer` and the other wrappers become value classes without identity, which lets the JVM flatten and scalarize them. Generic specialization (a real `List<int>`) is a later Valhalla phase, so plan for primitive-specialized code for years yet.

Java

```
var mx = (com.sun.management.ThreadMXBean) ManagementFactory.getThreadMXBean();
long me = Thread.currentThread().threadId();
int n = 1_000_000;
long a0 = mx.getThreadAllocatedBytes(me);
```

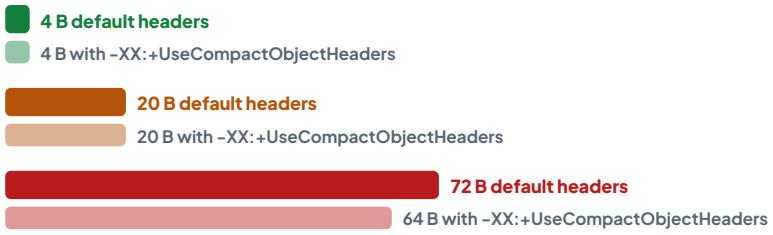

© TechTribers. For personal use only. Do not redistribute.

```
List<Integer> boxed = new ArrayList<>(n); for (int i = 0; i < n; i++) boxed.add(i + 1_000);     //
outside the -128..127 cache
long a1 = mx.getThreadAllocatedBytes(me);
int[] prim = new int[n]; for (int i = 0; i < n; i++) prim[i] = i + 1_000;
long a2 = mx.getThreadAllocatedBytes(me);
System.out.printf("allocated: List<Integer> %d B/element, int[] %d B/element%n", Math.round((a1 -
a0) / (double) n), Math.round((a2 - a1) / (double) n));
List<Integer> ids = List.of(5, 6, 7);
Map<Long, String> users = Map.of(5L, "Asha");
System.out.println("ids.contains(5L) = " + ids.contains(5L) + ", users.get(5) = " + users.get(5) +
", users.get(5L) = " + users.get(5L));
System.out.println("Integer.valueOf(5).equals(5L) = " + Integer.valueOf(5).equals(5L));
```

▶ Output

```
allocated: List<Integer> 20 B/element, int[] 4 B/element
ids.contains(5L) = false, users.get(5) = null, users.get(5L) = Asha
Integer.valueOf(5).equals(5L) = false
```

Memory per element, 1,000,000 elements (Temurin 25.0.4.1, compressed oops, measured)



ArrayList<Integer> = 4-byte reference + 16-byte Integer object (alignment keeps Integer at 16 B even with compact headers).
Also: boxing allocates (20 B/element measured by ThreadMXBean), and List<Integer>.contains(5L) is always false.

⚠️ **Interview trap**

The `contains(5L)` bug is common in real code: an ID column is `Long`, a list of IDs is built from `int`s, and an authorization check `allowedIds.contains(userId)` is always `false`. Error Prone's `CollectionIncompatibleType` check catches it at compile time.

Q763 GR-024 • Advanced

Coding: write a type-safe event bus with `Class<T>` keys and `Consumer<? super T>` handlers.

⚡ **Quick answer**

Key the handler registry by event `Class`, accept handlers as `Consumer<? super E>` (so a handler for a supertype can subscribe to a subtype), wrap each handler so it does a **checked** `type.cast` before calling it, and on `publish` walk the event's class, its superclasses and its interfaces so `subscribe(Event.class, ...)` sees every event. Use concurrent structures so subscription and publishing can happen from different threads.

© TechTribers. For personal use only. Do not redistribute.

The generics doing the work:

- `subscribe(Class<E>, Consumer<? super E>)` : the class token fixes `E` , and the wildcard lets an `audit` handler typed `Consumer<Object>` subscribe to `OrderCancelled` . Without `? super E` that line wouldn't compile.
- The stored lambda `e -> handler.accept(type.cast(e))` erases each handler to `Consumer<Object>` safely: the checked cast happens once, at dispatch.
- Dispatch order is deterministic: the concrete class, then its interfaces, then superclasses up to `Object` .

Production concerns to raise in a design interview: handler exceptions (isolate them per handler so one failure doesn't stop delivery), async delivery (an `Executor` per subscriber), weak references to avoid leaking subscribers, and the cost of walking the hierarchy on every publish (cache the resolved class list per event type, as Guava's `EventBus` does).

Java

```
static final class EventBus {
    private final Map<Class<?>, List<Consumer<Object>>> handlers = new ConcurrentHashMap<>();
    <E> void subscribe(Class<E> type, Consumer<? super E> handler) {
        handlers.computeIfAbsent(type, k -> new CopyOnWriteArrayList<>()).add(e ->
handler.accept(type.cast(e)));
    }
    void publish(Object event) {
        Set<Class<?>> seen = new LinkedHashSet<>();
        for (Class<?> c = event.getClass(); c != null; c = c.getSuperclass()) { seen.add(c);
collectInterfaces(c, seen); }
        for (Class<?> c : seen) handlers.getOrDefault(c, List.of()).forEach(h -> h.accept(event));
    }
    private static void collectInterfaces(Class<?> c, Set<Class<?>> out) {
        for (Class<?> i : c.getInterfaces()) if (out.add(i)) collectInterfaces(i, out);
    }
}
sealed interface OrderEvent permits OrderPlaced, OrderCancelled { String id(); }
record OrderPlaced(String id, int amount) implements OrderEvent { }
record OrderCancelled(String id, String reason) implements OrderEvent { }

EventBus bus = new EventBus();
Consumer<Object> audit = e -> System.out.println("  audit   <- " + e);
bus.subscribe(OrderPlaced.class, e -> System.out.println("  billing <- charge " + e.amount() + " for
" + e.id()));
bus.subscribe(OrderEvent.class, e -> System.out.println("  metrics <- " +
e.getClass().getSimpleName() + " " + e.id()));
bus.subscribe(OrderCancelled.class, audit);                                    // Consumer<Object>
for an OrderCancelled: ? super E
bus.subscribe(Object.class, e -> System.out.println("  catch-all <- " +
e.getClass().getSimpleName()));
System.out.println("publish OrderPlaced:"); bus.publish(new OrderPlaced("A-17", 499));
System.out.println("publish OrderCancelled:"); bus.publish(new OrderCancelled("A-17", "payment
failed"));
```

▶ Output

```
publish OrderPlaced:
  billing <- charge 499 for A-17
  metrics <- OrderPlaced A-17
  catch-all <- OrderPlaced
publish OrderCancelled:
```


© TechTribers. For personal use only. Do not redistribute.

```
audit   <- OrderCancelled[id=A-17, reason=payment failed]
metrics <- OrderCancelled A-17
catch-all <- OrderCancelled
```

🎯 **Likely follow-up**

How does Spring do it? `ApplicationEventPublisher` plus `@EventListener` methods; the listener's parameter type is the subscription key, resolved with generics support (`ResolvableType`), so `@EventListener void on(EntityCreated<Order> e)` filters by the type argument too.

Section 03

Reflection I: Class Objects, Members and Access



Class objects for every kind of type, declared versus inherited members, reflective construction and invocation done correctly, private and final fields under JEP 500, records and sealed types, the Type hierarchy, reflective arrays, and caching with ClassValue.

Questions 764–772 · 9 questions

Q764 GR-025 • Intermediate

What are the ways to get a `Class` object, and what is special about primitives, arrays and `getClass()` 's return type?

⚡ **Quick answer**

Three ways: a **class literal** (`String.class` , checked at compile time), `obj.getClass()` (the run-time class), and `Class.forName("java.lang.String")` (by binary name, runs static initialisers by default). Primitives and `void` have their own `Class` objects (`int.class == Integer.TYPE` , distinct from `Integer.class`); every array type has one `Class` per **component type and dimension**. `getClass()` is typed `Class<? extends |X|>` (the erasure of the static type), which is why `Class<? extends List> c = list.getClass()` compiles but `Class<List<String>>` never does. A class is identified by **name plus defining class loader**.

Choosing between them:

- **Class literal** when the type is known at compile time; it's the cheapest and refactoring-safe, and it doesn't initialise the class.
- `getClass()` when you need the dynamic type (for `equals` implementations: `getClass() != o.getClass()`).
- `Class.forName(name)` for names that come from configuration or plugins. The three-argument form `forName(name, false, loader)` lets you choose the loader and skip static initialisation, which frameworks

© TechTribers. For personal use only. Do not redistribute.

prefer.

Array names use JVM descriptors (`[I` , `[[I` , `[Ljava.lang.String;`), and `Class.forName` accepts those names, but not primitive names such as `"int"` . Java 12 added `componentType()` , `arrayType()` and `descriptorString()` (from `TypeDescriptor.OfField`), which make array and descriptor handling much less error-prone than string manipulation.

Java

```
List<String> list = new ArrayList<>();
Class<? extends List> fromObject = list.getClass();                    // static type erased:
Class<? extends List>
Class<?> fromName = Class.forName("java.util.ArrayList");
System.out.println("same Class object: " + (fromObject == fromName) + ", " + fromObject.getName());
System.out.println("int.class == Integer.TYPE: " + (int.class == Integer.TYPE) + ", int.class ==
Integer.class: " + (int.class == (Object) Integer.class));
System.out.println("primitive? " + int.class.isPrimitive() + " " + void.class.isPrimitive() + " " +
Integer.class.isPrimitive());
int[] a = new int[3]; int[][] b = new int[2][2];
System.out.println("int[] class shared by all int[]: " + (a.getClass() == new int[99].getClass()) +
", int[] vs int[][]: " + (a.getClass() == (Object) b.getClass()));
System.out.println("names: " + a.getClass().getName() + " " + b.getClass().getName() + " " +
String[].class.getName());
System.out.println("componentType: " + b.getClass().componentType() + ", arrayType: " +
int.class.arrayType().getSimpleName());
System.out.println("descriptors: " + int.class.descriptorString() + " " +
String.class.descriptorString() + " " + String[].class.descriptorString());
try { Class.forName("int"); } catch (ClassNotFoundException e) {
System.out.println("forName(\"int\") fails: " + e.getMessage()); }
System.out.println("forName for arrays: " + Class.forName("[I").getSimpleName() + " " +
Class.forName("[Ljava.lang.String;").getSimpleName());
```

▶ Output

```
same Class object: true, java.util.ArrayList
int.class == Integer.TYPE: true, int.class == Integer.class: false
primitive? true true false
int[] class shared by all int[]: true, int[] vs int[][]: false
names: [I [[I [Ljava.lang.String;
componentType: class [I, arrayType: int[]
descriptors: I Ljava/lang/String; [Ljava/lang/String;
forName("int") fails: int
forName for arrays: int[] String[]
```

⚠️ **Interview trap**

`if (x.getClass() == Integer.TYPE)` is never true: a value in an `Object` is always boxed, so its class is `Integer.class` . Frameworks that map primitives must handle both (`MethodType.wrap()` / `unwrap()` , or a primitive-to-wrapper table).

© TechTribers. For personal use only. Do not redistribute.

Q765 GR-026 • Intermediate

What is the difference between `getMethods()` and `getDeclaredMethods()` (and fields, constructors), and how do you collect all fields up the hierarchy?

⚡ **Quick answer**

`getXxx()` returns **public** members, **including inherited** ones (from superclasses and, for methods, superinterfaces). `getDeclaredXxx()` returns members **declared in this class only**, with **any** access level, and no inherited ones. Constructors are never inherited, so `getConstructors()` is just the public ones of this class. To get every field of an entity (what ORMs and serializers need), walk `getSuperclass()` up to `Object`, calling `getDeclaredFields()` on each level and skipping `static` and synthetic fields.

Things the table hides:

- `getMethods()` includes `Object`'s public methods (`toString`, `wait`, ...) and **default methods** inherited from interfaces (`name` above). `getDeclaredMethods()` includes synthetic methods, such as bridges and lambda bodies (`lambda$main$0`), so filter with `isSynthetic()` in frameworks.
- **Order is unspecified** for all of these. Sort if output or behaviour depends on it.
- The returned arrays are **copies**; each call allocates. Cache them (see Q772).
- `getField("secret")` throws `NoSuchFieldException` even though the field exists: it's private. Use `getDeclaredField` plus access checks.

Java

```
static class Base { public int id; protected String createdBy; private long version; public void
save() { } private void audit() { } }
interface Named { default String name() { return "named"; } }
static class Customer extends Base implements Named { public String email; private String secret;
public Customer() { } private Customer(String s) { } private void hash() { } }

static List<String> names(Member[] ms) { return Arrays.stream(ms).map(Member::getName).filter(n ->
!n.startsWith("$")).sorted().toList(); }

static List<Field> allInstanceFields(Class<?> c) {
    List<Field> out = new ArrayList<>();
    for (; c != null && c != Object.class; c = c.getSuperclass())
        for (Field f : c.getDeclaredFields()) if (!Modifier.isStatic(f.getModifiers()) &&
!f.isSynthetic()) out.add(f);
    return out;
}

System.out.println("getFields:           " + names(Customer.class.getFields()));
System.out.println("getDeclaredFields:   " + names(Customer.class.getDeclaredFields()));
System.out.println("all instance fields: " + allInstanceFields(Customer.class).stream().map(f ->
f.getDeclaringClass().getSimpleName() + "." + f.getName()).toList());
System.out.println("getMethods (non-Object): " +
names(Customer.class.getMethods()).stream().filter(n -> Set.of("save", "name", "hash",
"audit").contains(n)).toList());
System.out.println("getDeclaredMethods:  " + names(Customer.class.getDeclaredMethods()));
System.out.println("constructors public/declared: " + Customer.class.getConstructors().length + "/"
+ Customer.class.getDeclaredConstructors().length);
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
getFields:           [email, id]
getDeclaredFields:   [email, secret]
all instance fields: [Customer.email, Customer.secret, Base.id, Base.createdBy, Base.version]
getMethods (non-Object): [name, save]
getDeclaredMethods:  [hash]
constructors public/declared: 1/2
```

getX() vs getDeclaredX()

	public only?	inherited?	includes private?	includes synthetic?
`getMethods()`	yes	yes (+Object, +defaults)	no	rarely
`getDeclaredMethods()`	no	no	yes	yes (bridges, lambdas)
`getFields()`	yes	yes	no	no
`getDeclaredFields()`	no	no	yes	yes (this$0 …)
`getConstructors()`	yes	never inherited	no	no
`getDeclaredConstructors()`	no	never inherited	yes	no

All fields of an entity: walk getSuperclass() calling getDeclaredFields(). Order is unspecified; results are copies.

⚠️ **Interview trap**

Using `getDeclaredFields()` alone in a mapper and silently dropping every field inherited from a `BaseEntity` (id, version, audit columns). That bug ships often because the tests use flat classes.

🏗️ **Q766** GR-027 • Intermediate

How do you create objects reflectively, why is `Class.newInstance()` deprecated, and what is `InvocationTargetException`?

⚡ **Quick answer**

Use `type.getDeclaredConstructor(paramTypes...).newInstance(args...)`. `Class.newInstance()` (deprecated since Java 9) only calls the no-arg constructor and, worse, **rethrows any checked exception the constructor throws without declaring it**, bypassing the compiler's checks. `Constructor.newInstance` and `Method.invoke` instead **wrap** whatever the target throws in an `InvocationTargetException`; always unwrap it with `getCause()` before logging or rethrowing, or you lose the real error.

Look at the last line of output: `Class.newInstance()` let a checked `IOException` escape from a call that declares only `InstantiationException` and `IllegalAccessException`. Code that catches those two would miss it entirely. That is the reason for the deprecation, not performance.

The `create` helper also shows a limitation people hit in DI containers: `getDeclaredConstructor(types)` needs the **exact** parameter types. Passing an `Integer` doesn't match an `int` or a `Number` parameter. Real containers iterate `getDeclaredConstructors()` and check assignability (including primitive/wrapper pairs) themselves.

© TechTribers. For personal use only. Do not redistribute.

```java
static class Conn {
    final String url;
    public Conn() throws IOException { throw new IOException("no default URL"); }
    public Conn(String url) { if (url.isBlank()) throw new IllegalArgumentException("blank url");
this.url = url; }
}

static <T> T create(Class<T> type, Object... args) throws Exception {
    Class<?>[] types = Arrays.stream(args).map(Object::getClass).toArray(Class<?>[]::new);
    try { return type.getDeclaredConstructor(types).newInstance(args); }
    catch (InvocationTargetException e) {                        // the constructor itself threw
        if (e.getCause() instanceof RuntimeException re) throw re;
        throw e;
    }
}

System.out.println("created: " + create(Conn.class, "jdbc:pg://db").url);
try { create(Conn.class, " "); } catch (IllegalArgumentException e) { System.out.println("unwrapped:
" + e.getMessage()); }
try { Conn.class.getDeclaredConstructor().newInstance(); }
catch (InvocationTargetException e) { System.out.println("wrapped: " + e.getClass().getSimpleName()
+ " cause=" + e.getCause()); }
try { create(Conn.class, 42); } catch (NoSuchMethodException e) { System.out.println("no match: " +
e.getMessage().replace("Main$", "")); }
@SuppressWarnings("deprecation") Runnable old = () -> { try { Conn.class.newInstance(); } catch
(Exception e) { System.out.println("Class.newInstance threw raw: " + e); } };
old.run();
```

▶ Output

```
created: jdbc:pg://db
unwrapped: blank url
wrapped: InvocationTargetException cause=java.io.IOException: no default URL
no match: Conn.<init>(java.lang.Integer)
Class.newInstance threw raw: java.io.IOException: no default URL
```

💡 **Pro tip**

Unwrap at the boundary in one place. Spring's `ReflectionUtils.handleReflectionException` and `InvocationTargetException.getTargetException()` (same as `getCause()`) exist for this. Logging an `InvocationTargetException` without its cause is a classic "the error message says nothing" incident.

© TechTribers. For personal use only. Do not redistribute.

Q767 GR-028 • Advanced

How do you find and invoke overloaded, varargs, static and generic methods reflectively?

⚡ **Quick answer**

`getMethod(name, paramTypes...)` matches the **exact erased** parameter types: `int.class` and `Integer.class` select different overloads, a generic method's `T` is looked up as its erasure (`Object` or its bound), and a varargs parameter is just an array type (`String[].class`). `invoke(target, args...)` takes `null` as the target for static methods and unboxes arguments for primitive parameters. The classic trap: passing a `String[]` to a varargs `main(String...)` with `invoke(null, array)` spreads the array into separate arguments; wrap it as `(Object) array` .

Reflection does none of the overload resolution javac does: no widening (`long` won't find `fmt(int)`), no boxing in lookup, no varargs expansion, no "most specific" choice. It's an exact lookup on the erased descriptor. **Invocation** does a little more: arguments are unboxed and widened to fit primitive parameters (`invoke(null, 7)` works for `fmt(int)` because `7` is boxed to `Integer` and unboxed again).

The spread trap: `invoke(Object obj, Object... args)` is itself varargs, so a `String[]` passed alone is treated as the whole argument array. For a single-parameter method that takes an array or `Object` , cast to `(Object)` . The same applies to invoking `main(String[] args)` reflectively, which plugin launchers and test runners do.

For an **instance** method, pass the target object; for an **interface default** method, `getMethod` on the implementing class finds it and `invoke` dispatches virtually as usual.

Java

```
public static String fmt(int n) { return "int:" + n; }

public static String fmt(Integer n) { return "Integer:" + n; }

public static String fmt(Object n) { return "Object:" + n; }

public static <T extends Number> String twice(T n) { return "twice:" + (n.doubleValue() * 2); }

public static String join(String sep, String... parts) { return String.join(sep, parts); }

for (Class<?> p : List.of(int.class, Integer.class, Object.class))
    System.out.println("getMethod(fmt, " + p.getSimpleName() + ") -> " + Main.class.getMethod("fmt",
p).invoke(null, 7));
System.out.println(Main.class.getMethod("twice", Number.class).invoke(null, 21));        // looked
up by erasure of T
Method join = Main.class.getMethod("join", String.class, String[].class);
System.out.println("isVarArgs=" + join.isVarArgs() + " -> " + join.invoke(null, "-", new String[]
{"a", "b", "c"}));
String[] parts = {"x", "y"};
try { System.out.println(Main.class.getMethod("fmt", Object.class).invoke(null, (Object[]) parts));
}
catch (IllegalArgumentException e) { System.out.println("spread trap: " + e.getMessage()); }
System.out.println(Main.class.getMethod("fmt", Object.class).invoke(null, (Object)
parts).toString().startsWith("Object:[Ljava.lang.String;"));
```


© TechTribers. For personal use only. Do not redistribute.

```
try { Main.class.getMethod("fmt", long.class); } catch (NoSuchMethodException e) {
System.out.println("no widening in lookup: " + e.getMessage()); }
```

▶ Output

```
getMethod(fmt, int) -> int:7
getMethod(fmt, Integer) -> Integer:7
getMethod(fmt, Object) -> Object:7
twice:42.0
isVarArgs=true -> a-b-c
spread trap: wrong number of arguments: 2 expected: 1
true
no widening in lookup: Main.fmt(long)
```

💡 **Pro tip**

When you need javac-like resolution (matching user-supplied arguments to the best overload), use `MethodHandles` with `asType` or a library (Apache Commons `MethodUtils.invokeMethod`) instead of writing assignability rules yourself.

🔓 **Q768** GR-029 • Advanced

Can reflection read private fields and change `final` fields? (`setAccessible`, `trySetAccessible`, JEP 500)

⚡ **Quick answer**

Within your own (unnamed or opened) modules, `setAccessible(true)` lets reflection read and write **private** members. **Instance** `final` fields of normal classes can still be written that way today, but it's unreliable: compile-time constants are inlined, and the JIT may have folded the old value. `static final` fields, **record** components and **hidden-class** fields can't be changed at all. **JDK 26** (JEP 500) starts closing the rest: mutating an instance final field reflectively now prints a warning, `--illegal-final-field-mutation=deny` turns it into an `IllegalAccessException`, and a future release will deny by default.

This snippet runs on **Temurin 26.0.2.1** with `--illegal-final-field-mutation=deny`, which is the behaviour JEP 500 plans to make the default. We also ran it with JDK 26's **default** setting (`warn`): the `instance final` line then reads `changed, field now 1000000`, and the VM writes three lines to standard error (not to `System.err`, so the program can't capture them): `WARNING: Final field balance in class Main$Account has been mutated reflectively by class Main in unnamed module ...`, `WARNING: Use --enable-final-field-mutation=ALL-UNNAMED to avoid a warning`, and `WARNING: Mutating final fields will be blocked in a future release unless final field mutation is enabled`.

On JDK 21 and 25 the same write succeeds silently. Records and `static final` fields have been protected since they existed (we measured identical `IllegalAccessException`s on 21, 25 and 26).

Why it was never reliable even when allowed: a `private final int limit = 5` is a **constant variable**, so javac inlines `5` into every read. We measured this in a separate run on JDK 21, 25 and 26: after a reflective write of `9`, `field.get()` returned `9` while code reading the field directly still printed the original value. And the JIT is allowed to treat other final fields as constants too, which is exactly the optimisation JEP 500 wants to make safe.

© TechTribers. For personal use only. Do not redistribute.

Who is affected: serialization libraries, mocking and DI frameworks that inject into `final` fields, and test code using `ReflectionTestUtils.setField` on final fields. Fix: constructor injection, non-final fields for injected values, or `--enable-final-field-mutation=<module>` as a documented, temporary escape hatch.

Java

```
static class Account { private final Integer balance = 100; private final int limit = 5; private
String owner = "asha"; }
record Point(int x, int y) { }
static final Integer MAX = 10;

static String attempt(String what, Object target, Class<?> c, String name, Object value) {
    try {
        Field f = c.getDeclaredField(name);
        if (!f.trySetAccessible()) return what + ": not accessible";
        f.set(target, value);
        return what + ": changed, field now " + f.get(target);
    } catch (IllegalAccessException | NoSuchFieldException e) {
        return what + ": " + e.getClass().getSimpleName() + " - " + e.getMessage().replaceAll(" \\
(in unnamed module @\\p{XDigit}+\\)", "").replaceAll(" @\\p{XDigit}+", "");
    }
}

Account acc = new Account();
System.out.println("JDK " + Runtime.version().feature());
System.out.println(attempt("private non-final", acc, Account.class, "owner", "eve"));
System.out.println(attempt("instance final", acc, Account.class, "balance", 1_000_000));
System.out.println(attempt("record component", new Point(1, 2), Point.class, "x", 9));
System.out.println(attempt("static final", null, Main.class, "MAX", 99));
```

▶ Output

```
JDK 26
private non-final: changed, field now eve
instance final: IllegalAccessException - class Main cannot set final field Main$Account.balance,
unnamed module is not allowed to mutate final fields
record component: IllegalAccessException - Can not set final int field Main$Point.x to
java.lang.Integer
static final: IllegalAccessException - Can not set static final java.lang.Integer field Main.MAX to
java.lang.Integer
```

⚠ Interview trap

Assuming `setAccessible(true)` works on JDK classes. Since Java 16/17 strong encapsulation (MJ-026) it throws `InaccessibleObjectException` for non-opened packages such as `java.lang`. Use `trySetAccessible()` when failing is acceptable; it returns `false` instead of throwing.

© TechTribers. For personal use only. Do not redistribute.

Q769 GR-030 • Intermediate

How do you reflect on records, enums and sealed types?

⚡ Quick answer

Records: `isRecord()`, `getRecordComponents()` (in declaration order, which is guaranteed), each with `getName()`, `getType()`, `getGenericType()` and `getAccessor()`, and the canonical constructor found from the component types. **Enums**: `isEnum()`, `getEnumConstants()` (in declaration order), and `Enum.valueOf(type, name)`. **Sealed types**: `isSealed()` and `getPermittedSubclasses()`. These are the APIs serializers and mappers use to handle modern data types without guessing.

Why records are friendlier to frameworks than JavaBeans: component **order** is part of the class file (the `Record` attribute), names are always available without `-parameters`, and there's exactly one canonical constructor whose parameter types match the components. Jackson (2.12+), Spring's data binding and JPA projections rely on these APIs.

Reflection limits that still apply: you can't `set` a record field (Q768), so "update" means calling the canonical constructor with a modified argument array, which is how `with`-style copy helpers are written today.

`getEnumConstants()` returns `null` for non-enum classes and a fresh copy of the array each call. For an enum constant with a body (`PAID { ... }`), `getClass()` is an anonymous subclass; use `getDeclaringClass()` to get the enum type.

Java

```
record Money(String currency, long cents) { }
record Order(String id, Money total, List<String> items) { }
enum Status { NEW, PAID, SHIPPED }
sealed interface Shape permits Circle, Square { }
record Circle(double r) implements Shape { }
record Square(double side) implements Shape { }

static Map<String, Object> toMap(Record r) throws Exception {
    Map<String, Object> m = new LinkedHashMap<>();
    for (RecordComponent c : r.getClass().getRecordComponents()) {
        Object v = c.getAccessor().invoke(r);
        m.put(c.getName(), v instanceof Record nested ? toMap(nested) : v);
    }
    return m;
}

static <R extends Record> R fromMap(Class<R> type, Map<String, Object> m) throws Exception {
    RecordComponent[] cs = type.getRecordComponents();
    Class<?>[] types = Arrays.stream(cs).map(RecordComponent::getType).toArray(Class<?>[]::new);
    Object[] args = Arrays.stream(cs).map(c -> m.get(c.getName())).toArray();
    return type.getDeclaredConstructor(types).newInstance(args);          // the canonical
constructor
}

Order o = new Order("A-1", new Money("INR", 49_900), List.of("book"));
Map<String, Object> m = toMap(o);
System.out.println(m);
System.out.println(fromMap(Money.class, Map.of("currency", "USD", "cents", 1_250L)));
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println("items generic type: " + Order.class.getRecordComponents()[2].getGenericType());
System.out.println("enum: " + Arrays.toString(Status.class.getEnumConstants()) + " valueOf -> " +
Enum.valueOf(Status.class, "PAID").ordinal());
System.out.println("sealed: " + Shape.class.isSealed() + " permits " +
Arrays.stream(Shape.class.getPermittedSubclasses()).map(Class::getSimpleName).toList());
```

▶ Output

```
{id=A-1, total={currency=INR, cents=49900}, items=[book]}
Money[currency=USD, cents=1250]
items generic type: java.util.List<java.lang.String>
enum: [NEW, PAID, SHIPPED] valueOf -> 1
sealed: true permits [Circle, Square]
```

🎯 **Likely follow-up**

How would you detect unhandled subtypes at run time? Walk `getPermittedSubclasses()` recursively and compare against your handler registry at startup. With pattern-matching `switch` the compiler already does this for you, which is the better tool.

🌲 **Q770** GR-031 • Advanced

What is the `java.lang.reflect.Type` hierarchy (`ParameterizedType`, `TypeVariable`, `WildcardType`, `GenericArrayType`)?

⚡ **Quick answer**

`Type` is the common interface for every type reflection can describe. A plain or raw type is a `Class`; `Map<String, Integer>` is a `ParameterizedType` (raw type plus actual type arguments plus owner type); `T` is a `TypeVariable` (name, bounds and generic declaration); `? extends Number` is a `WildcardType` (upper and lower bounds); `T[]` or `List<String>[]` is a `GenericArrayType`. Generic-aware code must handle all five, usually recursively.

Practical uses: building a JSON or config binder that must know the element type of `List<Address>`, resolving a `Repository<T, ID>`'s actual arguments for a subclass, or writing a DI container that injects `Provider<Service>`.

Two details trip people up:

- A `TypeVariable` must be **resolved** against the concrete class. For `class UserRepo extends Repo<User>`, the field `T entity` declared in `Repo` reports `T`; you have to map `T` to `User` via `UserRepo.class.getGenericSuperclass()`. Spring's `ResolvableType`, Guava's `TypeToken.resolveType` and Jackson's `TypeFactory` exist to do this properly.
- A wildcard **always** has an upper bound (`Object` for `?` and `? super X`), so check lower bounds first, as `render` does.

The snippet's `switch` on a `Type` uses pattern matching with guards (Java 21); before that, this was a chain of `instanceof` checks.

© TechTribers. For personal use only. Do not redistribute.

Java

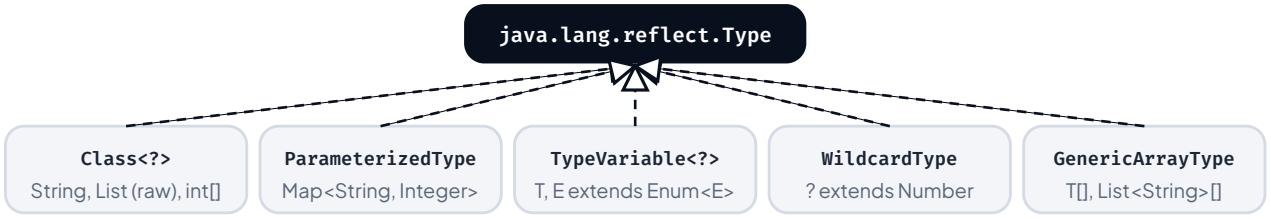
```
static String render(Type t) {
    return switch (t) {
        case Class<?> c when c.isArray() -> render(c.componentType()) + "[]";
        case Class<?> c -> c.getSimpleName();
        case ParameterizedType p -> render(p.getRawType()) +
Arrays.stream(p.getActualTypeArguments()).map(Main::render).collect(Collectors.joining(", ", "<",
">"));
        case TypeVariable<?> v -> v.getName();                          // never expand bounds here:
T extends Comparable<T> recurses
        case WildcardType w -> w.getLowerBounds().length > 0 ? "? super " +
render(w.getLowerBounds()[0])
                               : w.getUpperBounds()[0] == Object.class ? "?" : "? extends " +
render(w.getUpperBounds()[0]);
        case GenericArrayType g -> render(g.getGenericComponentType()) + "[]";
        default -> t.getTypeName();
    };
}

static class Holder<T extends Comparable<T>> {
    Map<String, List<? extends Number>> a; Comparator<? super T> b; T[] c; List<String>[] d;
Optional<Map.Entry<String, int[]>> e;
}

TypeVariable<?> tv = Holder.class.getTypeParameters()[0];
System.out.println("declared: " + tv + " extends " + render(tv.getBounds()[0]) + ", declared by " +
((Class<?>) tv.getGenericDeclaration()).getSimpleName());
for (Field f : Holder.class.getDeclaredFields())
    if (!f.isSynthetic()) System.out.println(f.getName() + ": " +
f.getGenericType().getClass().getInterfaces()[0].getSimpleName() + " -> " +
render(f.getGenericType()));
```

▶ Output

```
declared: T extends Comparable<T>, declared by Holder
a: ParameterizedType -> Map<String, List<? extends Number>>
b: ParameterizedType -> Comparator<? super T>
c: GenericArrayType -> T[]
d: GenericArrayType -> List<String>[]
e: ParameterizedType -> Optional<Entry<String, int[]>>
```



ParameterizedType: getRawType(), getActualTypeArguments(), getOwnerType()
TypeVariable: getName(), getBounds(), getGenericDeclaration() — resolve against the concrete subclass
WildcardType: check getLowerBounds() first; getUpperBounds() is [Object] for ? and ? super X

Never expand a TypeVariable's bounds recursively: T extends Comparable<T> never terminates.

© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Two classic bugs in hand-written type walkers. First, expanding a type variable's bounds recursively: with `T extends Comparable<T>` that never terminates (our first version of this snippet died with a `StackOverflowError`), so print or resolve type variables by name and handle bounds once. Second, casting `field.getGenericType()` straight to `ParameterizedType`. It is a plain `Class` when the field isn't generic (`String name`) and when it's raw (`List items`). Always branch on the actual subtype.

🔢 **Q771** GR-032 • Intermediate

How do you create and manipulate arrays reflectively with `java.lang.reflect.Array`?

⚡ **Quick answer**

`java.lang.reflect.Array` creates and accesses arrays whose component type is only known at run time: `Array.newInstance(componentType, length)` (or `newInstance(type, dims...)` for multi-dimensional arrays), `Array.getLength(arr)`, `Array.get/set(arr, i)` (with boxing), and primitive-specific `getInt` / `setInt` and so on. It works for primitive arrays too, which `Object[]` casts can't handle. It's how `Arrays.copyOf(T[], int)` keeps the array's run-time type.

Why a dedicated API: `int[]` is not an `Object[]` (the cast fails, as shown), so generic array code that must handle primitives either overloads for all eight primitive types (as `java.util.Arrays` does) or goes through `Array`.

`Array.set` performs the same checks as a normal array store and also refuses to unbox into the wrong type, so it's safe but slower than direct indexing. Use it at the edges (generic copying, deserialization, test utilities), not in a hot loop.

Java

```
static Object resize(Object array, int newLength) {                    // works for int[], String[], anything
    Class<?> type = array.getClass();
    if (!type.isArray()) throw new IllegalArgumentException("not an array: " + type);
    Object copy = Array.newInstance(type.componentType(), newLength);
    System.arraycopy(array, 0, copy, 0, Math.min(Array.getLength(array), newLength));
    return copy;
}

static String dump(Object array) {
    StringJoiner sj = new StringJoiner(", ", "[", "]");
    for (int i = 0; i < Array.getLength(array); i++) {
        Object v = Array.get(array, i);                                // primitives come back boxed
        sj.add(v != null && v.getClass().isArray() ? dump(v) : String.valueOf(v));
    }
    return sj.toString();
}

int[] ints = (int[]) resize(new int[]{1, 2, 3}, 5);
String[] strs = (String[]) resize(new String[]{"a", "b", "c"}, 2);
Object grid = Array.newInstance(int.class, 2, 3);                      // int[2][3]
```


© TechTribers. For personal use only. Do not redistribute.

```
Array.setInt(Array.get(grid, 1), 2, 42);
System.out.println(dump(ints) + " " + ints.getClass().getSimpleName() + " | " + dump(strs) + " " +
strs.getClass().getSimpleName());
System.out.println(dump(grid) + " " + grid.getClass().getSimpleName());
try { Object[] boxed = (Object[]) (Object) ints; } catch (ClassCastException e) {
System.out.println("int[] is not Object[]: " + e.getMessage().split(" \\(")[0]); }
try { Array.set(strs, 0, 7); } catch (IllegalArgumentException e) { System.out.println("type
checked: " + e.getMessage()); }
```

▶ Output

```
[1, 2, 3, 0, 0] int[] | [a, b] String[]
[[0, 0, 0], [0, 0, 42]] int[][]
int[] is not Object[]: class [I cannot be cast to class [Ljava.lang.Object;
type checked: array element type mismatch
```

💡 Pro tip

`Arrays.copyOf(original, newLength, newType)` already does the reflective part for object arrays; reach for `Array.newInstance` when the component type comes from a `Class` or `Type` you resolved yourself.

⏱️ **Q772** GR-033 • Advanced

How slow is reflection really, and how should you cache reflective lookups? (measured, plus `ClassValue`)

⚡ Quick answer

On JDK 25 the expensive part is the **lookup**, not the call. We measured about **39 ns** per call when doing `getMethod` + `invoke` every time, about **9 ns** for an `invoke` through a cached `Method` stored in a non-final field, about **1.7 ns** when the `Method` is in a `static final` field (the JIT can constant-fold it), and **0.4–0.7 ns** for a direct call, a `static final MethodHandle` or a `LambdaMetafactory` lambda. So: **cache** lookups, keep hot handles in `static final` fields, and for per-class metadata use `ClassValue`, which caches per `Class` without preventing class unloading.

Measurement (Temurin 25.0.4.1, 1 vCPU Xeon 2.1 GHz, a hand-written timing loop calling a trivial getter 50 million times after warm-up, median of 3 runs; treat it as order-of-magnitude, not a JMH result): direct 0.68 ns, `static final MethodHandle.invokeExact` 0.39 ns, `LambdaMetafactory` lambda 0.39 ns, `static final Method.invoke` 1.65 ns, `MethodHandle` in a non-final field 2.57 ns, `Method.invoke` through a non-final field 8.81 ns, `getMethod` + `invoke` on every call 39.0 ns. Since JDK 18, core reflection is implemented on top of method handles (MJ-037), which is why a constant `Method` is now close to direct calls.

Why `ClassValue` and not a `ConcurrentHashMap<Class<?>, ...>`? A static map holding `Class` keys (and values that reference `Field`s of that class) keeps every class and its class loader alive: in app servers, hot-redeploy containers and plugin systems that is a classic metaspace leak. `ClassValue` stores the value on the `Class` object itself, so it disappears with the class. Jackson, Groovy and the JDK (`java.lang.invoke`) use it.

© TechTribers. For personal use only. Do not redistribute.

```java
static final AtomicInteger computations = new AtomicInteger();
static final ClassValue<List<Field>> FIELDS = new ClassValue<>() {                // per-class
cache, unload-friendly
    @Override protected List<Field> computeValue(Class<?> type) {
        computations.incrementAndGet();
        List<Field> fs = new ArrayList<>();
        for (Class<?> c = type; c != null && c != Object.class; c = c.getSuperclass())
            for (Field f : c.getDeclaredFields()) if (!Modifier.isStatic(f.getModifiers()) &&
f.trySetAccessible()) fs.add(f);
        return List.copyOf(fs);
    }
};
record User(String name, int age) { }
record Team(String name, List<User> members) { }

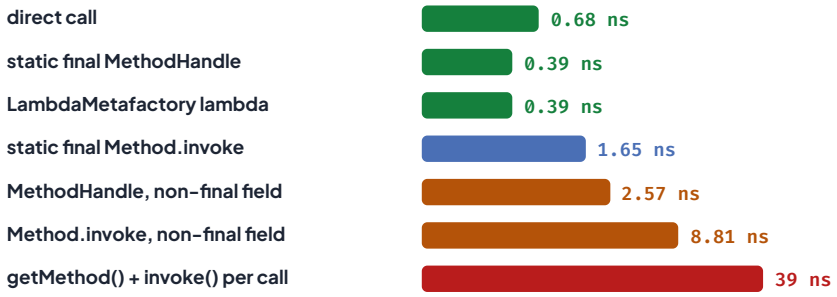
static String describe(Object o) throws IllegalAccessException {
    StringJoiner sj = new StringJoiner(", ", o.getClass().getSimpleName() + "{", "}");
    for (Field f : FIELDS.get(o.getClass())) sj.add(f.getName() + "=" + f.get(o));
    return sj.toString();
}

for (int i = 0; i < 1_000; i++) describe(new User("u" + i, i));
System.out.println(describe(new User("Asha", 31)) + " | " + describe(new Team("core", List.of())));
System.out.println("lookups: 1,002 describe() calls, reflective scans: " + computations.get());
```

▶ Output

```
User{name=Asha, age=31} | Team{name=core, members=[]}
lookups: 1,002 describe() calls, reflective scans: 2
```

Cost per call of a trivial getter, ns (JDK 25.0.4.1, median of 3 runs, log scale)



Lookup is the expensive part: cache Method / MethodHandle objects, keep hot ones in static final fields.

Hand-written loop, 50M calls after warm-up, 1 vCPU: order-of-magnitude figures, not JMH results.

⚠️ **Interview trap**

Benchmarking reflection with a loop that also does string concatenation or boxing, or with all variants behind one interface. We tried the latter first: the shared call site turned megamorphic and every variant looked the same (1–6 ns). Use JMH for numbers you'll publish; use separate monomorphic loops at minimum.

© TechTribers. For personal use only. Do not redistribute.

Section 04

Reflection II: Proxies, Method Handles and Dynamic Code

JDK dynamic proxies and why Spring's @Transactional fails on self-invocation, MethodHandles and LambdaMetafactory, hidden classes created at run time, class-loader identity and run-time compilation, ServiceLoader, when to avoid reflection, and a reflective audit differ.

Questions 773–781 · 9 questions



Q773 GR-034

• Advanced

How do JDK dynamic proxies work (`Proxy`, `InvocationHandler`, `invokeDefault`)?

⚡ **Quick answer**

`Proxy.newProxyInstance(loader, interfaces, handler)` generates a class at run time (an ordinary, non-hidden class named like `jdk.proxy1.$Proxy0`, placed in a dynamic module the JDK creates) that implements the given **interfaces** and routes **every** call, including `equals`, `hashCode` and `toString`, to `handler.invoke(proxy, method, args)`. The handler decides what to do: delegate to a real target, add cross-cutting logic (timing, retries, caching), or implement the interface entirely (as Spring Data and Retrofit do). Interface **default** methods can be called through `InvocationHandler.invokeDefault` (Java 16). Proxies only work for interfaces.

Notice the log: calling the default `refund` through the proxy also logs its inner `charge` call, because `invokeDefault` runs the default method with `this` = the proxy. Compare that with the self-invocation problem of class-based proxies in the next question.

Rules worth knowing:

- The handler's exceptions: throwing a checked exception the interface method doesn't declare produces an `UndeclaredThrowableException`. Always unwrap `InvocationTargetException` (as above) so callers see the original exception.
- Returning `null` for a primitive return type throws `NullPointerException`; returning the wrong type throws `ClassCastException`.
- The proxy class is cached per (loader, interfaces) combination; creating proxies is cheap after the first one.

Where you meet them: Spring AOP for interface-based beans, Spring Data repository interfaces, MyBatis mappers, Retrofit and Feign HTTP clients, JDBC connection-pool wrappers, Mockito's `mock(Interface.class)` in some configurations, and annotation instances (Q789).

© TechTribers. For personal use only. Do not redistribute.

Java

```
public interface PaymentService {
    String charge(String customer, int amount);
    default String refund(String customer) { return "refund for " + customer + " via " +
charge(customer, -1); }
}
static class RealPayments implements PaymentService {
    public String charge(String c, int amount) { return "charged " + c + " " + amount; }
}

@SuppressWarnings("unchecked")
static <T> T logging(Class<T> iface, T target, List<String> log) {
    InvocationHandler h = (proxy, m, args) -> {
        if (m.getDeclaringClass() == Object.class) return m.invoke(target, args);   //
equals/hashCode/toString
        log.add(m.getName() + Arrays.toString(args));
        try {
            return m.isDefault() ? InvocationHandler.invokeDefault(proxy, m, args) :
m.invoke(target, args);
        } catch (InvocationTargetException e) { throw e.getCause(); }               // rethrow the
real exception
    };
    return (T) Proxy.newProxyInstance(iface.getClassLoader(), new Class<?>[]{iface}, h);
}

List<String> log = new ArrayList<>();
PaymentService p = logging(PaymentService.class, new RealPayments(), log);
System.out.println(p.charge("asha", 500));
System.out.println(p.refund("ravi"));                       // default method: its inner charge()
goes through the proxy too
System.out.println("log: " + log);
System.out.println("isProxyClass=" + Proxy.isProxyClass(p.getClass()) + ", instanceof RealPayments="
+ (p instanceof RealPayments) + ", name starts with $Proxy: " +
p.getClass().getSimpleName().startsWith("$Proxy"));
```

▶ Output

```
charged asha 500
refund for ravi via charged ravi -1
log: [charge[asha, 500], refund[ravi], charge[ravi, -1]]
isProxyClass=true, instanceof RealPayments=false, name starts with $Proxy: true
```

Proxy.newProxyInstance(loader, interfaces, handler)



Every method goes to the handler — including equals, hashCode and toString.
Default methods: InvocationHandler.invokeDefault(proxy, m, args) (Java 16) runs them with this = proxy.
Unwrap InvocationTargetException, or callers get UndeclaredThrowableException instead of your exception.
Interfaces only. Classes need a generated subclass (ByteBuddy, Spring's repackaged CGLIB).

© TechTribers. For personal use only. Do not redistribute.

🎯 **Likely follow-up**

How do you proxy a class without an interface? You need a subclass generated at run time: ByteBuddy (what Mockito and Hibernate use), or CGLIB repackaged in Spring. The JDK alone can't do it.

Q774 GR-035 • Advanced

JDK proxies vs subclass proxies (CGLIB, ByteBuddy): why does self-invocation bypass `@Transactional`?

⚡ **Quick answer**

Both proxy styles wrap the target and intercept calls that come **through the proxy reference**. A call from inside the target to its own method (`this.save()`) never touches the proxy, so the interceptor (transaction, cache, security, retry, async) doesn't run. JDK proxies implement interfaces and delegate to a separate target object; subclass proxies (CGLIB/ByteBuddy, Spring's default for classes since Boot 2.0) extend the class, so they can't intercept **final** classes or methods, **private** methods or **static** methods either. Fixes: move the method to another bean, inject the bean's own proxy, or use AspectJ weaving.

The output is the bug people ship: the second internal `saveAudit` runs with no transaction and no error. In Spring the same happens with `@Transactional` , `@Cacheable` , `@Async` , `@Retryable` and method-level security whenever the annotated method is called via `this` .

How to fix it in Spring, in order of preference:

- **Split responsibilities**: put `saveAudit` in an `AuditService` bean and inject it. Usually the cleanest design anyway.
- **Self-injection**: inject the bean into itself (`@Lazy` or `ObjectProvider<OrderService>`) and call `self.saveAudit(...)` .
- **Programmatic transactions**: `TransactionTemplate.execute(...)` around the inner work.
- **AspectJ mode** (`@EnableTransactionManagement(mode = ASPECTJ)` with load-time or compile-time weaving), which modifies the class itself so `this` calls are woven.

Subclass-proxy limits: `final` classes (Kotlin classes are final by default, hence the `kotlin-spring` plugin), `final` or `private` methods (silently not intercepted), and constructors calling overridable methods (they run before the proxy is set up).

Java

```
public interface OrderService { void placeOrder(String id); void saveAudit(String id); }
static class OrderServiceImpl implements OrderService {
    public void placeOrder(String id) { System.out.println("  placeOrder " + id); saveAudit(id); }
// this.saveAudit(): bypasses the proxy
    public void saveAudit(String id) { System.out.println("  saveAudit " + id + " (tx active: " +
TX.get() + ")"); }
}
static final ThreadLocal<Boolean> TX = ThreadLocal.withInitial(() -> false);
static final Set<String> TRANSACTIONAL = Set.of("saveAudit");                    // pretend
```


© TechTribers. For personal use only. Do not redistribute.

```
saveAudit is @Transactional(REQUIRES_NEW)

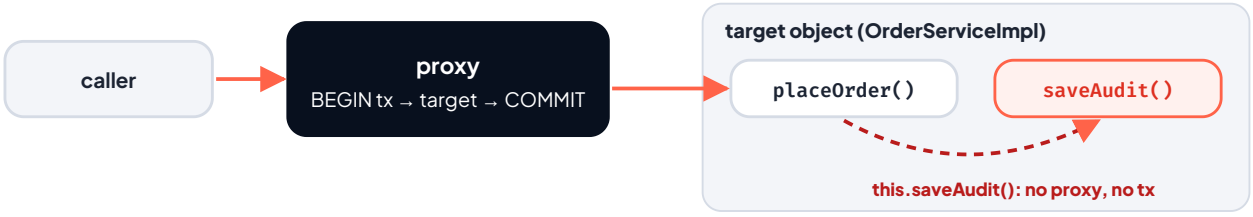
static OrderService txProxy(OrderService target) {
    return (OrderService) Proxy.newProxyInstance(OrderService.class.getClassLoader(), new Class<?>[]
{OrderService.class}, (proxy, m, args) -> {
        boolean tx = TRANSACTIONAL.contains(m.getName());
        if (tx) { TX.set(true); System.out.println("  [proxy] BEGIN tx for " + m.getName()); }
        try { return m.invoke(target, args); }
        finally { if (tx) { TX.set(false); System.out.println("  [proxy] COMMIT"); } }
    });
}

OrderService svc = txProxy(new OrderServiceImpl());
System.out.println("external call to saveAudit:"); svc.saveAudit("A-1");
System.out.println("external call to placeOrder (calls saveAudit internally):"); svc.placeOrder("A-
2");
```

▶ Output

```
external call to saveAudit:
  [proxy] BEGIN tx for saveAudit
  saveAudit A-1 (tx active: true)
  [proxy] COMMIT
external call to placeOrder (calls saveAudit internally):
  placeOrder A-2
  saveAudit A-2 (tx active: false)
```

Why this.saveAudit() skips @Transactional



Fixes: move the method to another bean · inject the proxy into itself · TransactionTemplate · AspectJ weaving.

Class proxies also skip final / private / static methods and cannot extend final classes.

⚠️ **Interview trap**

Putting `@Transactional` on a `private` method and assuming it works. Spring doesn't intercept it at all, without any warning; recent IDEs flag it, the framework does not.

© TechTribers. For personal use only. Do not redistribute.

🔧 Q775 GR-036 • Advanced

What are `MethodHandles`, and how do they differ from core reflection (`Lookup`, `invokeExact`, `invoke`, combinators)?

⚡ Quick answer

A `MethodHandle` is a typed, directly executable reference to a method, constructor or field, created from a `Lookup` object whose access rights are those of the class that created it (checked **once**, at lookup time). Each handle has a `MethodType`; `invokeExact` requires the call site's static types to match it exactly, while `invoke` adapts with casts and boxing. Handles can be transformed with **combinators** (`bindTo`, `insertArguments`, `filterReturnValue`, `guardWithTest`). When stored in a `static final` field the JIT inlines them like a direct call, which is why lambdas, string concatenation, records' `equals` and modern reflection are built on them.

Core reflection versus method handles:

- **Access checks**: `Method.invoke` checks access on every call (unless `setAccessible`); a handle checks once when the `Lookup` finds it. Passing a `Lookup` to a library is how you grant it your access rights, deliberately (that's how `MethodHandles.privateLookupIn` and frameworks with JPMS work).
- **Typing**: `invoke(Object...)` boxes everything into an array; a handle call site is compiled with its exact static signature (it's signature-polymorphic), so no array, no boxing with `invokeExact`.
- **Composition**: handles compose into new handles without generating classes, which `invokedynamic` bootstrap methods rely on.

`invokeExact`'s strictness is the most common surprise: the cast `(String)` on the call is part of the call-site type. Assigning to `Object` (as in the failing line) changes the call-site type to `(Greeter,String)Object` and throws `WrongMethodTypeException`.

Java

```
static final class Greeter {
    private final String greeting;
    Greeter(String g) { greeting = g; }
    String greet(String name) { return greeting + ", " + name; }
    private static String shout(String s) { return s.toUpperCase(); }
}

public static void main(String[] args) throws Throwable {          // invokeExact is declared to
throw Throwable
    MethodHandles.Lookup lookup = MethodHandles.lookup();
    MethodHandle greet = lookup.findVirtual(Greeter.class, "greet",
MethodType.methodType(String.class, String.class));
    MethodHandle ctor = lookup.findConstructor(Greeter.class, MethodType.methodType(void.class,
String.class));
    MethodHandle shout = lookup.findStatic(Greeter.class, "shout",
MethodType.methodType(String.class, String.class));   // private: allowed, same nest
    MethodHandle field = lookup.findGetter(Greeter.class, "greeting", String.class);
    Greeter g = (Greeter) ctor.invokeExact("Namaste");
    System.out.println("type: " + greet.type() + " -> " + (String) greet.invokeExact(g, "Asha"));
    System.out.println("field getter: " + (String) field.invokeExact(g));
    try { Object r = greet.invokeExact(g, "x"); }                                  // return type
```


© TechTribers. For personal use only. Do not redistribute.

```
Object != String
    catch (WrongMethodTypeException e) { System.out.println("invokeExact: " +
e.getMessage().replace("Main$", "")); }
    System.out.println("invoke adapts: " + greet.invoke(g, "Ravi"));
    MethodHandle loud = MethodHandles.filterReturnValue(greet.bindTo(new Greeter("hello")), shout);
// combinators
    System.out.println("bindTo + filterReturnValue: " + (String) loud.invokeExact("meera"));
    try { MethodHandles.publicLookup().findStatic(Greeter.class, "shout",
MethodType.methodType(String.class, String.class)); }
    catch (IllegalAccessException e) { System.out.println("publicLookup denied: " +
e.getMessage().split(",")[0].replace("Main$", "")); }
}
```

▶ Output

```
type: (Greeter,String)String -> Namaste, Asha
field getter: Namaste
invokeExact: handle's method type (Greeter,String)String but found (Greeter,String)Object
invoke adapts: Namaste, Ravi
bindTo + filterReturnValue: HELLO, MEERA
publicLookup denied: symbolic reference class is not accessible: class Greeter
```

💡 **Pro tip**

Use handles for framework code that calls the same members many times (serializers, mappers, expression engines). Store them in `static final` fields or `ClassValue` caches; a handle in a mutable field is several times slower (Q772).

⚡ **Q776** GR-037 • Advanced

What is `LambdaMetafactory`, and how do frameworks turn reflective calls into fast lambdas?

⚡ **Quick answer**

`LambdaMetafactory.metafactory` is the bootstrap method javac uses for every lambda and method reference. Called directly, it turns a `MethodHandle` into an instance of a **functional interface** (`Function`, `ToIntFunction`, `BiConsumer` ...) backed by a generated hidden class, so a getter or setter found once by reflection can afterwards be called through an ordinary interface call that the JIT inlines like a direct call. Measured on JDK 25: about **0.4 ns per call**, versus 2.6 to 9 ns for non-constant method handles or `Method.invoke`.

This is how libraries avoid reflection cost at steady state: Jackson's Afterburner/Blackbird modules, Hibernate's bytecode-enhanced accessors, JDBI's row mappers and several DI containers generate accessors once per property and then call plain interfaces.

Practical caveats:

- The `Lookup` must have access to the target member; pass in the caller's lookup (or `privateLookupIn`) rather than using a lookup from the library's own class.

© TechTribers. For personal use only. Do not redistribute.

- Primitive returns: `age()` returns `int`, but `Function.apply` returns `Object`. Declaring the specialised signature with the **wrapper** type (`Integer`) lets the metafactory insert the boxing; for hot primitive paths generate a `ToIntFunction` instead and skip boxing altogether.
- Each call to `metafactory` spins a new hidden class; cache the resulting lambda per member.

Java

```
public record Customer(String name, int age) { }

@SuppressWarnings("unchecked")
static <T> Function<T, Object> getter(MethodHandles.Lookup lookup, Class<T> type, String name)
throws Throwable {
    Class<?> rt = type.getMethod(name).getReturnType();
    MethodHandle target = lookup.findVirtual(type, name, MethodType.methodType(rt));
    Class<?> boxed = MethodType.methodType(rt).wrap().returnType();               // int -> Integer
    CallSite site = LambdaMetafactory.metafactory(lookup,
            "apply",                                                  // the functional method's
name
            MethodType.methodType(Function.class),                    // factory type: () ->
Function
            MethodType.methodType(Object.class, Object.class),        // Function.apply's erased
signature
            target,                                                   // the implementation
            MethodType.methodType(boxed, type));                      // specialised signature
(boxing added for us)
    return (Function<T, Object>) site.getTarget().invokeExact();
}

public static void main(String[] args) throws Throwable {          // invokeExact is declared to
throw Throwable
    Map<String, Function<Customer, Object>> accessors = new LinkedHashMap<>();
    for (RecordComponent c : Customer.class.getRecordComponents())                    //
reflection ONCE, at setup
        accessors.put(c.getName(), getter(MethodHandles.lookup(), Customer.class, c.getName()));
    for (Customer c : List.of(new Customer("Asha", 31), new Customer("Ravi", 28))) {      // plain
interface calls afterwards
        StringJoiner sj = new StringJoiner(", ", "{", "}");
        accessors.forEach((k, f) -> sj.add(k + "=" + f.apply(c)));
        System.out.println(sj);
    }
    Function<Customer, Object> name = accessors.get("name");
    System.out.println("hidden lambda class: " + name.getClass().isHidden() + ", age value type: " +
accessors.get("age").apply(new Customer("x", 5)).getClass().getSimpleName());
}
```

▶ Output

```
{name=Asha, age=31}
{name=Ravi, age=28}
hidden lambda class: true, age value type: Integer
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> The benchmark behind Q772 used exactly this technique with a `ToIntFunction<P>`: 0.39 ns per call on JDK 25, the same as a direct call.

👻 **Q777** GR-038 • Advanced

What are hidden classes (JEP 371), and how do you define one at run time?

> ⚡ **Quick answer**
> A **hidden class** (Java 15) is created from bytes with `Lookup.defineHiddenClass(bytes, initialize, options)`. It is **not discoverable by name** (`Class.forName` fails, other classes can't link to it), it's a **nestmate-capable member of the lookup class's package**, and it can be **unloaded independently** of its class loader unless defined with `ClassOption.STRONG`. Lambda classes and the JDK's generated string-concatenation helpers are hidden classes (JDK `Proxy` classes and annotation instances are **not**: we checked, `isHidden()` is `false` on JDK 21 and 25); code generators such as ByteBuddy and Groovy use them instead of the old `Unsafe.defineAnonymousClass`.

Why the JDK added them: frameworks used to generate helper classes either as normal classes in a class loader (they leak until the whole loader dies, and they pollute the namespace) or through `sun.misc.Unsafe.defineAnonymousClass`, an internal API that was removed in Java 17. Hidden classes give a supported way to define a class that is private to its creator and collectable on its own.

Properties to remember: the name has a `/` suffix with an address (`PriceRule/0x...`) that isn't a valid binary name, `getCanonicalName()` is `null`, they can't be referenced from other classes' bytecode or by reflection-by-name, and stack traces hide their frames unless you run with `-XX:+UnlockDiagnosticVMOptions -XX:+ShowHiddenFrames`.

The bytes don't have to come from `javac`. Code generators produce them directly; since JDK 24 the standard **Class-File API** (`java.lang.classfile`) is the supported way to do that without ASM.

Java

```
public static void main(String[] args) throws Throwable {          // invokeExact is declared to
throw Throwable
    Path dir = Files.createTempDirectory("hidden");
    Files.writeString(dir.resolve("PriceRule.java"), "public class PriceRule { public static long
apply(long cents) { return cents * 90 / 100; } }");
    int rc = javax.tools.ToolProvider.getSystemJavaCompiler().run(null, null, null, "-d",
dir.toString(), dir.resolve("PriceRule.java").toString());
    byte[] bytes = Files.readAllBytes(dir.resolve("PriceRule.class"));                    //
generated or compiled at run time

    MethodHandles.Lookup hiddenLookup = MethodHandles.lookup().defineHiddenClass(bytes, true);
    Class<?> rule = hiddenLookup.lookupClass();
    MethodHandle apply = hiddenLookup.findStatic(rule, "apply", MethodType.methodType(long.class,
long.class));
    System.out.println("javac rc=" + rc + ", result: " + (long) apply.invokeExact(10_000L));
    System.out.println("isHidden=" + rule.isHidden() + ", name=" +
rule.getName().replaceAll("0x\\p{XDigit}+", "0x...") + ", canonical name=" +
```


© TechTribers. For personal use only. Do not redistribute.

```
rule.getCanonicalName());
    try { Class.forName(rule.getName()); } catch (ClassNotFoundException e) {
System.out.println("Class.forName: ClassNotFoundException"); }
    Runnable lambda = () -> { };
    System.out.println("lambdas are hidden too: " + lambda.getClass().isHidden());
}
```

▶ Output

```
javac rc=0, result: 9000
isHidden=true, name=PriceRule/0x..., canonical name=null
Class.forName: ClassNotFoundException
lambdas are hidden too: true
```

🎯 **Likely follow-up**

Can a hidden class access private members of the class that defined it? Yes, if defined with `ClassOption.NESTMATE`; it then joins the lookup class's nest, which is exactly how lambda bodies (private synthetic methods) are called from lambda classes.

🧳 **Q778** GR-039 • Advanced

Why can't the same class loaded by two class loaders be cast to itself, and how do you compile and load code at run time?

⚡ **Quick answer**

At run time a class is identified by its **name plus its defining class loader**. Two loaders that each define `com.acme.Plugin` from the same bytes produce **two different** `Class` **objects**, so an instance of one is not an instance of the other: casting fails with the famous `ClassCastException: Plugin cannot be cast to Plugin`. That's the basis of isolation in app servers and plugin systems, and also the cause of many hot-reload bugs. Compiling at run time is `javax.tools.JavaCompiler` plus a `URLClassLoader` (or an in-memory loader) to load the output.

The working line is the design rule for every plugin system: the host and the plugins must share their **API types** through a common parent loader (here `Supplier`, from the bootstrap loader), and only implementation classes live in the child loaders. Talk to plugins only through those shared interfaces.

Where you meet the failure in practice:

- **Spring Boot DevTools** restarts the app in a new "restart" loader; a library holding objects created before the restart (a cache, a session) produces `X cannot be cast to X`.
- **App servers / Tomcat**: a JAR duplicated in both `WEB-INF/lib` and the server's `lib` gives two copies of the same class.
- **Deserialization** with the wrong thread context class loader.

Diagnosing it: compare `getClassLoader()` of both classes, or run with `-Xlog:class+load` to see which loader defined each copy.

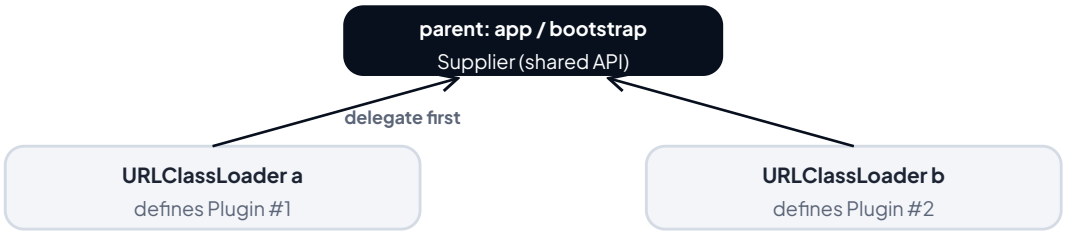
© TechTribers. For personal use only. Do not redistribute.

```java
Path dir = Files.createTempDirectory("plugins");
Files.writeString(dir.resolve("Plugin.java"), "public class Plugin implements
java.util.function.Supplier<String> { public String get() { return \"plugin v1\"; } }");
int rc = javax.tools.ToolProvider.getSystemJavaCompiler().run(null, null, null, "-d",
dir.toString(), dir.resolve("Plugin.java").toString());
URL[] cp = { dir.toUri().toURL() };
try (URLClassLoader a = new URLClassLoader(cp, Main.class.getClassLoader());
     URLClassLoader b = new URLClassLoader(cp, Main.class.getClassLoader())) {
    Class<?> fromA = a.loadClass("Plugin"), fromB = b.loadClass("Plugin");
    System.out.println("javac rc=" + rc + ", same name: " + fromA.getName().equals(fromB.getName())
+ ", same Class: " + (fromA == fromB));
    Object pluginA = fromA.getDeclaredConstructor().newInstance();
    Supplier<?> s = (Supplier<?>) pluginA;                              // works: Supplier comes
from the shared parent
    System.out.println("via shared interface: " + s.get());
    System.out.println("isInstance across loaders: " + fromB.isInstance(pluginA));
    try { fromB.cast(pluginA); } catch (ClassCastException e) { System.out.println("cast: " +
e.getMessage()); }
    System.out.println("loaders: " + (fromA.getClassLoader() == a) + " " + (fromB.getClassLoader()
== b) + ", Supplier from bootstrap: " + (Supplier.class.getClassLoader() == null));
}
```

▶ Output

```
javac rc=0, same name: true, same Class: false
via shared interface: plugin v1
isInstance across loaders: false
cast: Cannot cast Plugin to Plugin
loaders: true true, Supplier from bootstrap: true
```

A class = binary name + defining class loader



`Plugin #1 ≠ Plugin #2 → ClassCastException: Cannot cast Plugin to Plugin`

Talk to plugins only through types from a COMMON parent loader (here Supplier): that cast works.

💡 **Pro tip**

Close `URLClassLoader` s you create (they're `Closeable` since Java 7), and drop every reference to the classes and instances they loaded, or the loader and all its classes stay in metaspace.

© TechTribers. For personal use only. Do not redistribute.

Q779 GR-040

• Intermediate

What is `ServiceLoader` (SPI), and how do provider files and module `provides` / `uses` work?

⚡ Quick answer

`ServiceLoader` discovers implementations of a **service interface** at run time, without the caller naming them. On the class path, a provider JAR lists implementation class names in `META-INF/services/<fully.qualified.Interface>`; in the module system, the provider module declares `provides Service with Impl;` and the consumer `uses Service;`. `ServiceLoader.load(Service.class)` instantiates providers lazily as you iterate, and `stream()` lets you inspect provider types before instantiating. JDBC drivers, `java.time` zone providers, SLF4J 2 bindings and Jackson modules are discovered this way.

The snippet writes a provider-configuration file into a temporary directory and points a class loader at it; on a real project that file sits in `src/main/resources/META-INF/services/` of the provider JAR, or is replaced by `provides ... with ...` in `module-info.java` (and then the provider class can even expose a `public static provider()` method instead of a public no-arg constructor).

Details interviewers probe:

- **Lazy instantiation**: providers are created during iteration; a broken provider throws `ServiceConfigurationError` at that point, not at `load`.
- **Caching**: a `ServiceLoader` instance caches providers it has created; `reload()` clears the cache. Each `load` call returns a new loader.
- **Class loader**: `ServiceLoader.load(type)` uses the **thread context class loader**, which matters in containers. Pass a loader explicitly when you know which one should see the providers.
- `DriverManager` uses `ServiceLoader` to find JDBC drivers, which is why `Class.forName("com.mysql.cj.jdbc.Driver")` has been unnecessary since JDBC 4.0.

Java

```
public interface PaymentGateway { String name(); boolean supports(String currency); }
public static class RazorpayGateway implements PaymentGateway { public String name() { return
"razorpay"; } public boolean supports(String c) { return c.equals("INR"); } }
public static class StripeGateway implements PaymentGateway { public String name() { return
"stripe"; } public boolean supports(String c) { return !c.equals("INR"); } }

Path root = Files.createTempDirectory("spi");
Path services = Files.createDirectories(root.resolve("META-INF/services"));
Files.writeString(services.resolve(PaymentGateway.class.getName()),
        "# one provider class name per line\n" + RazorpayGateway.class.getName() + "\n" +
StripeGateway.class.getName() + "\n");
try (URLClassLoader loader = new URLClassLoader(new URL[]{root.toUri().toURL()},
Main.class.getClassLoader())) {
    ServiceLoader<PaymentGateway> sl = ServiceLoader.load(PaymentGateway.class, loader);
    System.out.println("provider types (not yet instantiated): " + sl.stream().map(p ->
p.type().getSimpleName()).toList());
    for (String currency : List.of("INR", "USD")) {
        PaymentGateway gw = sl.stream().map(ServiceLoader.Provider::get).filter(g ->
g.supports(currency)).findFirst().orElseThrow();
```


© TechTribers. For personal use only. Do not redistribute.

```
        System.out.println(currency + " -> " + gw.name());
    }
}
```

▶ Output

```
provider types (not yet instantiated): [RazorpayGateway, StripeGateway]
INR -> razorpay
USD -> stripe
```

🎯 **Likely follow-up**

How does Spring Boot's auto-configuration differ? It uses its own file, `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` (Boot 2.7+; earlier `spring.factories`), read by Spring's own loader, not `java.util.ServiceLoader`, because it needs conditions and ordering.

Q780 GR-041 • Intermediate

When should you avoid reflection? (AOT and native images, refactoring safety, performance, encapsulation)

⚡ **Quick answer**

Avoid reflection when ordinary code can express the same thing. It costs **compile-time safety** (names are strings; renames break at run time), **encapsulation** (it bypasses access control, increasingly blocked by JPMS and JEP 500), **performance** at lookup time, and **AOT friendliness**: GraalVM native images and Spring AOT must know reflective targets at build time, so dynamic reflection needs reachability metadata or fails at run time. Prefer interfaces, method references, `MethodHandles` with explicit `Lookup`s, records with pattern matching, annotation processing (compile-time code generation), or a registry of lambdas.

The registry version is the same length, fails with a useful message, survives renames in an IDE, is visible to static analysis and "find usages", and needs no native-image configuration.

When reflection **is** the right tool: frameworks and libraries that operate on types they've never seen (serializers, ORMs, DI containers, test runners), developer tooling, and one-off diagnostics. Even there, the trend is to move work to build time: Micronaut, Quarkus and Dagger generate code with annotation processors; Spring Boot 3 has AOT processing that turns bean definitions into generated code for native images; Jackson and Hibernate can generate accessors.

GraalVM native image specifics: reflection on classes not registered in `reflect-config.json` (now part of `reachability-metadata.json`) fails at run time with `ClassNotFoundException` / `NoSuchMethodException` even though it worked on the JVM, which is a common first-deployment surprise.

Java

```
public static class ReportJobs {
    public String daily()  { return "daily report"; }
    public String weekly() { return "weekly report"; }
```


© TechTribers. For personal use only. Do not redistribute.

```
}

static String runByReflection(ReportJobs jobs, String name) throws Exception {
    return (String) ReportJobs.class.getMethod(name).invoke(jobs);          // stringly typed:
breaks on rename
}

static final Map<String, Function<ReportJobs, String>> JOBS = Map.of(       // checked at compile
time
        "daily", ReportJobs::daily,
        "weekly", ReportJobs::weekly);

static String runByRegistry(ReportJobs jobs, String name) {
    Function<ReportJobs, String> job = JOBS.get(name);
    if (job == null) throw new IllegalArgumentException("unknown job " + name + ", known: " + new
TreeSet<>(JOBS.keySet()));
    return job.apply(jobs);
}

ReportJobs jobs = new ReportJobs();
System.out.println(runByReflection(jobs, "daily") + " | " + runByRegistry(jobs, "weekly"));
try { runByReflection(jobs, "monthly"); } catch (NoSuchMethodException e) {
System.out.println("reflection: " + e.getClass().getSimpleName() + " " +
e.getMessage().replace("Main$", "")); }
try { runByRegistry(jobs, "monthly"); } catch (IllegalArgumentException e) {
System.out.println("registry: " + e.getMessage()); }
```

▶ Output

```
daily report | weekly report
reflection: NoSuchMethodException ReportJobs.monthly()
registry: unknown job monthly, known: [daily, weekly]
```

> 💡 **Pro tip**
> In code review, flag `getMethod("...")`, `getDeclaredField("...")` and `Class.forName("...")` with string literals in business code. There's almost always a typed alternative.

Q781 GR-042 • Advanced

Coding: write a reflective object differ for audit logging ("what changed between two versions of an entity?").

> ⚡ **Quick answer**
> Walk all instance fields of the class and its superclasses (cached per class), make them accessible, compare the old and new values with `Objects.deepEquals` (so arrays compare by content), recurse into nested value objects up to a depth limit, skip fields marked as excluded (secrets, timestamps) and return a list of `path: old -> new` changes. Records can be handled through their accessors so no private-field access is needed.

Design choices that make it production-worthy:

© TechTribers. For personal use only. Do not redistribute.

- **Value-type boundary**: JDK types (`String` , `BigDecimal` , `LocalDate` , collections) are compared as values with `deepEquals` ; only your own types are recursed into. That avoids walking into `String` 's internals (which JPMS would block anyway).
- **Arrays by content** (`tags` didn't change, so it isn't reported) and a **depth limit** against cycles. A real version also tracks visited objects with an `IdentityHashMap` .
- **Exclusion by annotation** keeps secrets out of audit logs (`passwordHash` changed but isn't reported).
- Cache the field list per class with `ClassValue` (Q772) when diffing at volume.

Alternatives: Hibernate Envers and Javers do this at the persistence layer; for records, comparing component accessors avoids field access entirely.

Java

```
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.FIELD) @interface NoAudit { }
static class Address { String city; String pin; Address(String c, String p) { city = c; pin = p; } }
static class BaseEntity { long id; int version; }
static class Customer extends BaseEntity {
    String name; Address address; String[] tags; @NoAudit String passwordHash;
    Customer(long id, int v, String n, Address a, String[] t, String h) { this.id = id; version = v;
name = n; address = a; tags = t; passwordHash = h; }
}

static List<String> diff(Object before, Object after) throws IllegalAccessException { List<String>
out = new ArrayList<>(); diff("", before, after, out, 0); return out; }

static void diff(String path, Object a, Object b, List<String> out, int depth) throws
IllegalAccessException {
    if (a == null || b == null || depth > 3 || a.getClass().getName().startsWith("java.")) {
        if (!Objects.deepEquals(a, b)) out.add(path + ": " + show(a) + " -> " + show(b));
        return;
    }
    for (Class<?> c = a.getClass(); c != Object.class; c = c.getSuperclass())
        for (Field f : c.getDeclaredFields()) {
            if (Modifier.isStatic(f.getModifiers()) || f.isSynthetic() ||
f.isAnnotationPresent(NoAudit.class) || !f.trySetAccessible()) continue;
            diff(path.isEmpty() ? f.getName() : path + "." + f.getName(), f.get(a), f.get(b), out,
depth + 1);
        }
}

static String show(Object o) { return o instanceof Object[] arr ? Arrays.toString(arr) :
String.valueOf(o); }

Customer v1 = new Customer(7, 1, "Asha", new Address("Pune", "411057"), new String[]{"gold"}, "h1");
Customer v2 = new Customer(7, 2, "Asha K", new Address("Pune", "411045"), new String[]{"gold"},
"h2");
diff(v1, v2).forEach(System.out::println);
```

▶ Output

```
name: Asha -> Asha K
address.pin: 411057 -> 411045
version: 1 -> 2
```


© TechTribers. For personal use only. Do not redistribute.

🎯 **Likely follow-up**

How would you diff collections? Lists by index or by an identity key (`id`), sets and maps by key, reporting `added` / `removed` / `changed` entries. That's where Javers earns its keep.

Section 05

Annotations I: Declaring, Meta-Annotations and Reading Them



How annotation types are declared and what javac enforces, the built-in annotations, retention and targets proven with javap, @Inherited and repeatable annotations, the present/declared/associated lookup rules, what an annotation object really is, type-use annotations with JSpecify, and how record annotations propagate.

Questions 782–791 · 10 questions

✍️ **Q782** GR-043 • Intermediate

How do you declare a custom annotation: element types, defaults, the `value` shorthand and the constant rules?

⚡ **Quick answer**

An annotation type is declared with `@interface` ; its **elements** are abstract-looking methods with no parameters, optionally with a `default` . Element types are restricted to **primitives**, `String` , `Class` **(or** `Class<? extends X>` **), enums, other annotations, and one-dimensional arrays of these**. Values must be **compile-time constants**, and `null` is never allowed, not even as a default. If an element is named `value` and it's the only one you set, you can omit the name: `@Table("orders")` ; a single-element array can drop its braces: `@Roles("ADMIN")` .

Each rule exists because annotation values are stored as **constants in the class file** (the `RuntimeVisibleAnnotations` attribute holds constant-pool entries, class names, enum names and nested arrays). Anything that would need code to run, such as `System.currentTimeMillis()` or a `List` , can't be stored. `PREFIX + "orders"` works because a `static final String` initialised with a literal is a constant expression.

Design guidelines:

- Give every optional element a sensible `default` ; use "" or an empty array as the "not set" sentinel, since `null` is impossible.
- Reserve `value` for the one element callers set most often.
- Prefer `Class<? extends X>` over `String` class names: it's checked by the compiler and survives renames.

© TechTribers. For personal use only. Do not redistribute.

Java

```java
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real diagnostics (helper shown in full in Q750)
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.TYPE)
@interface Table {
    String value();                                  // required: no default
    String schema() default "public";
    int version() default 1;
    Class<? extends Number> idType() default Long.class;
    String[] indexes() default { };
}
@Table(value = "orders", indexes = "idx_customer") static class Order { }
@Table("audit_log") static class AuditLog { }                    // value shorthand

for (Class<?> c : List.of(Order.class, AuditLog.class)) {
    Table t = c.getAnnotation(Table.class);
    System.out.println(c.getSimpleName() + " -> " + t.schema() + "." + t.value() + " v" +
t.version() + " id=" + t.idType().getSimpleName() + " indexes=" + Arrays.toString(t.indexes()));
}
String[] cases = {
    "@interface A { Object value(); }",
    "@interface A { String value() default null; }",
    "@interface A { long at() default System.currentTimeMillis(); }",
    "@interface A { String name(); } @A(\"x\") class Use { }",
    "@interface A { List<String> items(); }",
    "@interface A { int[][] grid(); }",
    "static final String PREFIX = \"api_\"; @interface A { String value() default PREFIX +
\"orders\"; }"
};
for (String c : cases) System.out.println(c + "\n   -> " + javac(c));
```

▶ Output

```
Order -> public.orders v1 id=Long indexes=[idx_customer]
AuditLog -> public.audit_log v1 id=Long indexes=[]
@interface A { Object value(); }
   -> error: invalid type for annotation interface element
@interface A { String value() default null; }
   -> error: element value must be a constant expression
@interface A { long at() default System.currentTimeMillis(); }
   -> error: expression not allowed as annotation value
@interface A { String name(); } @A("x") class Use { }
   -> error: cannot find symbol | error: annotation @T.A is missing a default value for the element
'name'
@interface A { List<String> items(); }
   -> error: invalid type for annotation interface element
@interface A { int[][] grid(); }
   -> error: invalid type for annotation interface element
static final String PREFIX = "api_"; @interface A { String value() default PREFIX + "orders"; }
   -> compiles
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
"An annotation can have a `null` default to mean not set." It can't (second case). Frameworks use sentinels instead, for example Spring's `ValueConstants.DEFAULT_NONE` in `@RequestParam(defaultValue = ...)`, and JUnit 4's `@Test(expected = Test.None.class)`.

Q783 GR-044 • Basic

What do the built-in annotations `@Override`, `@FunctionalInterface`, `@Deprecated` and `@SuppressWarnings` actually enforce?

⚡ **Quick answer**
`@Override` makes javac **fail** if the method doesn't override or implement anything (catching typos and signature drift). `@FunctionalInterface` makes javac **fail** if the interface doesn't have exactly one abstract method. `@Deprecated` doesn't change behaviour; it makes **callers** get a warning, and with `forRemoval = true` a stronger **removal** warning that is on by default. `@SuppressWarnings("unchecked" | "deprecation" | "removal" | "rawtypes" ...)` silences named javac lint categories for the annotated element. `@SafeVarargs` is covered in Q754.

Notes on each:

- `@Override`: the first case is the classic bug it prevents: `save(String)` is an overload, not an override, so polymorphic calls through `Base` never reach it.
- `@FunctionalInterface`: default methods and methods that are public in `Object` (`equals`, `hashCode`, `toString`) don't count as abstract methods, so the third case is a valid functional interface.
- `@Deprecated` (Java 9 added `since` and `forRemoval`): an ordinary deprecation (`Thread.getId()`, deprecated in 19) is reported in detail only with `-Xlint:deprecation`; a **removal** warning (`new Integer(5)`, deprecated for removal since 16) is reported by default, because the API will disappear. `jdeprscan` scans JARs for uses of deprecated JDK APIs.
- javac **never** warns about uses of a deprecated member from inside the same outermost class. Our first version of this snippet declared the deprecated API next to its caller and got no warnings at all, which is why it now uses JDK methods.
- `@SuppressWarnings` works on declarations only (class, method, field, local variable, parameter), so its narrowest scope is a local variable declaration. Common keys: `unchecked`, `rawtypes`, `deprecation`, `removal`, `serial`, `this-escape` (Java 21).

Java

```
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real
diagnostics (helper shown in full in Q750)
String base = "static class Base { void save(Object o) { } } ";
String[] cases = {
    base + "static class Repo extends Base { @Override void save(String s) { } }",
    "@FunctionalInterface interface Handler { void handle(); void close(); }",
    "@FunctionalInterface interface Handler { void handle(); default void close() { } boolean
equals(Object o); }",
    "long id() { return Thread.currentThread().getId(); }",
```


© TechTribers. For personal use only. Do not redistribute.

```
    "Integer box() { return new Integer(5); }",
    "@SuppressWarnings(\"removal\") Integer box() { return new Integer(5); }"
};
for (String c : cases) System.out.println(c.replace(base, "") + "\n   default: " + javac(c) + "\n
-Xlint:deprecation: " + javac(c, "-Xlint:deprecation"));
Deprecated d = Thread.class.getMethod("stop").getAnnotation(Deprecated.class);
System.out.println("Thread.stop: since=" + d.since() + " forRemoval=" + d.forRemoval());
```

▶ Output

```
static class Repo extends Base { @Override void save(String s) { } }
   default: error: method does not override or implement a method from a supertype
   -Xlint:deprecation: error: method does not override or implement a method from a supertype
@FunctionalInterface interface Handler { void handle(); void close(); }
   default: error: Unexpected @FunctionalInterface annotation
   -Xlint:deprecation: error: Unexpected @FunctionalInterface annotation
@FunctionalInterface interface Handler { void handle(); default void close() { } boolean
equals(Object o); }
   default: compiles
   -Xlint:deprecation: compiles
long id() { return Thread.currentThread().getId(); }
   default: compiles
   -Xlint:deprecation: warning: getId() in java.lang.Thread has been deprecated
Integer box() { return new Integer(5); }
   default: warning: Integer(int) in java.lang.Integer has been deprecated and marked for removal
   -Xlint:deprecation: warning: Integer(int) in java.lang.Integer has been deprecated and marked for
removal
@SuppressWarnings("removal") Integer box() { return new Integer(5); }
   default: compiles
   -Xlint:deprecation: compiles
Thread.stop: since=1.2 forRemoval=true
```

💡 **Pro tip**

Pair `@Deprecated` with a Javadoc `@deprecated` tag naming the replacement. The annotation drives tooling; the tag tells humans what to do.

⏳ **Q784** GR-045 • Intermediate

What is `@Retention` (`SOURCE`, `CLASS`, `RUNTIME`), and why can't reflection see most annotations?

⚡ **Quick answer**

Retention decides **how long** an annotation survives. `SOURCE`: discarded by javac (only the compiler and annotation processors see it; `@Override`, `@SuppressWarnings`, Lombok's annotations). `CLASS` (the **default** when no `@Retention` is present): written to the class file as *invisible* annotations, readable by bytecode tools but **not** by reflection. `RUNTIME`: written as *visible* annotations and returned by reflection. A framework annotation that reflection must read **has to** be `RUNTIME`; forgetting `@Retention` is the most common reason a custom annotation "doesn't work".

© TechTribers. For personal use only. Do not redistribute.

The snippet compiles a tiny class at run time and runs the real `javap` (through `java.util.spi.ToolProvider`) on it. The class file contains a `RuntimeVisibleAnnotations` attribute (for `@Rt`) and a `RuntimeInvisibleAnnotations` attribute (for `@Cls`); `@Src` doesn't exist in the bytecode at all.

Who uses `CLASS` retention? Tools that read bytecode without loading classes: build-time class scanners (Jandex in Quarkus), bytecode enhancers, static analysers such as SpotBugs (`@SuppressFBWarnings` is `CLASS`), and JSpecify's nullness annotations are `RUNTIME` precisely so that both tools and reflection can see them.

The JVM ignores `CLASS` annotations completely at class-loading time, which is why the default is "cheap": no reflection cost, but still available to tools.

Java

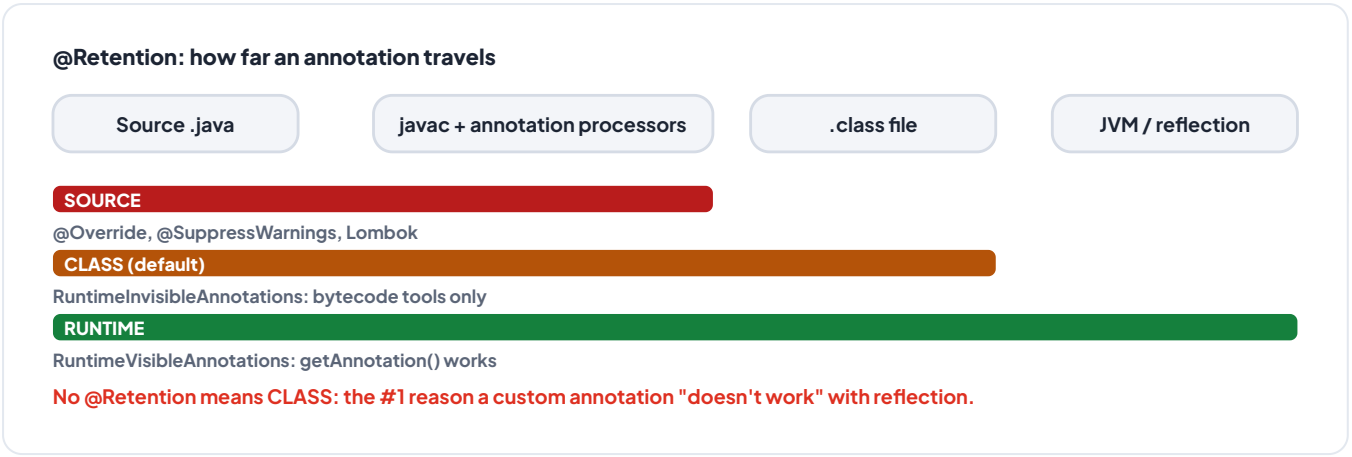
```
@Retention(RetentionPolicy.SOURCE) @interface Src { }
@Retention(RetentionPolicy.CLASS) @interface Cls { }
@interface NoRetention { }                                    // default = CLASS
@Retention(RetentionPolicy.RUNTIME) @interface Rt { }
@Src @Cls @NoRetention @Rt static class Target { }

System.out.println("reflection sees: " + Arrays.stream(Target.class.getAnnotations()).map(a ->
a.annotationType().getSimpleName()).toList());
Path dir = Files.createTempDirectory("ret");
Files.writeString(dir.resolve("T.java"), """
        import java.lang.annotation.*;
        @Retention(RetentionPolicy.SOURCE) @interface Src { }
        @Retention(RetentionPolicy.CLASS) @interface Cls { }
        @Retention(RetentionPolicy.RUNTIME) @interface Rt { }
        @Src @Cls @Rt public class T { }
        """);
javax.tools.ToolProvider.getSystemJavaCompiler().run(null, null, null, "-d", dir.toString(),
dir.resolve("T.java").toString());
StringWriter javap = new StringWriter();
java.util.spi.ToolProvider.findFirst("javap").orElseThrow().run(new PrintWriter(javap), new
PrintWriter(System.err), "-v", dir.resolve("T.class").toString());
javap.toString().lines().map(String::strip).filter(l -> l.startsWith("Runtime") || l.matches("\\d+:
#\\d+\\(\\)")).forEach(l -> System.out.println("javap: " + l));
javap.toString().lines().filter(l -> l.matches("\\s*#\\d+ =
Utf8\\s+L\\w+;")).map(String::strip).forEach(l -> System.out.println("javap: constant " +
l.replaceAll("\\s+", " ")));
```

▶ Output

```
reflection sees: [Rt]
javap: RuntimeVisibleAnnotations:
javap: 0: #14()
javap: RuntimeInvisibleAnnotations:
javap: 0: #16()
javap: constant #14 = Utf8 LRt;
javap: constant #16 = Utf8 LCls;
```


© TechTribers. For personal use only. Do not redistribute.



⚠️ **Interview trap**

Writing a custom `@Audited` annotation, checking `method.isAnnotationPresent(Audited.class)` in an aspect, getting `false` every time, and debugging the aspect for an hour. The annotation has no `@Retention`, so it's `CLASS`.

🎯 **Q785** GR-046 • Intermediate

What is `@Target`, and which `ElementType` values exist (including `TYPE_USE`, `TYPE_PARAMETER`, `RECORD_COMPONENT`, `MODULE`)?

⚡ **Quick answer**

`@Target` lists the **kinds of declarations or type uses** an annotation may be applied to; javac rejects other placements. Without `@Target`, an annotation is allowed on **all declaration contexts** (since Java 17 the JLS spells this out; type-use contexts are excluded). `ElementType` covers `TYPE` (classes, interfaces, enums, records, annotation types), `FIELD`, `METHOD`, `PARAMETER`, `CONSTRUCTOR`, `LOCAL_VARIABLE`, `ANNOTATION_TYPE`, `PACKAGE`, `TYPE_PARAMETER` and `TYPE_USE` (Java 8), `MODULE` (Java 9) and `RECORD_COMPONENT` (Java 16).

`TYPE_USE` is the powerful one: it allows an annotation **anywhere a type is written**, including type arguments (`List<@NonEmpty String>`), casts, `new` expressions, `throws` clauses, array dimensions and type-parameter bounds. It is what pluggable type checkers (Checker Framework, NullAway, JSpecify) and Bean Validation 2.0's container element constraints (`List<@Email String>`) build on.

An annotation that targets **both** a declaration kind and `TYPE_USE`, for example `{FIELD, TYPE_USE}`, can be ambiguous on a field: JLS 9.7.4 says it applies to both the field and its type. Validation frameworks rely on the exact target to decide whether `@NotNull` constrains the container or its elements.

Note `TYPE_PARAMETER` vs `TYPE_USE`: `<@A T>` needs `TYPE_PARAMETER` (or no `@Target`); an annotation with only `TYPE_USE` is rejected there, as the last case shows.

```java
// javac(body, opts...) compiles a class body in memory with javax.tools and returns the real diagnostics (helper shown in full in Q750)
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(Arrays.toString(ElementType.values()));
String decl = "@Target(ElementType.METHOD) @interface OnlyMethods { } @Target(ElementType.TYPE_USE)
@interface NonEmpty { } @interface Anywhere { } ";
String[] cases = {
    "@OnlyMethods String field;",
    "@OnlyMethods void run() { }",
    "List<@NonEmpty String> names;",
    "List<@Anywhere String> names;",
    "@Anywhere String field; @Anywhere void run(@Anywhere int x) { @Anywhere int local = 1; }",
    "void run() { Object o = (@NonEmpty String) \"x\"; }",
    "<@NonEmpty T> void generic() { }"
};
for (String c : cases) System.out.println(c + "\n   -> " + javac(decl + c));
```

▶ Output

```
[TYPE, FIELD, METHOD, PARAMETER, CONSTRUCTOR, LOCAL_VARIABLE, ANNOTATION_TYPE, PACKAGE,
TYPE_PARAMETER, TYPE_USE, MODULE, RECORD_COMPONENT]
@OnlyMethods String field;
   -> error: annotation interface not applicable to this kind of declaration
@OnlyMethods void run() { }
   -> compiles
List<@NonEmpty String> names;
   -> compiles
List<@Anywhere String> names;
   -> error: annotation @T.Anywhere not applicable in this type context
@Anywhere String field; @Anywhere void run(@Anywhere int x) { @Anywhere int local = 1; }
   -> compiles
void run() { Object o = (@NonEmpty String) "x"; }
   -> compiles
<@NonEmpty T> void generic() { }
   -> compiles
```

💡 **Pro tip**

Always declare `@Target` explicitly. It documents intent, prevents misuse, and stops a later JDK's new element types from silently becoming valid places for your annotation.

Q786 GR-047 • Intermediate

How does `@Inherited` work, and what does it not inherit?

⚡ **Quick answer**

`@Inherited` is a meta-annotation that makes `Class.getAnnotation` look **up the superclass chain** for a class-level annotation the subclass doesn't declare itself. That's all it does. It has **no effect** on annotations on **interfaces** (an implementing class never inherits them), on **methods, fields or parameters** (an overriding method doesn't inherit its parent's annotations), and on annotations that aren't marked `@Inherited`. Frameworks that need more (Spring's `@Transactional` on an interface method) do their own hierarchy search.

© TechTribers. For personal use only. Do not redistribute.

The last line is the subtle one: `UserController` implements `Admin`, which is annotated `@Secured(role = "OPS")`, but `getAnnotation` returns the **superclass's** `ADMIN` value; interfaces are never consulted.

That's why Spring ships `AnnotatedElementUtils.findMergedAnnotation` and `MergedAnnotations.from(element, SearchStrategy.TYPE_HIERARCHY)`: they search superclasses, interfaces, overridden methods (including interface methods), bridge methods and meta-annotations. When someone asks why `@Transactional` on an interface method works with JDK proxies but has historically been fragile with class proxies, this is the background.

Java

```
import java.lang.annotation.Annotation;                                    // the harness also imports
java.text.*, which has its own Annotation
@Inherited @Retention(RetentionPolicy.RUNTIME) @interface Secured { String role(); }
@Retention(RetentionPolicy.RUNTIME) @interface Audited { }
@Secured(role = "ADMIN") @Audited static class BaseController { @Secured(role = "OPS") public void
restart() { } }
@Secured(role = "OPS") interface Admin { }
static class UserController extends BaseController implements Admin { @Override public void
restart() { } }

static String has(AnnotatedElement e, Class<? extends Annotation> a) { return a.getSimpleName() +
"=" + (e.getAnnotation(a) != null); }

System.out.println("class, @Inherited superclass annotation: " + has(UserController.class,
Secured.class) + " role=" + UserController.class.getAnnotation(Secured.class).role());
System.out.println("class, non-@Inherited annotation:        " + has(UserController.class,
Audited.class));
System.out.println("declared only (no inheritance):          " +
UserController.class.getDeclaredAnnotations().length + " annotations");
System.out.println("overriding method:                       " +
has(UserController.class.getMethod("restart"), Secured.class));
System.out.println("annotation on implemented interface: role found = " +
UserController.class.getAnnotation(Secured.class).role() + " (from BaseController, not Admin)");
```

▶ Output

```
class, @Inherited superclass annotation: Secured=true role=ADMIN
class, non-@Inherited annotation:        Audited=false
declared only (no inheritance):          0 annotations
overriding method:                       Secured=false
annotation on implemented interface: role found = ADMIN (from BaseController, not Admin)
```

⚠️ **Interview trap**

Expecting a subclass's **overriding method** to keep the parent method's `@Secured` or `@Cacheable`. With plain reflection it doesn't, `@Inherited` or not. Re-annotate, or use a framework utility that searches the method hierarchy.

© TechTribers. For personal use only. Do not redistribute.

🔁 **Q787** GR-048 • Advanced

How do repeatable annotations work, and why does `getAnnotation` return `null` for repeated ones?

⚡ **Quick answer**

Since Java 8, an annotation type marked `@Repeatable(Container.class)` can appear several times on one element. javac **wraps** the repeats into the **container annotation** (`@Schedules({@Schedule(...), @Schedule(...)})`), so the class file contains only the container. Consequently `getAnnotation(Schedule.class)` returns `null` when there are two or more, but the annotation itself when there's only one. Use `getAnnotationsByType(Schedule.class)` , which looks through containers and works for both cases.

Rules javac checks for a valid container: it must have a `value()` element returning an array of the repeatable type, a retention at least as long, and a target that is a subset of the repeatable annotation's targets, and it must be `@Documented` / `@Inherited` consistently.

Why this design: it kept the class-file format and every pre-Java-8 reflection API unchanged; old tools still see just one annotation (the container). Real repeatable annotations you'll meet: Spring's `@PropertySource` , `@ComponentScan` , `@Scheduled` , JUnit 5's `@Tag` and `@ExtendWith` , and JPA's `@NamedQuery` (with `@NamedQueries` as the container).

Java

```
@Retention(RetentionPolicy.RUNTIME) @Repeatable(Schedules.class) @interface Schedule { String
cron(); }
@Retention(RetentionPolicy.RUNTIME) @interface Schedules { Schedule[] value(); }
static class Jobs {
    @Schedule(cron = "0 0 2 * * *") public void backup() { }
    @Schedule(cron = "0 */15 * * * *") @Schedule(cron = "0 0 9 * * MON") public void report() { }
}

for (String name : List.of("backup", "report")) {
    Method m = Jobs.class.getMethod(name);
    Schedule single = m.getAnnotation(Schedule.class);
    Schedules container = m.getAnnotation(Schedules.class);
    System.out.println(name + ": getAnnotation(Schedule)=" + (single == null ? "null" :
single.cron())
            + ", container=" + (container == null ? "null" : container.value().length + " inside")
            + ", byType=" +
Arrays.stream(m.getAnnotationsByType(Schedule.class)).map(Schedule::cron).toList());
}
```

▶ **Output**

```
backup: getAnnotation(Schedule)=0 0 2 * * *, container=null, byType=[0 0 2 * * *]
report: getAnnotation(Schedule)=null, container=2 inside, byType=[0 */15 * * * *, 0 0 9 * * MON]
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
A framework that reads `getAnnotation(Schedule.class)` works in every test that has one schedule and silently ignores the method the day someone adds a second one. Always use `getAnnotationsByType` (or `getDeclaredAnnotationsByType`) for repeatable types.

Q788 GR-049 • Advanced

What is the difference between `getAnnotations`, `getDeclaredAnnotations` and `getAnnotationsByType` (present, directly present, associated)?

⚡ **Quick answer**
The `AnnotatedElement` Javadoc defines four kinds of presence. **Directly present**: written on the element itself. **Indirectly present**: inside a container on the element (repeatable). **Present**: directly present, or inherited through `@Inherited` from a superclass. **Associated**: directly or indirectly present, or inherited. `getDeclaredAnnotation(s)` returns **directly present**; `getAnnotation(s)` returns **present**; `getDeclaredAnnotationsByType` returns directly or indirectly present; `getAnnotationsByType` returns **associated**. For anything except classes, inheritance never applies, so the declared and non-declared forms are equal.

Read lines two and four together, because they disagree. `getAnnotations()` returns the subclass's own `@Tag("refunds")` **and** the superclass's inherited `@Tags` container with two more tags, since `Tag` and `Tags` are different annotation types. `getAnnotationsByType(Tag.class)` returns only `refunds`: for repeatable annotations, inherited ones are consulted only when the subclass has **none** of its own. Code that walks `getAnnotations()` and unpacks containers itself will therefore see three tags where `getAnnotationsByType` sees one. On `BaseService` both repeated tags come back through the container.

Use this as a lookup table in framework code:

- "Is this element itself marked?" → `getDeclaredAnnotation`.
- "Does this class carry the marker, including from a superclass?" → `getAnnotation` (needs `@Inherited`).
- "Give me every occurrence of a repeatable annotation" → `getAnnotationsByType`.

None of these search interfaces, overridden methods or meta-annotations; that's always framework code (Q794).

Java

```
import java.lang.annotation.Annotation;                    // the harness also imports
java.text.*, which has its own Annotation
@Inherited @Retention(RetentionPolicy.RUNTIME) @Repeatable(Tags.class) @interface Tag { String
value(); }
@Inherited @Retention(RetentionPolicy.RUNTIME) @interface Tags { Tag[] value(); }
@Inherited @Retention(RetentionPolicy.RUNTIME) @interface Owner { String value(); }
@Tag("payments") @Tag("critical") @Owner("team-a") static class BaseService { }
@Tag("refunds") static class RefundService extends BaseService { }

static String show(Annotation[] as) {
    return Arrays.stream(as).map(a -> a instanceof Tag t ? "Tag(" + t.value() + ")" : a instanceof
```


© TechTribers. For personal use only. Do not redistribute.

```
Owner o ? "Owner(" + o.value() + ")" : a instanceof Tags ts ? "Tags[" + ts.value().length + "]" :
"?").sorted().toList().toString();
}

Class<?> c = RefundService.class;
System.out.println("getDeclaredAnnotations:            " + show(c.getDeclaredAnnotations()));
System.out.println("getAnnotations:                    " + show(c.getAnnotations()));
System.out.println("getDeclaredAnnotationsByType(Tag): " +
show(c.getDeclaredAnnotationsByType(Tag.class)));
System.out.println("getAnnotationsByType(Tag):         " + show(c.getAnnotationsByType(Tag.class)));
System.out.println("getAnnotationsByType on BaseService: " +
show(BaseService.class.getAnnotationsByType(Tag.class)));
```

▶ Output

```
getDeclaredAnnotations:            [Tag(refunds)]
getAnnotations:                    [Owner(team-a), Tag(refunds), Tags[2]]
getDeclaredAnnotationsByType(Tag): [Tag(refunds)]
getAnnotationsByType(Tag):         [Tag(refunds)]
getAnnotationsByType on BaseService: [Tag(critical), Tag(payments)]
```

Which method sees which annotation? (AnnotatedElement Javadoc terms)

	directly present	inside a container	inherited (@Inherited)
`getDeclaredAnnotation(s)`	✓	—	—
`getAnnotation(s)`	✓	—	✓
`getDeclaredAnnotationsByType`	✓	✓	—
`getAnnotationsByType`	✓	✓	✓ if none declared

None of them search interfaces, overridden methods or meta-annotations: that is framework code.

🎯 **Likely follow-up**

Are the returned arrays cached? The annotation **objects** are cached per element, but every call returns a **new array**. In hot paths, cache the result (or a derived boolean) per element.

© TechTribers. For personal use only. Do not redistribute.

Q789 GR-050 • Advanced

How are annotations represented at run time, and what are their `equals`, `hashCode` and `toString` rules?

⚡ Quick answer

At run time an annotation is an object implementing the annotation **interface**: the JDK creates it as a **dynamic proxy** (`$ProxyN` , defined in the annotation's own package when the annotation type isn't public, otherwise in a `jdk.proxyN` module; backed by `AnnotationInvocationHandler`) holding the element values parsed from the class file, created lazily and cached per element. `java.lang.annotation.Annotation` specifies the contract: two annotations are `equal` if they have the same `annotationType()` and equal element values (arrays compared by content), and `hashCode` is a fixed formula over element names and values. So you can **implement an annotation interface yourself** (handy in tests and for programmatic configuration), and if you follow the contract it equals the JDK's instance.

What the output tells you:

- The JDK's annotation object is a `Proxy` , but **not** a hidden class (we checked `isHidden()` ; it's `false`).
- `toString()` is source-like since Java 19 (`@Main.Range(min=0, max=120)` style); don't parse it, the format isn't specified.
- `getAnnotation` returns the cached instance for the same field object each time, so frameworks can compare with `==` within one `Field` , but use `equals` across elements.
- The JDK's `equals` only calls the other object's element methods and `annotationType()` , so a correct hand-written implementation compares equal both ways.

Real-world uses of hand-made instances: Hibernate Validator's `AnnotationDescriptor` , CDI's `AnnotationLiteral` (e.g. `new NamedLiteral("x")` to select beans programmatically), Guice's `Names.named("x")` , and tests that need to pass an annotation to code under test.

Java

```
import java.lang.annotation.Annotation;                                    // the harness also imports
java.text.*, which has its own Annotation
@Retention(RetentionPolicy.RUNTIME) @interface Range { int min() default 0; int max(); }
static class Form { @Range(max = 120) int age; }

record RangeLiteral(int min, int max) implements Range {                          // a hand-made
annotation instance
    public Class<? extends Annotation> annotationType() { return Range.class; }
    @Override public int hashCode() { return (127 * "min".hashCode() ^ Integer.hashCode(min)) + (127
* "max".hashCode() ^ Integer.hashCode(max)); }
    @Override public boolean equals(Object o) { return o instanceof Range r && r.min() == min &&
r.max() == max; }
}

Range fromClass = Form.class.getDeclaredField("age").getAnnotation(Range.class);
System.out.println("class: " + fromClass.getClass().getName().replaceAll("\\d+", "N") + ", proxy: "
+ Proxy.isProxyClass(fromClass.getClass()));
System.out.println("toString: " + fromClass);
System.out.println("cached: " + (fromClass ==
```


© TechTribers. For personal use only. Do not redistribute.

```
Form.class.getDeclaredField("age").getAnnotation(Range.class)));
Range mine = new RangeLiteral(0, 120);
System.out.println("JDK equals mine: " + fromClass.equals(mine) + ", mine equals JDK: " +
mine.equals(fromClass) + ", same hashCode: " + (fromClass.hashCode() == mine.hashCode()));
System.out.println("different value: " + fromClass.equals(new RangeLiteral(0, 99)));
```

▶ Output

```
class: $ProxyN, proxy: true
toString: @Main.Range(min=0, max=120)
cached: true
JDK equals mine: true, mine equals JDK: true, same hashCode: true
different value: false
```

⚠️ **Interview trap**

Implementing an annotation interface with a default `record` `hashCode` / `equals`. Records compare by components but compute `hashCode` differently from the `Annotation` formula, so a `HashSet` mixing JDK and hand-made instances misbehaves. Implement the documented formula, as above.

Q790 GR-051 • Advanced

What are type annotations, where do they attach (`@Nullable String[]` vs `String @Nullable []`), and what is JSpecify?

⚡ **Quick answer**

A `TYPE_USE` annotation annotates a **use of a type**, so its position matters: `@Nullable String[] a` means *an array of nullable strings*, `String @Nullable [] b` means *a nullable array of strings*, and `List<@Nullable String>` means *a list whose elements may be null*. Reflection exposes them through `AnnotatedType` (`getAnnotatedType()` , `AnnotatedArrayType` , `AnnotatedParameterizedType`). **JSpecify** (1.0, 2024) is the cross-vendor standard set of `TYPE_USE` nullness annotations (`@Nullable` , `@NonNull` , `@NullMarked` , `@NullUnmarked`), adopted by Spring Framework 7 and checked at build time by NullAway, the Checker Framework and Kotlin.

Two observations from the output: the annotation is attached exactly where it is written in the type, and `f.getAnnotations()` is **empty** for every field, because a pure `TYPE_USE` annotation is not a declaration annotation. Code that only checks `field.isAnnotationPresent(Nullable.class)` misses all of them.

Why the ecosystem moved to `TYPE_USE` nullness: older annotations (JSR 305's `javax.annotation.Nullable` , Spring's `org.springframework.lang.Nullable` , which Spring 7 deprecates) were declaration annotations, so they couldn't say anything about generic arguments or array components. Spring 7's migration notes call out exactly this change: `@Nullable Object[] array` must be rewritten as `Object @Nullable [] array` .

How JSpecify is used: annotate a package or module `@NullMarked` (everything is non-null by default there), mark exceptions with `@Nullable` , and run **NullAway** with Error Prone in the build so violations fail compilation. Kotlin reads JSpecify annotations and exposes the types as nullable or non-null.

© TechTribers. For personal use only. Do not redistribute.

Java

```java
@Target(ElementType.TYPE_USE) @Retention(RetentionPolicy.RUNTIME) @interface Nullable { }
static class Dto {
    @Nullable String[] a;                // elements may be null
    String @Nullable [] b;               // the array may be null
    List<@Nullable String> c;            // elements may be null
    @Nullable List<String> d;            // the list may be null
    Map<String, @Nullable Integer> e;
}

static String where(AnnotatedType t) {
    String self = t.isAnnotationPresent(Nullable.class) ? "@Nullable " : "";
    return switch (t) {
        case AnnotatedArrayType arr -> where(arr.getAnnotatedGenericComponentType()) + " " + self +
"[]";
        case AnnotatedParameterizedType p -> self + ((Class<?>) ((ParameterizedType)
p.getType()).getRawType()).getSimpleName()
                +
Arrays.stream(p.getAnnotatedActualTypeArguments()).map(Main::where).collect(Collectors.joining(", ",
"<", ">"));
        default -> self + ((Class<?>) t.getType()).getSimpleName();
    };
}

for (Field f : Dto.class.getDeclaredFields())
    if (!f.isSynthetic()) System.out.println(f.getName() + ": " + where(f.getAnnotatedType()) + "
| declaration annotations: " + f.getAnnotations().length);
```

▶ Output

```
a: @Nullable String []   | declaration annotations: 0
b: String @Nullable []   | declaration annotations: 0
c: List<@Nullable String>   | declaration annotations: 0
d: @Nullable List<String>   | declaration annotations: 0
e: Map<String, @Nullable Integer>   | declaration annotations: 0
```

TYPE_USE annotations attach to the type exactly where you write them

`@Nullable String[] a`	→ array of NULLABLE strings
`String @Nullable [] b`	→ the ARRAY may be null
`List<@Nullable String> c`	→ list ELEMENTS may be null
`@Nullable List<String> d`	→ the LIST may be null

JSpecify (@NullMarked, @Nullable) + NullAway checks this at build time. Spring 7: Object @Nullable [] not @Nullable Object[].

Pure TYPE_USE annotations are NOT declaration annotations: field.getAnnotations() is empty. Read getAnnotatedType().

© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
Writing `@Nullable String[] items` when you mean the array can be null. With declaration-style annotations it meant the field; with `TYPE_USE` annotations it means the **elements**. Spring 7 migrations hit this repeatedly.

Q791 GR-052 • Advanced

How do annotations on record components propagate to fields, accessors and constructor parameters?

⚡ **Quick answer**
An annotation written on a record component is copied by javac to **every member derived from the component whose kind the annotation's `@Target` allows**: the private field (`FIELD`), the accessor method (`METHOD`), the canonical-constructor parameter (`PARAMETER`), the record component itself (`RECORD_COMPONENT`), and type uses (`TYPE_USE`). An annotation targeting only `METHOD` therefore ends up only on the accessor. If you write an explicit accessor or canonical constructor, the copy doesn't happen for that member: annotate it yourself.

This is why Bean Validation and Jackson work on records with no extra effort: `@NotBlank` (targets include `FIELD` , `METHOD` , `PARAMETER` ...) lands on the field, the accessor and the constructor parameter, and Hibernate Validator finds it wherever it looks. Jackson reads `@JsonProperty` from the constructor parameter it uses for creation.

The second component shows the trap: `name()` is written by hand, so `@Anywhere` is **not** copied to it, while the field and the constructor parameter still get it. A framework reading annotations from accessors then sees nothing.

```java
import java.lang.annotation.Annotation;                                  // the harness also imports
java.text.*, which has its own Annotation
@Retention(RetentionPolicy.RUNTIME) @interface Anywhere { }                                  // no
@Target: all declarations
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.FIELD) @interface OnField { }
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.METHOD) @interface OnAccessor { }
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.PARAMETER) @interface OnParam { }
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.RECORD_COMPONENT) @interface OnComponent { }
record User(@Anywhere @OnField @OnAccessor @OnParam @OnComponent String email, @Anywhere String
name) {
    public String name() { return name.strip(); }                                        //
explicit accessor
}

static List<String> names(Annotation[] as) { return Arrays.stream(as).map(a ->
a.annotationType().getSimpleName()).sorted().toList(); }

for (String comp : List.of("email", "name")) {
    int idx = comp.equals("email") ? 0 : 1;
    System.out.println(comp + ":");
    System.out.println("  component:    " + names(User.class.getRecordComponents()
[idx].getAnnotations()));
```


© TechTribers. For personal use only. Do not redistribute.

```
    System.out.println("  field:       " +
names(User.class.getDeclaredField(comp).getAnnotations()));
    System.out.println("  accessor:    " + names(User.class.getMethod(comp).getAnnotations()));
    System.out.println("  ctor param:  " + names(User.class.getDeclaredConstructors()
[0].getParameters()[idx].getAnnotations()));
}
```

▶ Output

```
email:
  component:   [Anywhere, OnComponent]
  field:       [Anywhere, OnField]
  accessor:    [Anywhere, OnAccessor]
  ctor param:  [Anywhere, OnParam]
name:
  component:   [Anywhere]
  field:       [Anywhere]
  accessor:    []
  ctor param:  [Anywhere]
```

⚠️ **Interview trap**

Adding a custom annotation with `@Target(ElementType.RECORD_COMPONENT)` only, and expecting a field-based framework to see it. It exists only on the `RecordComponent` ; choose targets that match where the framework actually looks.

© TechTribers. For personal use only. Do not redistribute.

Section 06

Annotations II: Processing and Building Annotation-Driven Frameworks

Compile-time annotation processors that validate and generate code, what Lombok does differently and the JDK 23 change that broke builds, Spring-style meta-annotations, then four small frameworks built from scratch: validation, dependency injection, a test runner and a request router, plus the -parameters flag.

Questions 792–799 · 8 questions



Q792 GR-053

• Advanced

What is annotation processing (JSR 269), and how do you write a processor that validates code and generates a class?

⚡ Quick answer

An **annotation processor** is a plug-in for javac (`javax.annotation.processing.Processor` , usually extending `AbstractProcessor`) that runs **during compilation** in **rounds**. Each round it receives the elements annotated with its supported annotations, can inspect them through the `javax.lang.model` API (types, modifiers, annotation values, but not method bodies), report errors and warnings at precise source positions through the `Messager` , and **generate new source or resource files** through the `Filer` . Generated sources are compiled in the next round. MapStruct, Dagger, Micronaut, Immutables, AutoValue, JPA metamodel and Spring's `spring-context-indexer` /configuration-metadata processors all work this way.

Reading the output:

- Compile 1 fails with an **error on line 5**, exactly where the non-public method is. That's the main selling point of processors over run-time reflection: misuse of your annotation becomes a compile error with a source position, not a startup crash.
- Compile 2 shows the **rounds**: round 1 sees `@Command` and generates `CommandRegistry.java` ; round 2 compiles the generated file (no new `@Command` elements, so the annotation set is empty); the final round has `over=true` . Generated code can itself contain annotations, which would trigger further rounds.
- The generated class is ordinary bytecode: no reflection at run time, fully visible to IDEs and to GraalVM native images.

Processor rules that matter in practice: processors must be **deterministic** and must not depend on round order beyond the API contract; they can only **create new files** (next question); they see signatures and annotations, not statement bodies; and `return true` from `process` claims the annotations so later processors don't see them, which is rarely what you want in shared builds.

© TechTribers. For personal use only. Do not redistribute.

Java

```
import javax.annotation.processing.*;
import javax.lang.model.SourceVersion;
import javax.lang.model.element.*;
import javax.lang.model.element.Modifier;                    // beats java.lang.reflect.Modifier from
the harness imports
import javax.tools.*;
static class CommandProcessor extends AbstractProcessor {
    private final List<String> names = new ArrayList<>();
    private int round;
    @Override public Set<String> getSupportedAnnotationTypes() { return Set.of("Command"); }
    @Override public SourceVersion getSupportedSourceVersion() { return
SourceVersion.latestSupported(); }
    @Override public boolean process(Set<? extends TypeElement> annotations, RoundEnvironment env) {
        Messager log = processingEnv.getMessager();
        log.printMessage(Diagnostic.Kind.NOTE, "round " + (++round) + " annotations=" +
annotations.stream().map(a -> a.getSimpleName().toString()).toList() + " over=" +
env.processingOver());
        for (TypeElement ann : annotations)
            for (Element e : env.getElementsAnnotatedWith(ann)) {
                if (!e.getModifiers().contains(Modifier.PUBLIC)) {
log.printMessage(Diagnostic.Kind.ERROR, "@Command method must be public", e); continue; }
                e.getAnnotationMirrors().forEach(m -> m.getElementValues().values().forEach(v ->
names.add(v.getValue().toString())));
            }
        if (!annotations.isEmpty() && !names.isEmpty()) {
            try (Writer w =
processingEnv.getFiler().createSourceFile("CommandRegistry").openWriter()) {
                w.write("public final class CommandRegistry { public static final
java.util.List<String> NAMES = java.util.List.of("
                        + names.stream().map(n -> "\"" + n + "\"").collect(Collectors.joining(", "))
+ "); }");
            } catch (IOException ex) { log.printMessage(Diagnostic.Kind.ERROR, ex.toString()); }
        }
        return true;                                                    // claim @Command: no
other processor gets it
    }
}

static boolean compile(Path dir, String jobsSource) throws IOException {
    Files.writeString(dir.resolve("Jobs.java"), jobsSource);
    JavaCompiler jc = ToolProvider.getSystemJavaCompiler();
    DiagnosticCollector<JavaFileObject> dc = new DiagnosticCollector<>();
    StandardJavaFileManager fm = jc.getStandardFileManager(dc, null, null);
    JavaCompiler.CompilationTask task = jc.getTask(null, fm, dc, List.of("-d", dir.toString(), "-s",
dir.toString()), null, fm.getJavaFileObjects(dir.resolve("Jobs.java")));
    task.setProcessors(List.of(new CommandProcessor()));
    boolean ok = task.call();
    dc.getDiagnostics().forEach(d -> System.out.println("  " + d.getKind() + (d.getLineNumber() > 0
? " line " + d.getLineNumber() : "") + ": " + d.getMessage(Locale.ENGLISH)));
    return ok;
}

String jobs = """
        import java.lang.annotation.*;
        @Retention(RetentionPolicy.SOURCE) @Target(ElementType.METHOD) @interface Command { String
```

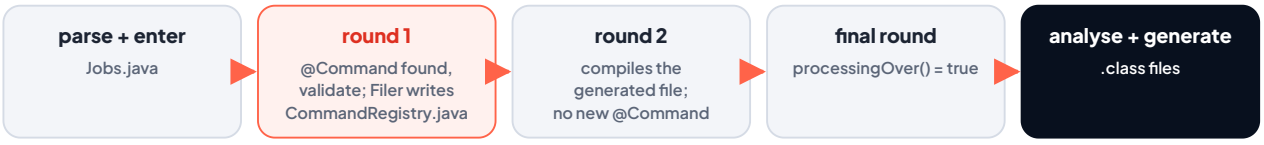

© TechTribers. For personal use only. Do not redistribute.

```
value(); }
        public class Jobs {
            @Command("backup") public void backup() { }
            @Command("purge") void purge() { }
        }
        """;
Path dir = Files.createTempDirectory("apt");
System.out.println("compile 1 ok=" + compile(dir, jobs));
System.out.println("compile 2 ok=" + compile(dir, jobs.replace("@Command(\"purge\") void",
"@Command(\"purge\") public void")));
try (URLClassLoader cl = new URLClassLoader(new URL[]{dir.toUri().toURL()})) {
    System.out.println("generated CommandRegistry.NAMES = " +
cl.loadClass("CommandRegistry").getField("NAMES").get(null));
}
```

▶ Output

```
  NOTE: round 1 annotations=[Command] over=false
  ERROR line 5: @Command method must be public
  NOTE: round 2 annotations=[] over=true
compile 1 ok=false
  NOTE: round 1 annotations=[Command] over=false
  NOTE: round 2 annotations=[] over=false
  NOTE: round 3 annotations=[] over=true
compile 2 ok=true
generated CommandRegistry.NAMES = [backup, purge]
```

Annotation processing rounds inside javac



Messager.printMessage(ERROR, msg, element) → a compile error at the exact source line (measured: line 5).
Processors may only CREATE files. Modifying an existing class is impossible via JSR 269 (Lombok hacks javac's AST).
JDK 23+: processors on the CLASS PATH no longer run implicitly — use a processor path or -proc:full.

💡 **Pro tip**

Register a processor for real builds in `META-INF/services/javax.annotation.processing.Processor` (or with `provides` in `module-info`), and put it on the **processor path** (Maven `annotationProcessorPaths`, Gradle `annotationProcessor` configuration), not on the compile class path. The next question explains why that matters since JDK 23.

© TechTribers. For personal use only. Do not redistribute.

Q793 GR-054 • Advanced

How does Lombok differ from a standard annotation processor, and what changed for processors in JDK 23?

⚡ Quick answer

The JSR 269 API can **only create new files**; it gives no way to modify the class being compiled. Lombok's `@Getter`, `@Builder` and `@Slf4j` add members to *your* class, which is impossible through the standard API, so Lombok hooks into **javac's internal AST** (`com.sun.tools.javac.tree`) and rewrites the tree in place. That's why Lombok needs updates for each new JDK and why it relies on internal-API access. Separately, **JDK 23** stopped running processors found on the **class path** implicitly: builds that relied on Lombok or MapStruct being picked up from the compile class path silently stopped generating code until `-proc:full` or an explicit processor path was configured.

The processor can **see** the class and its private field, but `Filer.createSourceFile("Person")` is rejected because `Person` already exists. There is no supported "modify" operation, by design: the JSR 269 authors wanted processors to be side-effect-free code generators so builds stay predictable.

JDK 23 change, measured here: we put a trivial processor in a JAR-style directory with a `META-INF/services` entry on the **class path** and compiled through `javax.tools`. On JDK 21 it ran, with a note warning that "a future release of javac may disable annotation processing unless at least one processor is specified by name (`-processor`), or a search path is specified (`--processor-path`, `--processor-module-path`), or annotation processing is enabled explicitly (`-proc:only`, `-proc:full`)". On **JDK 25 and 26 it did not run at all**, with no message. Fix for Maven: list processors under `maven-compiler-plugin` → `annotationProcessorPaths` (Spring Boot's parent POM and Lombok's docs now show this), or add `-proc:full`. Gradle has used a separate `annotationProcessor` configuration for years, so most Gradle builds were unaffected.

Alternatives to Lombok's AST mutation: **records** for data carriers, IDE generation, **Immutables/AutoValue/RecordBuilder** (standard processors that generate a *new* class such as `ImmutablePerson` or `PersonBuilder`), and Kotlin data classes.

Java

```
import javax.annotation.processing.*;
import javax.lang.model.SourceVersion;
import javax.lang.model.element.*;
import javax.lang.model.element.Modifier;                    // beats java.lang.reflect.Modifier from
the harness imports
import javax.tools.*;
static class RecreateProcessor extends AbstractProcessor {
    @Override public Set<String> getSupportedAnnotationTypes() { return Set.of("*"); }
    @Override public SourceVersion getSupportedSourceVersion() { return
SourceVersion.latestSupported(); }
    @Override public boolean process(Set<? extends TypeElement> annotations, RoundEnvironment env) {
        for (Element e : env.getRootElements()) {
            System.out.println("  sees " + e.getKind() + " " + e + " members: " +
e.getEnclosedElements().stream().map(m -> m.getKind() + " " + m.getSimpleName()).toList());
            try { processingEnv.getFiler().createSourceFile(e.toString()); }                    // try
to "add a getter" by rewriting the class
            catch (IOException ex) { System.out.println("  Filer refuses: " + ex.getMessage()); }
        }
```


© TechTribers. For personal use only. Do not redistribute.

```
        return false;
    }
}

Path dir = Files.createTempDirectory("lombok-like");
Files.writeString(dir.resolve("Person.java"), "public class Person { private String name; }");
JavaCompiler jc = ToolProvider.getSystemJavaCompiler();
StandardJavaFileManager fm = jc.getStandardFileManager(null, null, null);
JavaCompiler.CompilationTask task = jc.getTask(null, fm, null, List.of("-d", dir.toString()), null,
fm.getJavaFileObjects(dir.resolve("Person.java")));
task.setProcessors(List.of(new RecreateProcessor()));
System.out.println("compiled: " + task.call());
```

▶ Output

```
  sees CLASS Person members: [CONSTRUCTOR <init>, FIELD name]
  Filer refuses: Attempt to recreate a file for type Person
compiled: true
```

⚠️ **Interview trap**

"Lombok generates source code." It doesn't: nothing is written to disk; it changes javac's in-memory tree, so the generated getters exist only in the bytecode. That's why debuggers and some tools need `delombok` to see them.

Q794 GR-055 • Advanced

How do composed annotations and meta-annotations work (Spring style), and how do you search for them correctly?

⚡ **Quick answer**

A **meta-annotation** is an annotation placed on another annotation type; a **composed annotation** bundles several (`@RestController` = `@Controller` + `@ResponseBody`, `@SpringBootApplication` = `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`). The JVM gives meta-annotations **no meaning**: `PlaceOrder.class.getAnnotation(Component.class)` returns `null` even if `@Component` sits two levels up. Frameworks walk `annotationType().getAnnotations()` recursively, and they must guard against **cycles**: `@Documented`, `@Retention` and `@Target` are annotated with themselves.

The naive search shows why cycle protection is mandatory: looking for any annotation that isn't there walks `@Documented → @Documented → ...` forever.

What Spring adds on top of this search (in `MergedAnnotations`, the engine behind `AnnotatedElementUtils` and `@AliasFor`):

- **Attribute overrides**: `@AliasFor(annotation = Component.class, attribute = "value")` lets `@UseCase("checkout")` set the bean name of the meta-annotated `@Component`.
- **Merged views**: `findMergedAnnotation(PlaceOrder.class, Component.class)` returns a synthesized `@Component` whose values reflect the overrides.
- **Search strategies**: direct, inherited, or the whole type hierarchy including interfaces.
- **Caching**, because component scanning evaluates this for every class on the class path.

© TechTribers. For personal use only. Do not redistribute.

Design advice: compose annotations to give your domain vocabulary (`@UseCase`, `@ApiController`, `@IntegrationTest` combining `@SpringBootTest`, `@Testcontainers`, `@ActiveProfiles`), but keep the meta-chain shallow; three or more levels make it hard to tell what a class really is.

Java

```
import java.lang.annotation.Annotation;                    // the harness also imports
java.text.*
@Retention(RetentionPolicy.RUNTIME) @Target({ElementType.TYPE, ElementType.ANNOTATION_TYPE})
@Documented @interface Component { }
@Retention(RetentionPolicy.RUNTIME) @Target({ElementType.TYPE, ElementType.ANNOTATION_TYPE})
@Documented @Component @interface Service { }
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.TYPE) @Documented @Service @interface
UseCase { String value(); }
@UseCase("checkout") static class PlaceOrder { }

static Optional<List<String>> findPath(AnnotatedElement e, Class<? extends Annotation> wanted,
Set<Class<?>> visited) {
    for (Annotation a : e.getAnnotations()) {
        Class<? extends Annotation> t = a.annotationType();
        if (t == wanted) return Optional.of(new ArrayList<>(List.of(t.getSimpleName())));
        if (t.getPackageName().equals("java.lang.annotation") || !visited.add(t)) continue;      //
skip JDK meta-annotations and cycles
        Optional<List<String>> p = findPath(t, wanted, visited);
        if (p.isPresent()) { p.get().add(0, t.getSimpleName()); return p; }
    }
    return Optional.empty();
}

static boolean naive(AnnotatedElement e, Class<? extends Annotation> wanted) {
    for (Annotation a : e.getAnnotations()) if (a.annotationType() == wanted ||
naive(a.annotationType(), wanted)) return true;
    return false;
}

System.out.println("getAnnotation(Component): " + PlaceOrder.class.getAnnotation(Component.class));
System.out.println("meta path: " + findPath(PlaceOrder.class, Component.class, new HashSet<>
()).map(p -> String.join(" -> ", p)).orElse("not found"));
System.out.println("Documented is meta-annotated with itself: " +
Documented.class.isAnnotationPresent(Documented.class));
try { naive(PlaceOrder.class, Inherited.class); } catch (StackOverflowError e) {
System.out.println("naive recursive search: StackOverflowError"); }
```

▶ Output

```
getAnnotation(Component): null
meta path: UseCase -> Service -> Component
Documented is meta-annotated with itself: true
naive recursive search: StackOverflowError
```


© TechTribers. For personal use only. Do not redistribute.

Composed annotations: the JVM ignores meta-annotations, frameworks walk them



Without the visited set the search loops on @Documented / @Retention / @Target (measured: StackOverflowError).
Spring adds @AliasFor attribute overrides and merged, synthesized views (MergedAnnotations).

🎯 **Likely follow-up**

Does `@Inherited` on `@Service` make `@Component` inherited? No. `@Inherited` affects only class-to-subclass inheritance of that one annotation type, never meta-annotation resolution.

✅ **Q795** GR-056 • Advanced

Coding: build a mini Bean Validation framework with runtime annotations.

⚡ **Quick answer**

Declare `RUNTIME`, `FIELD`-targeted constraint annotations (`@NotBlank`, `@Range`, `@Pattern`, `@Valid` for nesting). Keep a **registry** mapping each constraint type to a checker function, so new constraints don't change the engine. The validator walks the object's fields (including superclasses, cached per class), runs every registered constraint found on each field, recurses into `@Valid` fields with a property path, and returns a list of violations instead of throwing on the first one.

This is the architecture of Jakarta Bean Validation (Hibernate Validator) in miniature: constraint annotations, a `ConstraintValidator` per constraint (our `CHECKS` map), property paths (`address.pin`), violation lists, and `@Valid` cascading.

What the real thing adds and an interviewer may ask about:

- **Constraint composition** via meta-annotations (`@Email` built from `@Pattern`), and the `@Constraint(validatedBy = ...)` link from annotation to validator.
- **Groups** (validate only `OnCreate` constraints) and **group sequences**.
- **Container element constraints** with `TYPE_USE`: `List<@Email String> emails`.
- **Method validation** (parameters and return values), which Spring applies through a proxy with `@Validated`, so the self-invocation trap from Q774 applies.
- **Message interpolation** (`{min}` placeholders, `ValidationMessages.properties`, locale).
- **Caching** of the per-class metadata, which matters a lot under load.

Java

```
import java.lang.annotation.Annotation;
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.FIELD) @interface NotBlank { String
```


© TechTribers. For personal use only. Do not redistribute.

```
message() default "must not be blank"; }
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.FIELD) @interface Range { long min() default
Long.MIN_VALUE; long max() default Long.MAX_VALUE; }
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.FIELD) @interface Pattern { String regexp();
String message() default "has invalid format"; }
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.FIELD) @interface Valid { }

static final Map<Class<? extends Annotation>, BiFunction<Annotation, Object, String>> CHECKS =
Map.of(
    NotBlank.class, (a, v) -> v instanceof String s && !s.isBlank() ? null : ((NotBlank)
a).message(),
    Range.class, (a, v) -> { Range r = (Range) a; long n = ((Number) v).longValue(); return n >=
r.min() && n <= r.max() ? null : "must be between " + r.min() + " and " + r.max(); },
    Pattern.class, (a, v) -> v == null || ((String) v).matches(((Pattern) a).regexp()) ? null :
((Pattern) a).message());

static List<String> validate(Object bean) throws IllegalAccessException { List<String> out = new
ArrayList<>(); validate("", bean, out); return out; }

static void validate(String path, Object bean, List<String> out) throws IllegalAccessException {
    for (Class<?> c = bean.getClass(); c != Object.class; c = c.getSuperclass())
        for (Field f : c.getDeclaredFields()) {
            if (Modifier.isStatic(f.getModifiers()) || !f.trySetAccessible()) continue;
            Object value = f.get(bean);
            for (Annotation a : f.getAnnotations()) {
                BiFunction<Annotation, Object, String> check = CHECKS.get(a.annotationType());
                String error = check == null ? null : check.apply(a, value);
                if (error != null) out.add(path + f.getName() + ": " + error + " (was " + value +
")");
            }
            if (f.isAnnotationPresent(Valid.class) && value != null) validate(path + f.getName() +
".", value, out);
        }
}

static class Address { @NotBlank String city; @Pattern(regexp = "\\d{6}", message = "must be a 6-
digit PIN") String pin;
    Address(String c, String p) { city = c; pin = p; } }
static class SignupForm { @NotBlank String name; @Range(min = 18, max = 120) int age; @Valid Address
address;
    SignupForm(String n, int a, Address ad) { name = n; age = a; address = ad; } }

System.out.println(validate(new SignupForm("Asha", 31, new Address("Pune", "411057"))));
validate(new SignupForm(" ", 16, new Address("", "4110"))).forEach(System.out::println);
```

▶ Output

```
[]
name: must not be blank (was  )
age: must be between 18 and 120 (was 16)
address.city: must not be blank (was )
address.pin: must be a 6-digit PIN (was 4110)
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> `null` handling convention from the spec: every constraint except `@NotNull` / `@NotBlank` / `@NotEmpty` treats `null` as valid, so constraints compose (`@NotNull @Email`). Our `@Pattern` follows that rule.

Q796 GR-057 • Advanced

Coding: build a mini dependency injection container with `@Inject` and `@Singleton` .

> ⚡ **Quick answer**
> Bind interfaces to implementations, then resolve a type by choosing its constructor (the one marked `@Inject` , or the only one), **recursively resolving each parameter type**, and instantiating it reflectively. Cache instances of `@Singleton` classes; create a new instance otherwise. Track the types currently being resolved on a stack so a **circular dependency** fails fast with the full cycle path instead of a `StackOverflowError` .

About 40 lines give you the core of Spring's `DefaultListableBeanFactory` , Guice's injector or Jakarta CDI: bindings, constructor selection, recursive resolution, scopes and cycle detection. Constructor injection makes the cycle **impossible to satisfy** (A needs a finished B, B needs a finished A), which is the same reason Spring Boot 2.6+ fails at startup on circular references by default.

What production containers add:

- **Qualifiers** (`@Named` , custom qualifier annotations) when several implementations of one interface exist, and `List<Plugin>` injection of all of them.
- **Providers / lazy injection** (`Provider<T>` , `ObjectProvider<T>`) which is also the standard way to break legitimate cycles.
- **Thread safety** for singleton creation (our container is single-threaded).
- **Lifecycle** callbacks (`@PostConstruct` , `@PreDestroy`) and proxies for scopes such as request or session.
- **Build-time wiring**: Dagger and Micronaut generate this resolution code with annotation processors, removing reflection from startup; Spring AOT does the same for native images.

```java
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.CONSTRUCTOR) @interface Inject { }
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.TYPE) @interface Singleton { }

static final class Container {
    private final Map<Class<?>, Class<?>> bindings = new HashMap<>();
    private final Map<Class<?>, Object> singletons = new HashMap<>();
    private final Deque<Class<?>> resolving = new ArrayDeque<>();
    <T> Container bind(Class<T> api, Class<? extends T> impl) { bindings.put(api, impl); return
this; }
    <T> T get(Class<T> type) {
        Class<?> impl = bindings.getOrDefault(type, type);
        if (singletons.containsKey(impl)) return type.cast(singletons.get(impl));
        if (impl.isInterface()) throw new IllegalStateException("no binding for " +
```


© TechTribers. For personal use only. Do not redistribute.

```
type.getSimpleName());
        if (resolving.contains(impl)) throw new IllegalStateException("cycle: " + cyclePath(impl));
        resolving.push(impl);
        try {
            Constructor<?> ctor = Arrays.stream(impl.getDeclaredConstructors()).filter(c ->
c.isAnnotationPresent(Inject.class)).findFirst()
                    .orElseGet(() -> impl.getDeclaredConstructors()[0]);
            Object[] args = Arrays.stream(ctor.getParameterTypes()).map(this::get).toArray();
            ctor.setAccessible(true);
            Object instance = ctor.newInstance(args);
            if (impl.isAnnotationPresent(Singleton.class)) singletons.put(impl, instance);
            return type.cast(instance);
        } catch (ReflectiveOperationException e) { throw new IllegalStateException("cannot create "
+ impl.getSimpleName(), e); }
        finally { resolving.pop(); }
    }
    private String cyclePath(Class<?> repeat) {
        List<String> path = new ArrayList<>(); for (Class<?> c : resolving) path.add(0,
c.getSimpleName());
        path.add(repeat.getSimpleName()); return String.join(" -> ",
path.subList(path.indexOf(repeat.getSimpleName()), path.size()));
    }
}
interface Clock { String now(); }
@Singleton static class FixedClock implements Clock { public String now() { return "2026-10-
15T10:00"; } }
interface OrderRepo { String save(String id); }
@Singleton static class InMemoryOrderRepo implements OrderRepo { private final Clock clock; @Inject
InMemoryOrderRepo(Clock c) { clock = c; } public String save(String id) { return id + " saved at " +
clock.now(); } }
static class OrderService { final OrderRepo repo; final Clock clock; @Inject OrderService(OrderRepo
r, Clock c) { repo = r; clock = c; } }
static class A { A(B b) { } }
static class B { B(A a) { } }

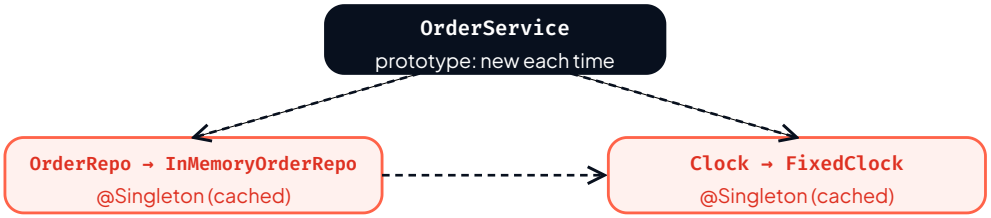
Container di = new Container().bind(Clock.class, FixedClock.class).bind(OrderRepo.class,
InMemoryOrderRepo.class);
OrderService s1 = di.get(OrderService.class), s2 = di.get(OrderService.class);
System.out.println(s1.repo.save("A-17"));
System.out.println("service is prototype: " + (s1 != s2) + ", repo is singleton: " + (s1.repo ==
s2.repo) + ", clock shared: " + (s1.clock == ((InMemoryOrderRepo) s1.repo).clock));
try { di.get(A.class); } catch (IllegalStateException e) { System.out.println(e.getMessage()); }
try { new Container().get(OrderService.class); } catch (IllegalStateException e) {
System.out.println(e.getMessage()); }
```

▶ Output

```
A-17 saved at 2026-10-15T10:00
service is prototype: true, repo is singleton: true, clock shared: true
cycle: A -> B -> A
no binding for OrderRepo
```


© TechTribers. For personal use only. Do not redistribute.

container.get(OrderService.class) — recursive constructor resolution



`resolving stack: [OrderService, InMemoryOrderRepo, FixedClock] → a repeat means a cycle`

Measured: A(B) + B(A) fails fast with "cycle: A -> B -> A" instead of a StackOverflowError.

⚠️ **Interview trap**

Field injection (`@Autowired private Repo repo;`) hides dependencies, makes `final` impossible (and is exactly the reflective final-field mutation JEP 500 warns about when combined with `final`), and lets cycles "work" with half-initialised objects. Constructor injection, as in this container, is the recommended default.

🧪 **Q797** GR-058 • Advanced

Coding: build a mini test runner like JUnit with `@Test` , `@BeforeEach` and expected exceptions.

⚡ **Quick answer**

Find public methods annotated `@Test` in the test class, sorted for a deterministic order. For each one, create a **new instance** (test isolation), run every `@BeforeEach` method, invoke the test, and unwrap `InvocationTargetException` to get the real failure. A test passes if it returns normally, or if it throws the `expected` exception type; assertion errors and other exceptions are failures; `@Disabled` tests are skipped. Report a summary.

Key decisions that JUnit 5 also makes:

- **New instance per test** (JUnit's default `PER_METHOD` lifecycle), so `eachTestGetsFreshState` can't leak state into other tests. `@TestInstance(PER_CLASS)` opts out.
- **Unwrap** `InvocationTargetException` before judging the result; otherwise every failing test reports the same useless wrapper.
- **Ordering**: JUnit 5's default method order is deterministic but intentionally not obvious (based on hash codes), to discourage order-dependent tests; `@TestMethodOrder` makes it explicit. We sort by name.
- **Expected exceptions**: JUnit 4's `@Test(expected=...)` was replaced in JUnit 5 by `assertThrows` , because an annotation can't tell *which line* threw: a test passes even if the exception came from the setup code. Our runner has the same weakness, which is a good thing to point out in an interview.

The `None` sentinel class nested in the annotation is the same trick JUnit 4 used (`Test.None`), because an annotation element can't default to `null` .

© TechTribers. For personal use only. Do not redistribute.

Java

```
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.METHOD) @interface Test { Class<? extends
Throwable> expected() default None.class; final class None extends Throwable { } }
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.METHOD) @interface BeforeEach { }
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.METHOD) @interface Disabled { String
value(); }

static void run(Class<?> testClass) throws ReflectiveOperationException {
    List<Method> tests = Arrays.stream(testClass.getMethods()).filter(m ->
m.isAnnotationPresent(Test.class)).sorted(Comparator.comparing(Method::getName)).toList();
    List<Method> befores = Arrays.stream(testClass.getMethods()).filter(m ->
m.isAnnotationPresent(BeforeEach.class)).toList();
    int passed = 0, failed = 0, skipped = 0;
    for (Method t : tests) {
        if (t.isAnnotationPresent(Disabled.class)) { skipped++; System.out.println("  SKIP " +
t.getName() + " (" + t.getAnnotation(Disabled.class).value() + ")"); continue; }
        Class<? extends Throwable> expected = t.getAnnotation(Test.class).expected();
        Object instance = testClass.getDeclaredConstructor().newInstance();                // fresh
instance per test
        Throwable failure = null;
        try {
            for (Method b : befores) b.invoke(instance);
            t.invoke(instance);
            if (expected != Test.None.class) failure = new AssertionError("expected " +
expected.getSimpleName());
        } catch (InvocationTargetException e) {
            Throwable real = e.getCause();
            if (!expected.isInstance(real)) failure = real;
        }
        if (failure == null) { passed++; System.out.println("  PASS " + t.getName()); }
        else { failed++; System.out.println("  FAIL " + t.getName() + ": " + failure); }
    }
    System.out.printf("%d tests: %d passed, %d failed, %d skipped%n", tests.size(), passed, failed,
skipped);
}

public static class CartTest {
    List<Integer> cart;
    @BeforeEach public void setUp() { cart = new ArrayList<>(List.of(100, 250)); }
    @Test public void totalIsSum() { check(cart.stream().mapToInt(i -> i).sum() == 350, "total"); }
    @Test public void eachTestGetsFreshState() { cart.add(1); check(cart.size() == 3, "size"); }
    @Test(expected = IndexOutOfBoundsException.class) public void outOfRange() { cart.get(5); }
    @Test(expected = IllegalStateException.class) public void missingException() { }
    @Test public void wrongDiscount() { check(cart.get(0) * 90 / 100 == 80, "10% of 100 should be
80"); }
    @Disabled("flaky on CI") @Test public void slowIntegration() { }
    static void check(boolean ok, String what) { if (!ok) throw new AssertionError(what); }
}

run(CartTest.class);
```

▶ Output

```
  PASS eachTestGetsFreshState
  FAIL missingException: java.lang.AssertionError: expected IllegalStateException
  PASS outOfRange
```


© TechTribers. For personal use only. Do not redistribute.

```
  SKIP slowIntegration (flaky on CI)
  PASS totalIsSum
  FAIL wrongDiscount: java.lang.AssertionError: 10% of 100 should be 80
6 tests: 3 passed, 2 failed, 1 skipped
```

🎯 **Likely follow-up**

How does JUnit 5 discover tests in a whole project? The JUnit Platform `Launcher` scans class-path roots for class files, loads candidates and filters them with engines (Jupiter, Vintage, Cucumber...), each of which applies its own annotation rules. Build tools and IDEs only talk to the Launcher API.

Q798 GR-059 • Advanced

Coding: build an annotation-driven request router with parameter annotations (like `@GetMapping` and `@PathVariable`).

⚡ **Quick answer**

At startup, scan controller methods for a route annotation (`@Get("/orders/{id}")`), compile each path template into a regex with **named groups**, and record which method parameters are bound from the path (`@PathVar`) or the query string (`@Query`) using **parameter annotations** (`Method.getParameterAnnotations()` or `Parameter.getAnnotation`). At request time, match the path, convert each string to the parameter's type through a converter table, and invoke the method. Unknown paths give a 404, failed conversions a 400.

Why the parameter annotations carry the names (`@PathVar("id")`) instead of reading the Java parameter name: without the `-parameters` compiler flag, parameter names are **not** in the class file and reflection reports `arg0` , `arg1` (next question). Spring MVC accepts `@PathVariable long id` without a name only because Spring Boot builds with `-parameters` .

What Spring MVC's `RequestMappingHandlerMapping` adds: HTTP-method and content negotiation conditions, specificity ordering when several patterns match (`/orders/new` before `/orders/{id}`), a `ConversionService` with many converters, argument resolvers for bodies, headers and principals, and validation of the bound arguments.

Performance note: all reflection (finding methods, reading annotations, compiling patterns) happens **once at registration**; each request only does a regex match and a `Method.invoke` . Real frameworks go further and pre-compute argument resolvers per parameter.

Java

```java
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.METHOD) @interface Get { String value(); }
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.PARAMETER) @interface PathVar { String
value(); }
@Retention(RetentionPolicy.RUNTIME) @Target(ElementType.PARAMETER) @interface Query { String
value(); String defaultValue() default ""; }

record Route(java.util.regex.Pattern pattern, Method method, Object controller) { }
static final Map<Class<?>, Function<String, Object>> CONVERTERS = Map.of(String.class, s -> s,
long.class, Long::parseLong, int.class, Integer::parseInt, boolean.class, Boolean::parseBoolean);
```


© TechTribers. For personal use only. Do not redistribute.

```
static final List<Route> ROUTES = new ArrayList<>();

static void register(Object controller) {
    for (Method m : controller.getClass().getMethods()) {
        Get get = m.getAnnotation(Get.class);
        if (get != null) ROUTES.add(new
Route(java.util.regex.Pattern.compile(get.value().replaceAll("\\{(\\w+)}", "(?<$1>[^/]+)")), m,
controller));
    }
}

static String handle(String url) {
    String path = url.contains("?") ? url.substring(0, url.indexOf('?')) : url;
    Map<String, String> query = new HashMap<>();
    if (url.contains("?")) for (String kv : url.substring(url.indexOf('?') + 1).split("&"))
query.put(kv.split("=")[0], kv.split("=", 2)[1]);
    for (Route r : ROUTES) {
        Matcher mt = r.pattern().matcher(path);
        if (!mt.matches()) continue;
        Parameter[] ps = r.method().getParameters(); Object[] args = new Object[ps.length];
        try {
            for (int i = 0; i < ps.length; i++) {
                PathVar pv = ps[i].getAnnotation(PathVar.class); Query q =
ps[i].getAnnotation(Query.class);
                String raw = pv != null ? mt.group(pv.value()) : query.getOrDefault(q.value(),
q.defaultValue());
                args[i] = CONVERTERS.get(ps[i].getType()).apply(raw);
            }
            return "200 " + r.method().invoke(r.controller(), args);
        } catch (NumberFormatException e) { return "400 bad parameter: " + e.getMessage(); }
        catch (ReflectiveOperationException e) { return "500 " + e.getCause(); }
    }
    return "404 no route for " + path;
}

public static class OrderController {
    @Get("/orders/{id}") public String order(@PathVar("id") long id, @Query(value = "expand",
defaultValue = "false") boolean expand) { return "order " + id + (expand ? " with items" : ""); }
    @Get("/customers/{cid}/orders/{oid}") public String nested(@PathVar("cid") String cid,
@PathVar("oid") int oid) { return "order " + oid + " of " + cid; }
}

register(new OrderController());
for (String url : List.of("/orders/42", "/orders/42?expand=true", "/customers/asha/orders/7",
"/orders/abc", "/invoices/1"))
    System.out.println(url + " -> " + handle(url));
```

▶ Output

```
/orders/42 -> 200 order 42
/orders/42?expand=true -> 200 order 42 with items
/customers/asha/orders/7 -> 200 order 7 of asha
/orders/abc -> 400 bad parameter: For input string: "abc"
/invoices/1 -> 404 no route for /invoices/1
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
Iterating routes in registration order and returning the first match. `getMethods()` order is unspecified, so `/orders/{id}` may shadow `/orders/new` depending on the JVM build. Sort routes by specificity (literal segments before variables), as Spring's `PathPattern` comparator does.

🔤 **Q799** `GR-060` • Intermediate

Why does `Parameter.getName()` return `arg0`, and what does the `-parameters` compiler flag change?

⚡ **Quick answer**
Method parameter names are **not stored** in the class file by default (only in the optional debug `LocalVariableTable` of method bodies, which interfaces and abstract methods don't have). Reflection then synthesizes `arg0`, `arg1`, and `Parameter.isNamePresent()` returns `false`. Compiling with `-parameters` writes a `MethodParameters` attribute, and reflection returns the real names. Spring 6.1 removed its fallback that read debug info (`LocalVariableTableParameterNameDiscoverer`), so `@PathVariable long id` without an explicit name now requires `-parameters`, which Spring Boot's build plugins enable for you. **Records** always keep component names.

Three compilations of the same source, loaded in isolated class loaders:

- **No flags** and `-g` (full debug info) both give `arg0, arg1...` through reflection. Debug info does contain the names of the concrete method's locals, but `java.lang.reflect.Parameter` never reads it; only libraries that parse bytecode (Spring before 6.1, Paranamer) could, and nothing can for interface methods, which have no body.
- `-parameters` gives real names for methods and interface methods alike.
- The **record's canonical constructor** has real names in all three cases: javac always emits a `MethodParameters` attribute for it (mandated parameters), which is one reason records work so smoothly with Jackson and Spring binding.

Who needs names: Spring MVC/WebFlux argument binding, Spring Data `@Query` named parameters, Jackson's `ParameterNamesModule` for constructor creators, MapStruct, and Micronaut. Maven: `<maven.compiler.parameters>true</maven.compiler.parameters>` (set by `spring-boot-starter-parent`); Gradle: `options.compilerArgs.add("-parameters")` (applied by the Spring Boot Gradle plugin).

Java

```
static Path compileTo(String src, String... extraOpts) throws IOException {
    Path dir = Files.createTempDirectory("params");
    Files.writeString(dir.resolve("PriceApi.java"), src);
    List<String> args = new ArrayList<>(List.of(extraOpts)); args.addAll(List.of("-d",
dir.toString(), dir.resolve("PriceApi.java").toString()));
    javax.tools.ToolProvider.getSystemJavaCompiler().run(null, null, null,
args.toArray(String[]::new));
    return dir;
}
```


© TechTribers. For personal use only. Do not redistribute.

```
String src = """
        public class PriceApi {
            public long price(String sku, int quantity, boolean member) { return 0; }
            public interface Lookup { String find(String region, long customerId); }
            public record Quote(String sku, long cents) { }
        }
        """;
for (String[] opts : List.of(new String[]{}, new String[]{"-g"}, new String[]{"-parameters"})) {
    try (URLClassLoader cl = new URLClassLoader(new URL[]{compileTo(src, opts).toUri().toURL()},
null)) {
        Method price = cl.loadClass("PriceApi").getMethod("price", String.class, int.class,
boolean.class);
        Method find = cl.loadClass("PriceApi$Lookup").getMethod("find", String.class, long.class);
        Constructor<?> quote = cl.loadClass("PriceApi$Quote").getDeclaredConstructors()[0];
        System.out.printf("%-12s method %s present=%b | interface %s | record ctor %s%n",
opts.length == 0 ? "(none)" : opts[0],
                Arrays.stream(price.getParameters()).map(Parameter::getName).toList(),
price.getParameters()[0].isNamePresent(),
                Arrays.stream(find.getParameters()).map(Parameter::getName).toList(),
                Arrays.stream(quote.getParameters()).map(Parameter::getName).toList());
    }
}
```

▶ Output

```
(none)       method [arg0, arg1, arg2] present=false | interface [arg0, arg1] | record ctor [sku,
cents]
-g           method [arg0, arg1, arg2] present=false | interface [arg0, arg1] | record ctor [sku,
cents]
-parameters  method [sku, quantity, member] present=true | interface [region, customerId] | record
ctor [sku, cents]
```

⚠️ Interview trap

Upgrading to Spring 6.1+ / Boot 3.2+ in a project that doesn't use the Boot parent POM or plugin, and getting `IllegalArgumentException: Name for argument of type [long] not specified, and parameter name information not available via reflection`. The fix is `-parameters`, or explicit names in the annotations.

© TechTribers. For personal use only. Do not redistribute.

Series progress

799 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 17 starts at Q800.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50
Part 6	Q255–Q304	50
Part 7	Q305–Q339	35
Part 8	Q340–Q372	33
Part 9	Q373–Q397	25
Part 10	Q398–Q427	30
Part 11	Q428–Q479	52
Part 12	Q480–Q529	50
Part 13	Q530–Q579	50
Part 14	Q580–Q679	100
Part 15	Q680–Q739	60
Part 16	Q740–Q799	60

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

TechTribers TechTribers

© TechTribers. For personal use only. Do not redistribute.